Table of Contents

- Introduction & Dataset analysis
  - Feature Overview
  - Statistical Summary
  - Data Types
  - Dataset Insights and Feature Types
- Data Treatment
  - Missing Values
  - Alert missing values imputation strategy
    - Location missing values imputation strategy
    - Country missing values imputation strategy
    - Continent missing values imputation strategy
  - Missing data findings and reasolutions
  - Encoding Categorical Features and Text Features
  - Further data visualization and issue identification
  - Scaling our features
    - Data Splitting
  - Feature Reduction, analysis and selection
    - Feature engineering
    - Feature Importance Ranking (SHAP and GINI combined) along with Domain Relevance
  - An imbalnced dataset
    - Dataset imbalance handling techniques discussion
- Data limitation and quality issues
- Ethics and Privacy Considerations
- Model Selection Rationale
  - Model 1: Logistic Regression with Class Weights
  - Model 2: Gradient Boosting classifier
- Research questions
- Model Training
  - Model 1: Logistic Regression
    - Hyperparameter Tuning for Logistic Regression
    - Exploring C (regularization strength)
    - Further Hyperparameter Tuning for general performance improvement (max iterations and tolerance)
    - Class weight tuning
    - Hyperparameter Optimization and Visualization
    - Grid Search for Hyperparameter Optimization
  - Model 2: Gradient Boosting Classifier
    - Hyperparameter Tuning for Gradient Boosting Classifier
    - Tree depth and number of estimators tuning
    - Hyperparameter Optimization and Visualization
    - Grid Search for Hyperparameter Optimization
- Results and Evaluation
  - Training, Validation, and Test Performance comparison
  - Precision, Recall, and F1-Score comparison
  - ROC and AUC performance comparison
  - Confusion Matrix Analysis
  - Statistical Significance Testing
  - Model comparison summary
  - Research questions
- Novelty
- Reflection
- References
- Appendix
  - AI Usage Disclosure

https://www.kaggle.com/datasets/warcoder/earthquake-dataset?select=earthquake_1995-2023.csv

In [447…
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import missingno as msno
import seaborn as sns
```

Introduction and Dataset Overview

This dataset was compiled by Chirag Chauhan and uploaded to Kaggle. It contains data on 782 significant earthquakes worldwide from 2001 to 2022. The dataset contains various features related to earthquakes. The data origninates from the United States Geological Survey (USGS) earthquake everywhere API (Chauhan, 2024). There are various real world applications for this dataset, including earthquake analysis, tsunami risk prediction, and seismic hazard assessment.

My objective is to build a model that utilises this dataset to predict the occurrence of tsunamis following earthquakes based on the characteristics of the seismic events, something this dataset is well suited for given its selection of features. This has been something of interest for the seismology community, the topic area the dataset belongs to for sometime, with various studies having been conducted on the relationship between earthquake characteristics and tsunami generation (Lomax, Michelini and Piatanesi, 2011). If successful, this model could potentially be used to enhance tsunami warning systems, providing timely alerts to vulnerable coastal regions and reducing the impact of tsunamis on human lives and infrastructure.

Feature Overview

Column	Description
title	Name given to earthquake
magnitude	Magnitude of earthquake
date_time	Date and time
cdi	Maximum reported intensity for event range
mmi	Maximum estimated instrumental intensity for event
alert	Alert level, “green”, “yellow”, “orange”, and “red”
tsunami	"1" for events in oceanic regions and "0" otherwise
sig	Number describing how significant the event is. Larger numbers indicate a more significant event. This value is determined by magnitude, maximum MMI, felt reports, and estimated impact
net	ID of a data contributor. Identifies the network considered to be the preferred source of information for this event.
nst	Total number of seismic stations used to determine earthquake location.
dmin	Horizontal distance from the epicenter to the nearest station
gap	Largest azimuthal gap between azimuthally adjacent stations (in degrees). Smaller numbers indicate more reliable positions. Gaps >180° have large uncertainties
magType	Method or algorithm used to calculate the preferred magnitude for the event
depth	Depth where the earthquake begins to rupture
latitude	Latitude coordinate of the earthquake
longitude	Longitude coordinate of the earthquake
location	Location within the country
continent	Continent of the earthquake-hit country
country	Affected country

(figure 1)

(Chauhan, 2024)

In [448…
```python
df = pd.read_csv('data/earthquake_data.csv')
```

In [449…
```python
df.hist(bins=15, figsize=(15,10), layout=(4,4))
plt.tight_layout()
plt.show()
```

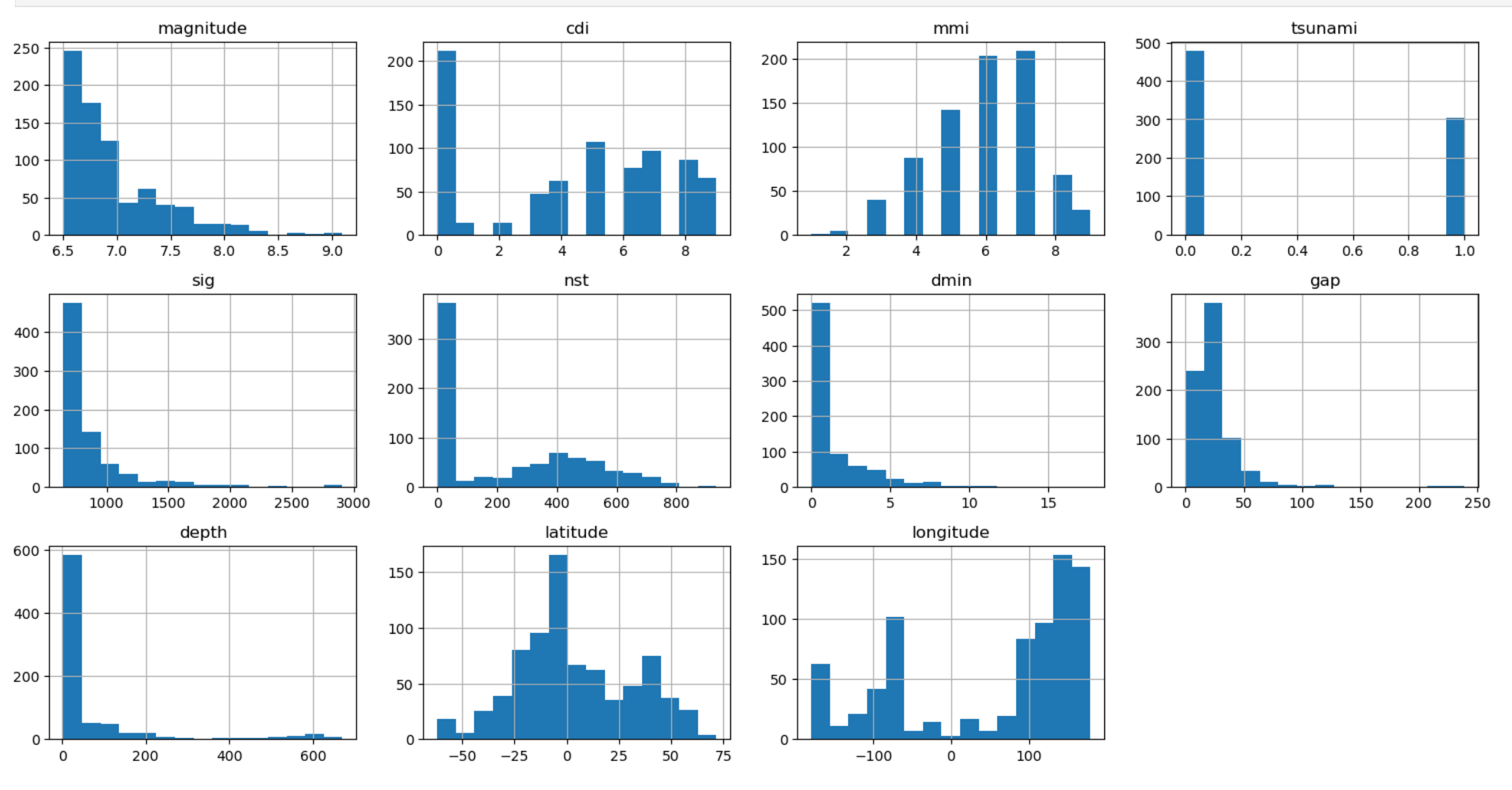


Figure 2: Histograms of Numerical Features

In [450…
```python
df.sample(16)
```

Out[450…

	title	magnitude	date_time	cdi	mmi	alert	tsunami	sig	net	nst	dmin	gap	magType	depth	latitude	longitude	location	continent	country
233	M 6.5 - 92 km SSW of Blenheim, New Zealand	6.5	13-11-2016 11:32	6	7	yellow	1	656	us	0	0.135	58.0	mww	10.00	-42.3205	173.6690	Blenheim, New Zealand	NaN	New Zealand
597	M 8.0 - 41 km SW of San Vicente de Cañete, Peru	8.0	15-08-2007 23:40	9	8	NaN	0	1385	us	625	0.000	25.7	mwc	39.00	-13.3860	-76.6030	San Vicente de Cañete, Peru	NaN	Peru
188	M 6.6 - 74 km E of Tadine, New Caledonia	6.6	19-11-2017 15:09	4	5	green	1	676	us	0	0.529	21.0	mww	13.00	-21.5027	168.5980	Tadine, New Caledonia	NaN	NaN
394	M 7.4 - 292 km SW of Vaini, Tonga	7.4	23-05-2013 17:19	0	6	green	1	842	us	626	0.000	16.0	mww	173.70	-23.0090	-177.2320	Vaini, Tonga	NaN	NaN
84	M 7.6 - 99 km SE of Sand Point, Alaska	7.6	19-10-2020 20:54	9	7	yellow	1	1165	us	0	0.230	36.0	mww	28.37	54.6620	-159.6750	Sand Point, Alaska	North America	United States of America
719	M 6.6 - 87 km SSE of Kokopo, Papua New Guinea	6.6	07-06-2003 00:32	0	6	NaN	0	670	us	419	0.000	27.8	mwc	33.00	-5.0950	152.5020	Kokopo, Papua New Guinea	NaN	Papua New Guinea
267	M 6.7 - 125 km SSW of Tarauacá, Brazil	6.7	26-11-2015 05:45	2	3	green	0	691	us	0	2.033	19.0	mww	602.75	-9.1825	-71.2574	Tarauacá, Brazil	South America	Brazil
316	M 6.5 - 236 km S of Punta de Burica, Panama	6.5	07-01-2015 05:07	1	4	green	1	650	us	0	4.318	61.0	mww	8.00	5.9045	-82.6576	Punta de Burica, Panama	NaN	NaN
641	M 6.5 - 57 km SE of Sinabang, Indonesia	6.5	19-11-2005 14:10	3	6	NaN	0	651	us	358	0.000	20.7	mwb	21.00	2.1640	96.7860	Sinabang, Indonesia	NaN	Indonesia
53	M 7.2 - Nippes, Haiti	7.2	14-08-2021 12:29	9	8	red	1	2331	us	0	1.178	23.0	mww	10.00	18.3521	-73.4804	Nippes, Haiti	North America	Haiti
220	M 6.9 - 225 km SW of Nadi, Fiji	6.9	03-01-2017 21:52	6	4	green	1	742	us	0	2.495	13.0	mww	12.00	-19.3733	176.0520	Nadi, Fiji	NaN	NaN
428	M 7.6 - 11 km ENE of Hojancha, Costa Rica	7.6	05-09-2012 14:42	8	7	NaN	0	1555	us	703	0.000	34.3	mww	35.00	10.0850	-85.3150	Hojancha, Costa Rica	NaN	Costa Rica
261	M 6.7 - 52 km SE of Shizunai-furukawach?, Japan	6.7	14-01-2016 03:25	5	6	green	1	717	us	0	0.281	22.0	mww	46.00	41.9723	142.7810	Shizunai-furukawach?, Japan	Asia	Japan
395	M 6.8 - Pagan region, Northern Mariana Islands	6.8	14-05-2013 00:32	0	3	green	0	711	us	131	0.000	18.0	mww	602.20	18.7280	145.2880	Pagan region, Northern Mariana Islands	NaN	NaN
79	M 6.9 - South Shetland Islands	6.9	23-01-2021 23:36	9	5	green	1	739	us	0	1.550	16.0	mww	15.00	-61.8484	-55.5590	South Shetland Islands	NaN	Antarctica
419	M 7.1 - Banda Sea	7.1	10-12-2012 16:53	5	5	NaN	0	843	us	517	0.000	8.7	mww	155.00	-6.5330	129.8250	Banda Sea	NaN	Indonesia

Statistical Summary

In [451…
```python
df.describe()
```

Out[451…

	magnitude	cdi	mmi	tsunami	sig	nst	dmin	gap	depth	latitude	longitude
count	782.000000	782.000000	782.000000	782.000000	782.000000	782.000000	782.000000	782.000000	782.000000	782.000000	782.000000
mean	6.941125	4.333760	5.964194	0.388747	870.108696	230.250639	1.325757	25.038990	75.883199	3.538100	52.609199
std	0.445514	3.169939	1.462724	0.487778	322.465367	250.188177	2.218805	24.225067	137.277078	27.303429	117.898886
min	6.500000	0.000000	1.000000	0.000000	650.000000	0.000000	0.000000	0.000000	2.700000	-61.848400	-179.968000
25%	6.600000	0.000000	5.000000	0.000000	691.000000	0.000000	0.000000	14.625000	14.000000	-14.595600	-71.668050
50%	6.800000	5.000000	6.000000	0.000000	754.000000	140.000000	0.000000	20.000000	26.295000	-2.572500	109.426000
75%	7.100000	7.000000	7.000000	1.000000	909.750000	445.000000	1.863000	30.000000	49.750000	24.654500	148.941000
max	9.100000	9.000000	9.000000	1.000000	2910.000000	934.000000	17.654000	239.000000	670.810000	71.631200	179.662000

In [452…
```python
print(f'Dataset Shape: {df.shape}')
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")
print('Column Names:', df.columns.tolist())
```

```
Dataset Shape: (782, 19)
Rows: 782, Columns: 19
Column Names: ['title', 'magnitude', 'date_time', 'cdi', 'mmi', 'alert', 'tsunami', 'sig', 'net', 'nst', 'dmin', 'gap', 'magType', 'depth', 'latitude', 'longitude', 'location', 'continent', 'country']
```

Data Types

In [453…
```python
unique_values = df.nunique()
print(f'Unique values per column:\n{unique_values}')
country_unique_variables_before_imputation = df['country'].nunique()
```

```
Unique values per column:
title        768
magnitude     24
date_time    773
cdi           10
mmi            9
alert          4
tsunami        2
sig          339
net           11
nst          312
dmin         369
gap          256
magType        9
depth        303
latitude     778
longitude    777
location     413
continent      6
country       49
dtype: int64
```

In [454…
```python
df.nunique(axis=0).plot(kind='bar')
plt.title('Unique Value Counts per Column')
plt.xlabel('Columns')
plt.ylabel('Unique Value Counts')
plt.show()
```

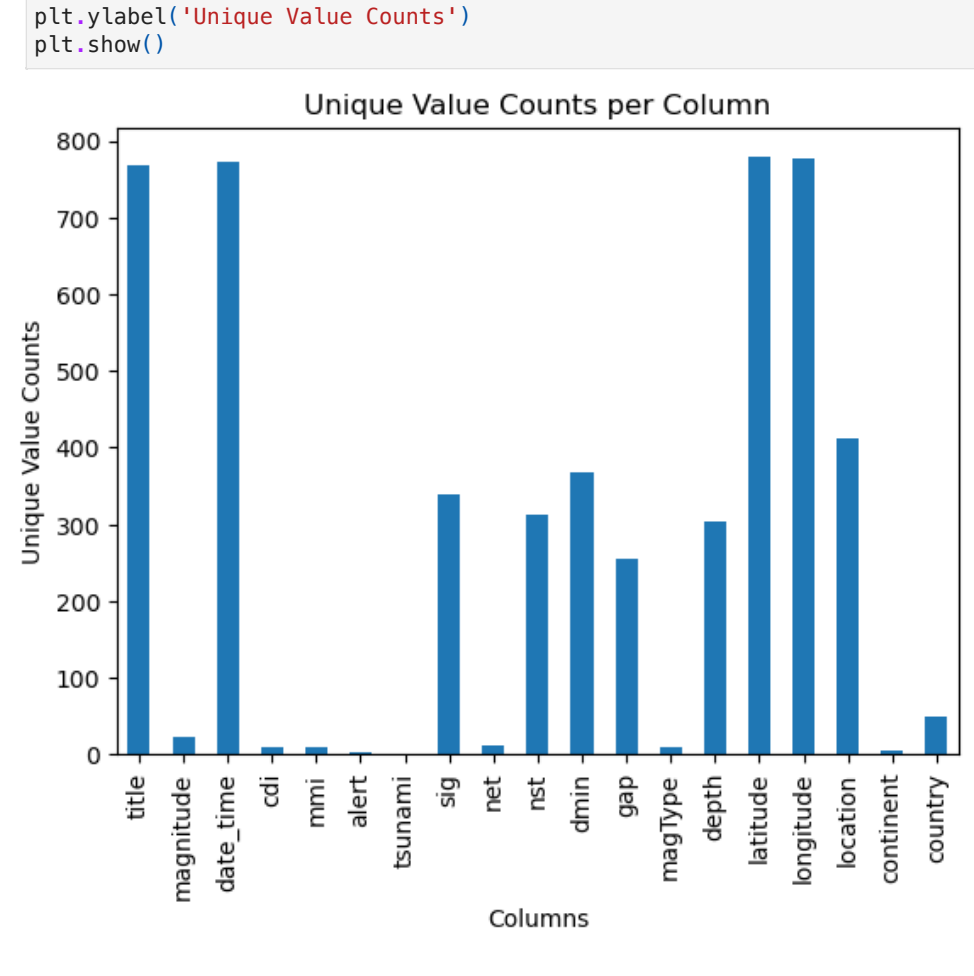


Figure 3: This chart displays the number of unique values in each dataset column, helping distinguish categorical features (with fewer unique values) from numerical features (with many).

In [455…
```python
print(df.dtypes)
```

```
title         object
magnitude    float64
date_time     object
cdi            int64
mmi            int64
alert         object
tsunami        int64
sig            int64
net           object
nst            int64
dmin         float64
gap          float64
magType       object
depth        float64
latitude     float64
longitude    float64
location      object
continent     object
country       object
dtype: object
```

Dataset Insights and Feature Types

To understand the dataset, a statistical summary and data type audit was conducted. These revealed that many numerical variables, particularly those related to earthquake magnitude and impact, exhibit strong left skewed distributions (see Figures 1 and 2). This indicates that most recorded earthquakes are low magnitude, low impact events, with relatively few high magnitude observations. Such imbalance may bias models toward predicting lower impact earthquakes.

Figure 3 and the data-type overview show that the dataset contains both numerical and categorical features, requiring tailored preprocessing. According to the USGS, `mmi` and `cdi` are discrete intensity scales, whereas `magnitude` represents a continuous measure of energy release; therefore, `magnitude` will be treated as numerical and cdi/mmi as ordered categorical variables (United States Geological Survey, no date; Wald et al., 2011; United States Geological Survey, 1989). `title` and `location` are unstructured text features due to their object types and high uniqueness, while `date_time` is a datetime variable.

The target variable, `tsunami`, is a binary numerical feature with two values (0 and 1), indicating whether a tsunami followed the earthquake. As noted in Chauhan (2024), this makes it well aligned with the with the goal of predicting tsunami occurrence via binary classification models.

Types of data	Features
Numerical Features	`magnitude`, `sig`, `nst`, `dmin`, `gap`, `depth`, `latitude`, `longitude`
Categorical Numerical Features	`cdi`, `mmi`
Datetime Feature	`date_time`
Categorical Text Feature	`alert`, `net`, `magType`, `continent`, `country`
Unstructured Text Feature	`title`, `location`
Target Variable (Binary Numerical)	`tsunami`

Figure 4: Data Types of Features and Target Variable

Data Treatment

Missing Values

In [456…
```python
print(df.isnull().sum())
```

```
title          0
magnitude      0
date_time      0
cdi            0
mmi            0
alert        367
tsunami        0
sig            0
net            0
nst            0
dmin           0
gap            0
magType        0
depth          0
latitude       0
longitude      0
location       5
continent    576
country      298
dtype: int64
```

In [457…
```python
place_holders = [-9999, 9999, np.inf, -np.inf, 'nan', 'NaN', 'NA', 'null', 'NULL', '']
print(df.isin(place_holders).sum().sum() + df.isnull().sum().sum())
```

```
1246
```

In [ ]:
```python
missing_percentage = (df.isnull().sum() / len(df)) * 100
missing_percentage = missing_percentage[missing_percentage > 0]
print(f'Percentage of missing values per column:\n{missing_percentage}')
```

```
Percentage of missing values per column:
alert        46.930946
location      0.639386
continent    73.657289
country      38.107417
dtype: float64
```

In [459…
```python
missing_values_datatypes = df[missing_percentage.index].dtypes
print(f'Data types of columns with missing values:\n{missing_values_datatypes}')
```

```
Data types of columns with missing values:
alert        object
location     object
continent    object
country      object
dtype: object
```

In [460…
```python
df[missing_percentage.index].sample(16)
```

Out[460…

	alert	location	continent	country
334	green	Mata-Utu, Wallis and Futuna	NaN	NaN
433	NaN	Kokopo, Papua New Guinea	NaN	Papua New Guinea
134	green	Azángaro, Peru	South America	Peru
115	green	Constitución, Chile	NaN	NaN
302	green	Panguna, Papua New Guinea	NaN	NaN
606	NaN	Gizo, Solomon Islands	NaN	Solomon Islands
398	green	Kuril'sk, Russia	Asia	Russia
154	green	Tadine, New Caledonia	NaN	NaN
688	NaN	Manokwari, Indonesia	NaN	Indonesia
377	green	Atiquipa, Peru	NaN	Peru
338	green	Kermadec Islands, New Zealand	NaN	NaN
310	green	Hihifo, Tonga	NaN	NaN
289	green	Abepura, Indonesia	NaN	Indonesia
352	green	Panguna, Papua New Guinea	NaN	Papua New Guinea
74	green	Kermadec Islands, New Zealand	NaN	NaN
321	green	Tobelo, Indonesia	NaN	Indonesia

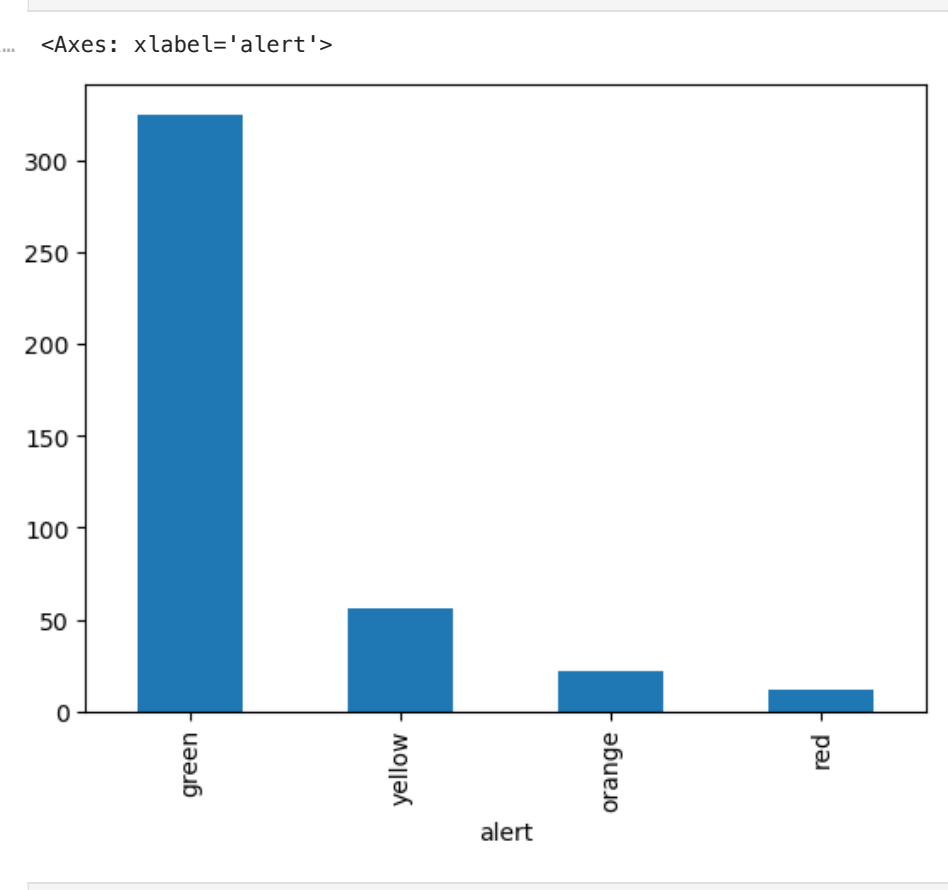
Feature	Type of Missingness	Reason / Explanation
Alert Level	Missing Not At Random (MNAR)	Missingness is related to the unobserved value itself, less significant earthquakes are less likely to have an alert level assigned.
Location	Missing At Random (MAR)	Missingness depends on other observed variables (latitude and longitude). Location may not be recorded because coordinates are already available.
Country	Missing At Random (MAR)	Missingness is related to latitude and longitude data, the country may be inferred from coordinates, making direct recording unnecessary.
Continent	Missing At Random (MAR)	Missingness depends on other observed information (such as latitude, longitude, or country) and is not directly related to the continent value itself.

Figure 10: Missingness Types for Features with Missing Data

Alert missing values imputation strategy

In [461…
```python
df['alert'].value_counts().plot(kind='bar')
```

Out[461…
```
<Axes: xlabel='alert'>
```



In [462…
```python
alert_order = {'green': 1, 'yellow': 2, 'orange': 3, 'red': 4}
df_temp = df.copy()
df_temp['alert_encoded'] = df_temp['alert'].map(alert_order)
numerical_cols = df_temp.select_dtypes(include=[np.number]).columns.tolist()
corr_matrix = df_temp[numerical_cols].corr()
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, annot=True)
plt.title('Correlation Matrix')
plt.show()
```

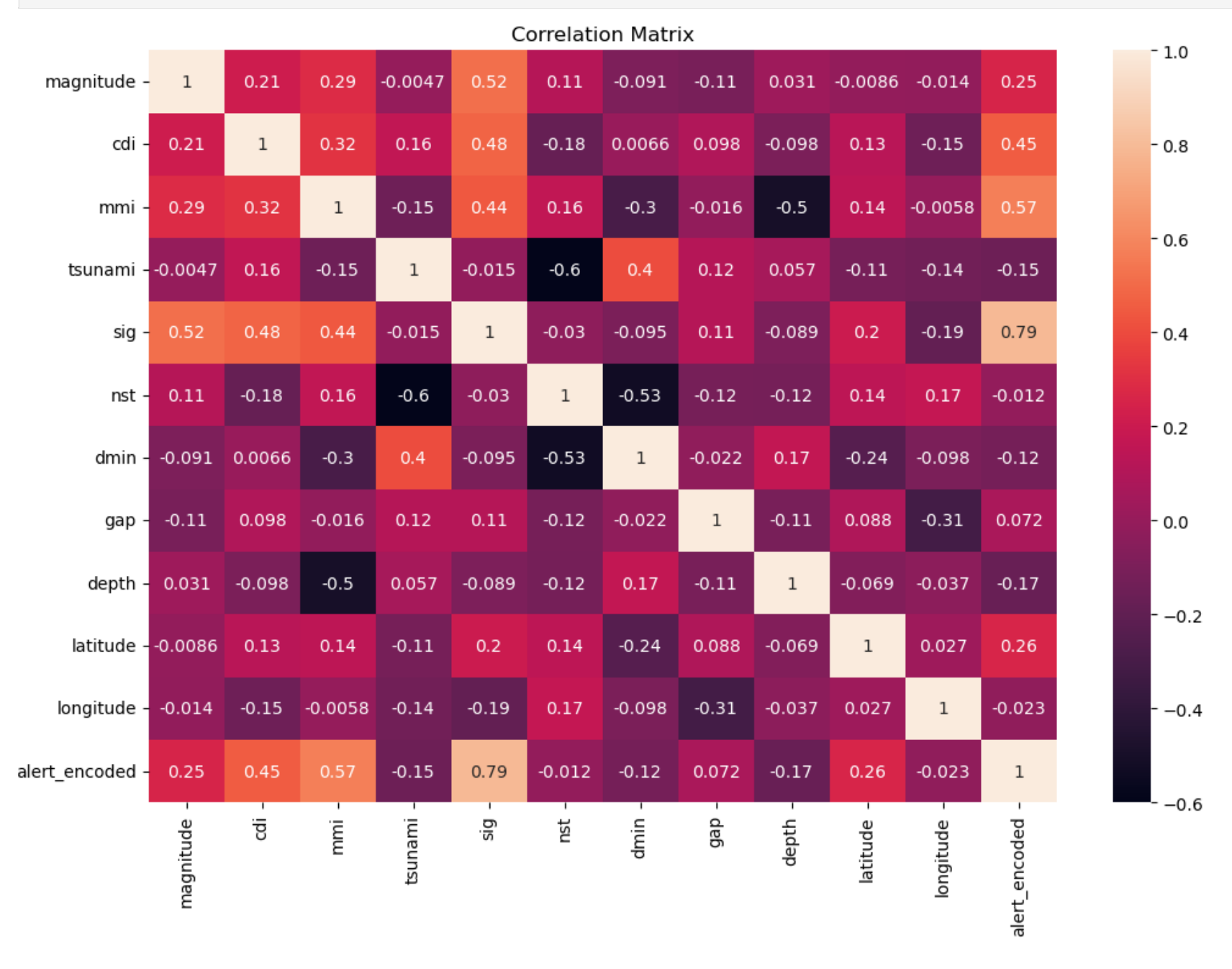


Figure 6: Correlation between Numerical Features and Encoded Alert Levels

In [463…
```python
print(df.groupby('alert').agg({
    'sig': ['mean']
}).sig)
```

```
               mean
alert
green    773.224615
orange  1382.136364
red     2475.000000
yellow  1047.696429
```

In [464…
```python
alert_thresholds = df.groupby('alert').agg({
    'sig': ['mean']
}).sig

green_threshold = alert_thresholds.loc['green', 'mean'] #index by alert level, get column 'mean'
yellow_threshold = alert_thresholds.loc['yellow', 'mean']
orange_threshold = alert_thresholds.loc['orange', 'mean']
red_threshold = alert_thresholds.loc['red', 'mean']

#Order of alert levels: red, orange, yellow, green - source: (United States Geological Survey, 2010)

def impute_alert(row):
    if pd.isnull(row['alert']):
        if row['sig'] >= red_threshold:
            return 'red'
        elif row['sig'] >= orange_threshold:
            return 'orange'
        elif row['sig'] >= yellow_threshold:
            return 'yellow'
        else:
            return 'green'
    else:
        return row['alert']
df['alert'] = df.apply(impute_alert, axis=1)
print(f'Missing values in alert column after imputation: {df["alert"].isnull().sum()}')
```

```
Missing values in alert column after imputation: 0
```

Location missing values imputation strategy

In [465…
```python
msno.matrix(df)
```

Out[465…
```
<Axes: >
```

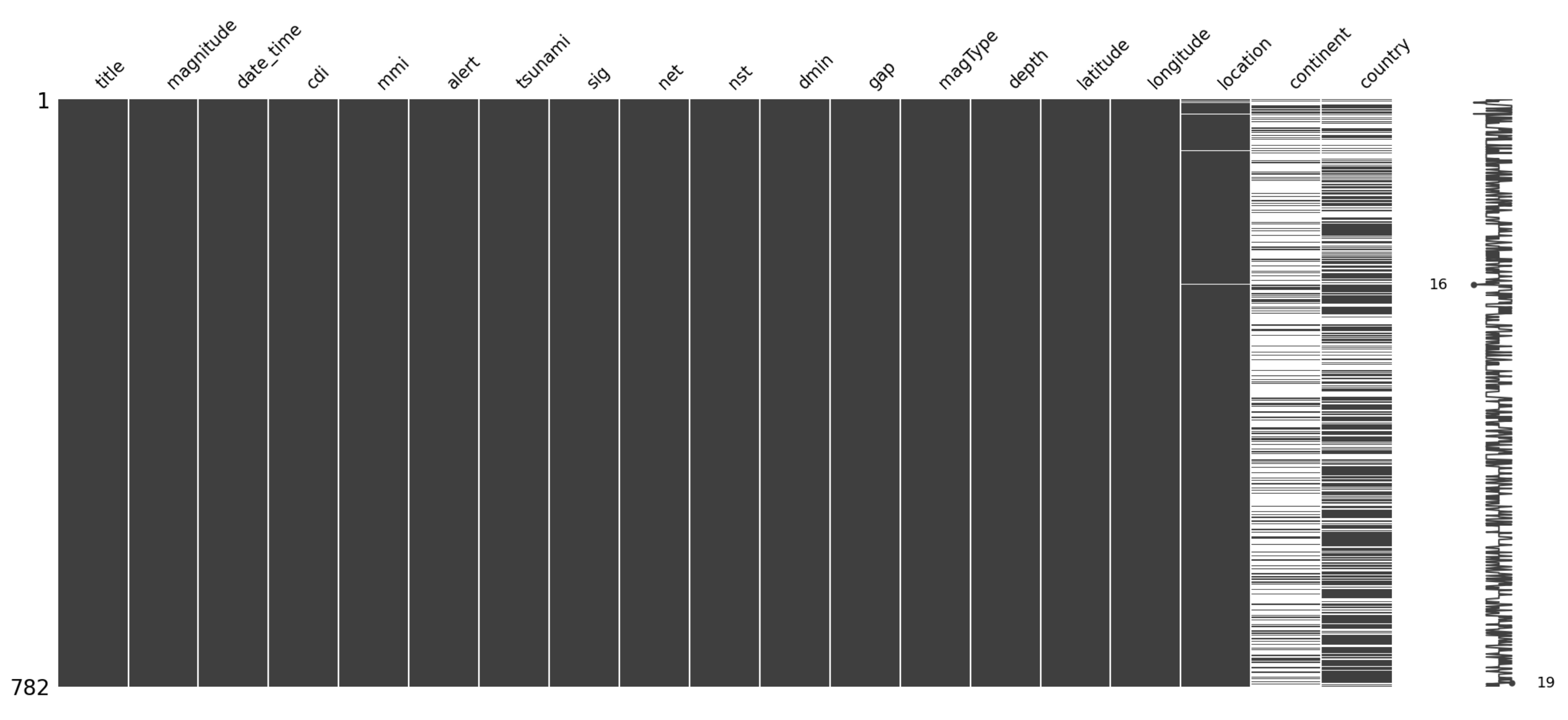


Figure 7: Missingness Pattern for All Features

In [466…
```python
print(f'Missing values in location column before imputation: {df["location"].isnull().sum()}')
def impute_location(row):
    if pd.isnull(row['location']):
        if not pd.isnull(row['country']):
            return f'{row["country"]}'
        else:
            return np.nan # cannot impute location
    else:
        return row['location']
df['location'] = df.apply(impute_location, axis=1)
print(f'Missing values in location column after imputation: {df["location"].isnull().sum()}')
```

```
Missing values in location column before imputation: 5
Missing values in location column after imputation: 3
```

In [467…
```python
df = df.dropna(subset=['location'])
print('Data shape after dropping rows with missing location:', df.shape)
```

```
Data shape after dropping rows with missing location: (779, 19)
```

In [468…
```python
print(df.location.sample(15))
```

```
53                    Nippes, Haiti
353                 Iquique, Chile
559                   Namie, Japan
191       Angoram, Papua New Guinea
225       Kirakira, Solomon Islands
610              Ternate, Indonesia
624         Kimbe, Papua New Guinea
343                  Vernon, Canada
27     Kermadec Islands, New Zealand
644        Kokopo, Papua New Guinea
15             the Kermadec Islands
253          Ashk?sham, Afghanistan
90              Bengkulu, Indonesia
601                   Mikuni, Japan
377                  Atiquipa, Peru
Name: location, dtype: object
```

Country missing values imputation strategy

In [469…
```python
coma_count = df['location'].str.contains(',').sum()
print(f'Percentage of locations with a comma: {(coma_count / len(df)) * 100:.2f}%')
```

```
Percentage of locations with a comma: 91.40%
```

Figure 18: Many location entries include an area–country format (e.g., "Los Angeles, USA"), enabling missing country values to be inferred.

In [470…
```python
print(f'Missing values in country column before imputation: {df["country"].isnull().sum()}')
def impute_country(row):
    if pd.isnull(row['country']):
        if ',' in row['location']:
            return row['location'].split(',')[-1].strip() # extract country from location using the last part after comma
        else:
            return np.nan # cannot impute country
    else:
        return row['country']

df['country'] = df.apply(impute_country, axis=1)
print(f'Missing values in country column after imputation: {df["country"].isnull().sum()}')
```

```
Missing values in country column before imputation: 295
Missing values in country column after imputation: 42
```

In [471…
```python
missing_country_locations = df[df['country'].isnull()]['location']
print(f'Locations with missing country values:\n{missing_country_locations}')
```

```
Locations with missing country values:
5                          the Fiji Islands
6                          the Fiji Islands
15                     the Kermadec Islands
22                      the Loyalty Islands
28                         the Fiji Islands
30                  Kermadec Islands region
45                           Vanuatu region
46                           Vanuatu region
52            South Sandwich Islands region
56            South Sandwich Islands region
57            South Sandwich Islands region
67               Mauritius - Reunion region
73                  Kermadec Islands region
78                      the Loyalty Islands
85               central Mid-Atlantic Ridge
88               central Mid-Atlantic Ridge
97                     the Kermadec Islands
137            Prince Edward Islands region
160                        the Fiji Islands
161                 Kermadec Islands region
186                    Bouvet Island region
208           South Sandwich Islands region
214                        the Fiji Islands
242                     the Loyalty Islands
245           South Sandwich Islands region
325                             Fiji region
329                       Micronesia region
335           South Sandwich Islands region
363                    the Kermadec Islands
365                             Fiji region
374                                 Okhotsk
389             northern Mid-Atlantic Ridge
392                                 Okhotsk
393                                 Okhotsk
440    off the west coast of northern Sumatra
441    off the west coast of northern Sumatra
448    off the west coast of northern Sumatra
464                 Kermadec Islands region
566              Philippine Islands region
611                      the Kuril Islands
614                          Kuril Islands
669                        Macquarie Island
Name: location, dtype: object
```

Many of the 42 samples lacking country data correspond to small islands, territories, ocean regions, or disputed areas.

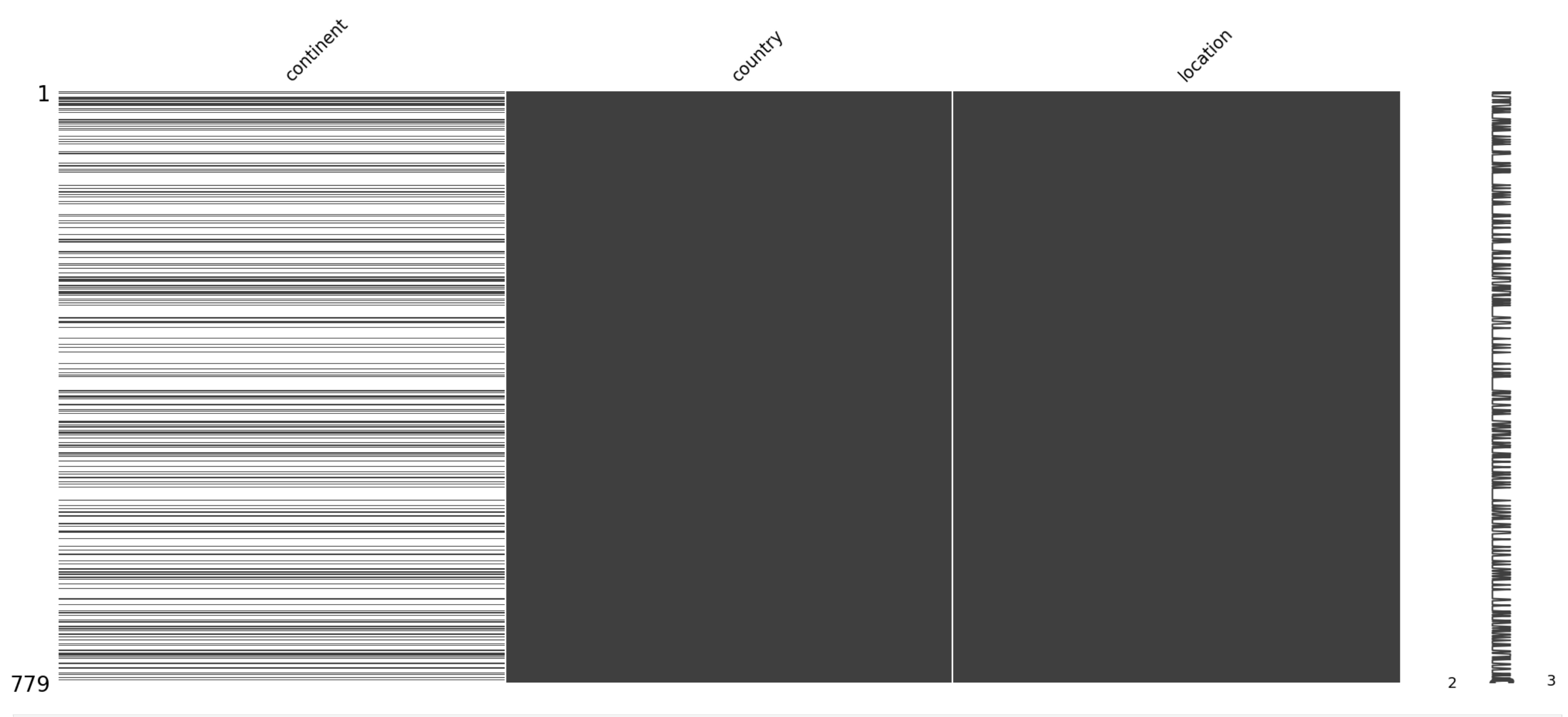
In [472…
```python
df['country'] = df['country'].fillna(df['location'])
print(f'Missing values in country column after final imputation: {df["country"].isnull().sum()}')
```

```
Missing values in country column after final imputation: 0
```

Continent missing values imputation strategy

In [473…
```python
msno.matrix(df[['continent', 'country', 'location']])
```

Out[473…
```
<Axes: >
```



In [474…
```python
df = df.drop(columns=['continent'])
print(df.columns)
```

```
Index(['title', 'magnitude', 'date_time', 'cdi', 'mmi', 'alert', 'tsunami',
       'sig', 'net', 'nst', 'dmin', 'gap', 'magType', 'depth', 'latitude',
       'longitude', 'location', 'country'],
      dtype='object')
```

In [475…
```python
total_missing_after = df.isnull().sum().sum()
print(f'Total missing values after imputation: {total_missing_after}')
```

```
Total missing values after imputation: 0
```

In [476…
```python
X, y = df.drop(columns='tsunami'), df['tsunami']
X.shape, y.shape, X.columns
```

Out[476…
```
((779, 17),
 (779,),
 Index(['title', 'magnitude', 'date_time', 'cdi', 'mmi', 'alert', 'sig', 'net',
        'nst', 'dmin', 'gap', 'magType', 'depth', 'latitude', 'longitude',
        'location', 'country'],
       dtype='object'))
```

Missing data findings and resolutions

I initially examined missing data patterns using both percentage based calculations and visualizations (Figure 7), identifying the extent and type of missingness across features (Figure 10). Imputation strategies were tailored accordingly: rule based imputation was applied to `alert` based on its correlation with `sig` (figure 6), while inter feature relationships guided imputation for `location` and `country`. The `continent` feature was dropped due to excessive missingness. These approaches aimed to preserve data integrity and minimize imputation bias.

Encoding Categorical Features and Text Features

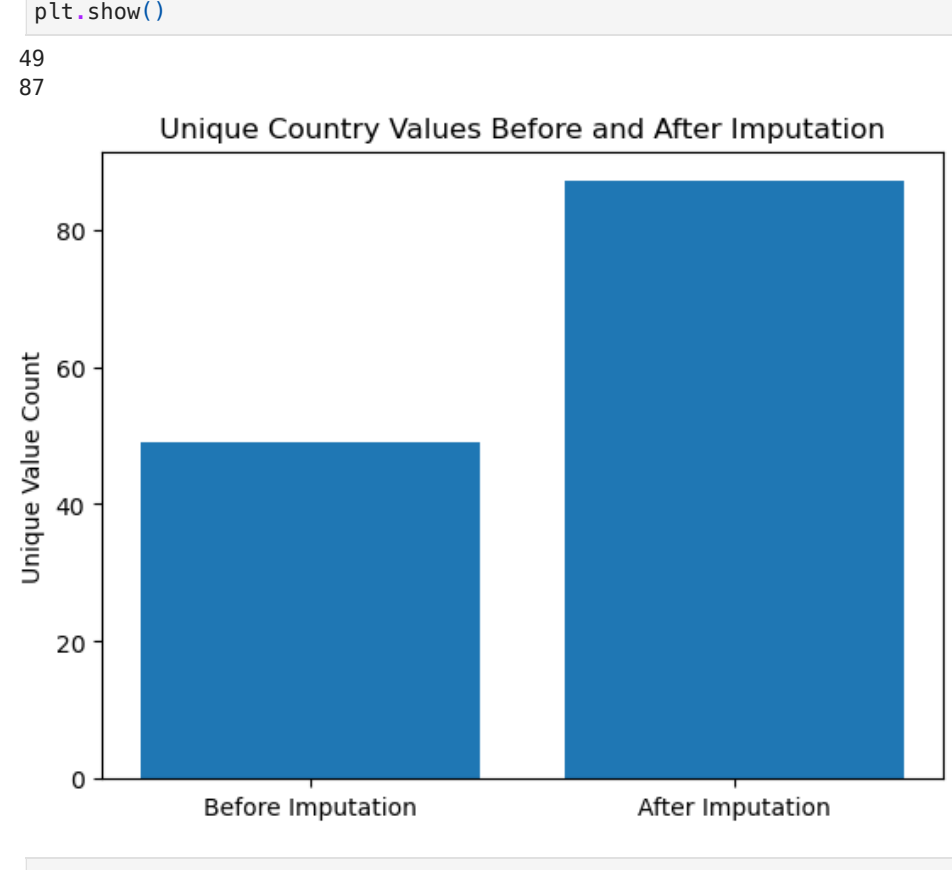
In [477…
```python
from sklearn.preprocessing import OrdinalEncoder
ordinal_features = ['alert']
ordinal_encoder = OrdinalEncoder(categories=[['green', 'yellow', 'orange', 'red']])
X[ordinal_features] = ordinal_encoder.fit_transform(X[ordinal_features])
```

I must also encode the remaining categorical features (`net`, `magType`, `country`, Figure 3). As these are nominal with no inherent order (Figure 1), I will apply one-hot encoding to convert each category into binary indicator variables.

In [478…
```python
#one-hot encoding of nominal categorical features
columns_before = X.shape[1]
nominal_features = ['net', 'magType']
X = pd.get_dummies(X, columns=nominal_features, drop_first=True)
columns_after = X.shape[1]
```

In [479…
```python
print(country_unique_variables_before_imputation)
print(df.country.nunique())
plt.bar(['Before Imputation', 'After Imputation'], [country_unique_variables_before_imputation, df.country.nunique()])
plt.title('Unique Country Values Before and After Imputation')
plt.ylabel('Unique Value Count')
plt.show()
```

```
49
87
```



In [480…
```python
X[['country', 'location']].sample(16)
```

Out[480…

	country	location
23	New Caledonia	Tadine, New Caledonia
130	Papua New Guinea	Namatanai, Papua New Guinea
525	Fiji	Ba, Fiji
443	Papua New Guinea	Kainantu, Papua New Guinea
326	El Salvador	Intipucá, El Salvador
681	Nicaragua	Masachapa, Nicaragua
378	Pakistan	Bela, Pakistan
414	Solomon Islands	Lata, Solomon Islands
59	Alaska	Perryville, Alaska
174	United States of America	Leilani Estates, Hawaii
613	Taiwan	Hengchun, Taiwan
563	Japan	Hasaki, Japan
76	New Zealand	Gisborne, New Zealand
272	Japan	Makurazaki, Japan
648	India region	Nicobar Islands, India region
471	Myanmar	Tachilek, Myanmar

figure 9: Location and Country content sample

In [481…
```python
X = X.drop(columns=['country'])
```

I now need to process the unstructured text features (`title` and `location`, Figure 4). As free-form text, they must be vectorised into numerical features before modelling. Since `Country` largely overlaps with `location` (Figures 9, 18), I keep only `location` to avoid redundancy and unnecessary dimensionality.

In [ ]:
```python
import gensim.downloader as api
word2vec_model = api.load('glove-wiki-gigaword-50') #pre-trained Word2Vec model with 50-dimensional vectors

def average_word_vectors(text, model): #Averaging vectors Inspired by (itdxer, 2017), novel approach for small datasets (keep dimensionality to 50)
    words = str(text).lower().split()
    vectors = []
    for word in words:
        if word in model: #When you iterate through model, you get the words in the vocabulary (it must have a custom __iter__ method)
            vectors.append(model[word])
    if vectors:
        return np.mean(vectors, axis=0)
    else:
        return np.zeros(model.vector_size)

title_vectors = X['title'].apply(lambda x: average_word_vectors(x, word2vec_model))
location_vectors = X['location'].apply(lambda x: average_word_vectors(x, word2vec_model))
title_df = pd.DataFrame(title_vectors.tolist(), index=X.index).add_prefix('title_vec_') #takes index from original X to align rows
location_df = pd.DataFrame(location_vectors.tolist(), index=X.index).add_prefix('location_vec_')
X = pd.concat([X, title_df, location_df], axis=1)
X = X.drop(columns=['title', 'location'])
X
```

Out[ ]:

	magnitude	date_time	cdi	mmi	alert	sig	nst	dmin	gap	depth	...	location_vec_40	location_vec_41	location_vec_42	location_vec_43	location_vec_44	location_vec_45	location_vec_46	location_vec_47	location_vec_48	location_vec_49
0	7.0	22-11-2022 02:03	8	7	0.0	768	117	0.509	17.0	14.0	...	-0.413625	1.011540	0.835310	-0.672395	-1.054485	0.468645	-0.376000	-0.737735	-0.421065	-0.443805
1	6.9	18-11-2022 13:37	4	4	0.0	735	99	2.229	34.0	25.0	...	-1.397400	0.328640	0.903000	-1.181100	0.236450	1.036200	-0.586330	0.309870	0.325110	-1.210100
2	7.0	12-11-2022 07:09	3	3	0.0	755	147	3.125	18.0	579.0	...	-0.678180	0.542310	0.366070	-0.350280	-0.664370	1.286700	-0.978370	0.141160	0.803460	-0.977450
3	7.3	11-11-2022 10:48	5	5	0.0	833	149	1.865	21.0	37.0	...	-0.083787	0.562320	0.343680	-0.498550	-0.811640	1.716400	-0.491980	0.052259	0.706380	-0.809750
5	7.0	09-11-2022 09:51	4	3	0.0	755	142	4.578	26.0	660.0	...	-0.508930	0.422533	0.538463	-0.348069	-0.877760	0.618010	-0.535445	-0.326447	0.172120	-0.906737
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
777	7.7	13-01-2001 17:33	0	8	0.0	912	427	0.000	0.0	60.0	...	-0.525199	0.588573	0.626350	-1.385473	-0.272928	-0.331966	-0.926685	-0.707303	0.474065	-0.023720
778	6.9	10-01-2001 16:02	5	7	0.0	745	0	0.000	0.0	36.4	...	-0.602380	0.546825	0.408775	0.057960	-0.438345	0.065940	-0.293385	-0.393100	-0.215465	-0.072315
779	7.1	09-01-2001 16:49	0	7	0.0	776	372	0.000	0.0	103.0	...	-0.660620	0.921090	0.486700	-1.248600	-1.001100	0.611220	-0.338700	0.311190	0.707830	-0.753600
780	6.8	01-01-2001 08:54	0	5	0.0	711	64	0.000	0.0	33.0	...	-1.447600	0.193520	0.699930	-1.508400	-0.045248	0.242790	-0.459550	-0.234900	0.175560	-0.527280
781	7.5	01-01-2001 06:57	0	7	0.0	865	324	0.000	0.0	33.0	...	-1.447600	0.193520	0.699930	-1.508400	-0.045248	0.242790	-0.459550	-0.234900	0.175560	-0.527280

779 rows × 130 columns

In [483…
```python
object_columns = X.select_dtypes(include=['object']).columns.tolist()
print(f'Object type columns in X: {object_columns}')
```

```
Object type columns in X: ['date_time']
```

In [484…
```python
X['time'] = pd.to_datetime(X['date_time'])
X['time'] = pd.to_datetime(X['time']).astype(int)/ 10**9 #unix timestamp - (AlwaysSunny, 2019)
X = X.drop(columns=['date_time'])
```

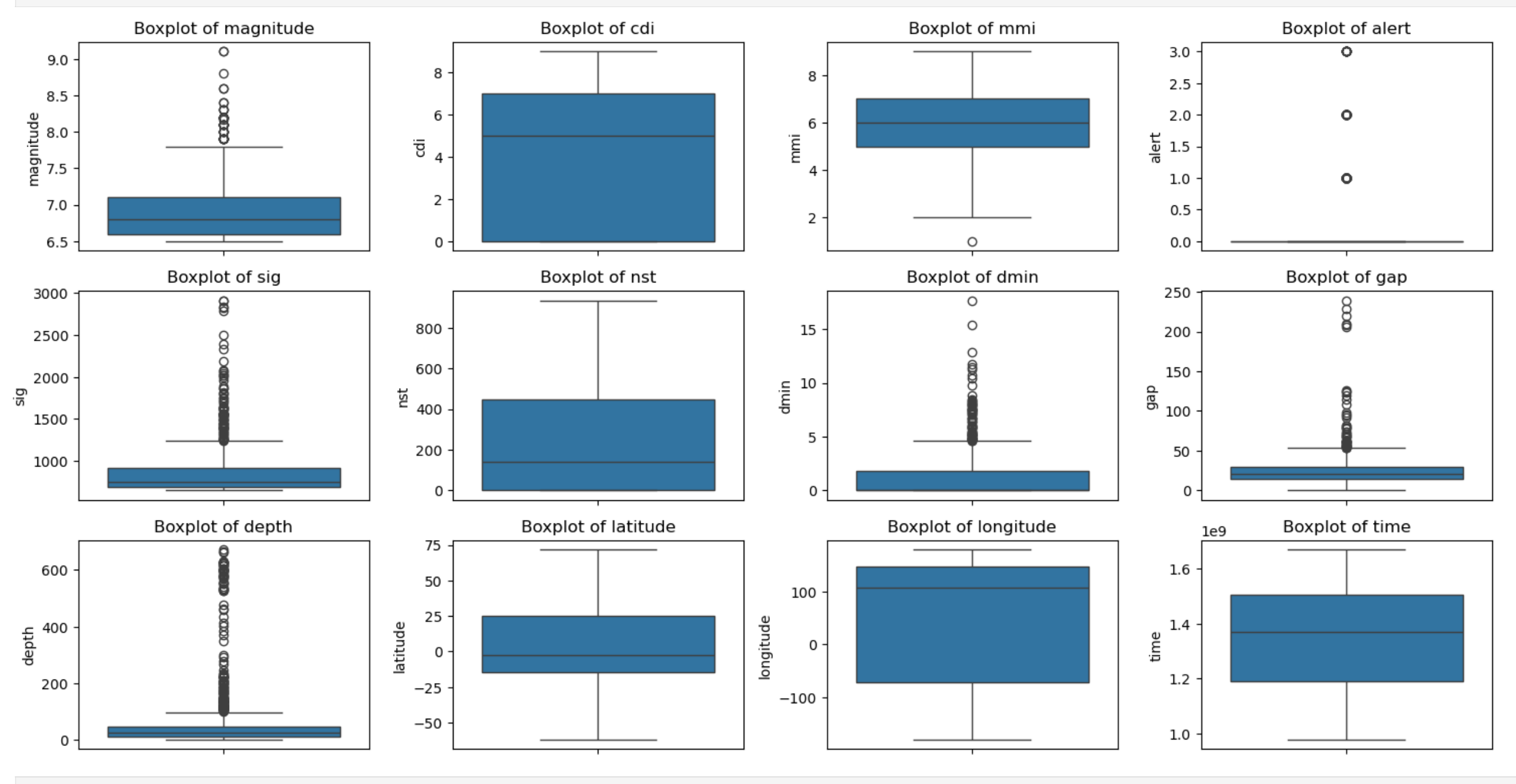
```
/var/folders/ch/bfx06mdj29ndjk1wx8vb4pz00000gn/T/ipykernel_55925/1096151919.py:1: UserWarning: Parsing dates in %d-%m-%Y %H:%M format when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specify a format to sil
ence this warning.
  X['time'] = pd.to_datetime(X['date_time'])
```

In [485…
```python
object_columns = X.select_dtypes(include=['object']).columns.tolist()
print(f'Object type columns in X: {object_columns}')
datatime_columns = X.select_dtypes(include=['datetime64[ns]']).columns.tolist()
print(f'Datetime type columns in X: {datatime_columns}')
```

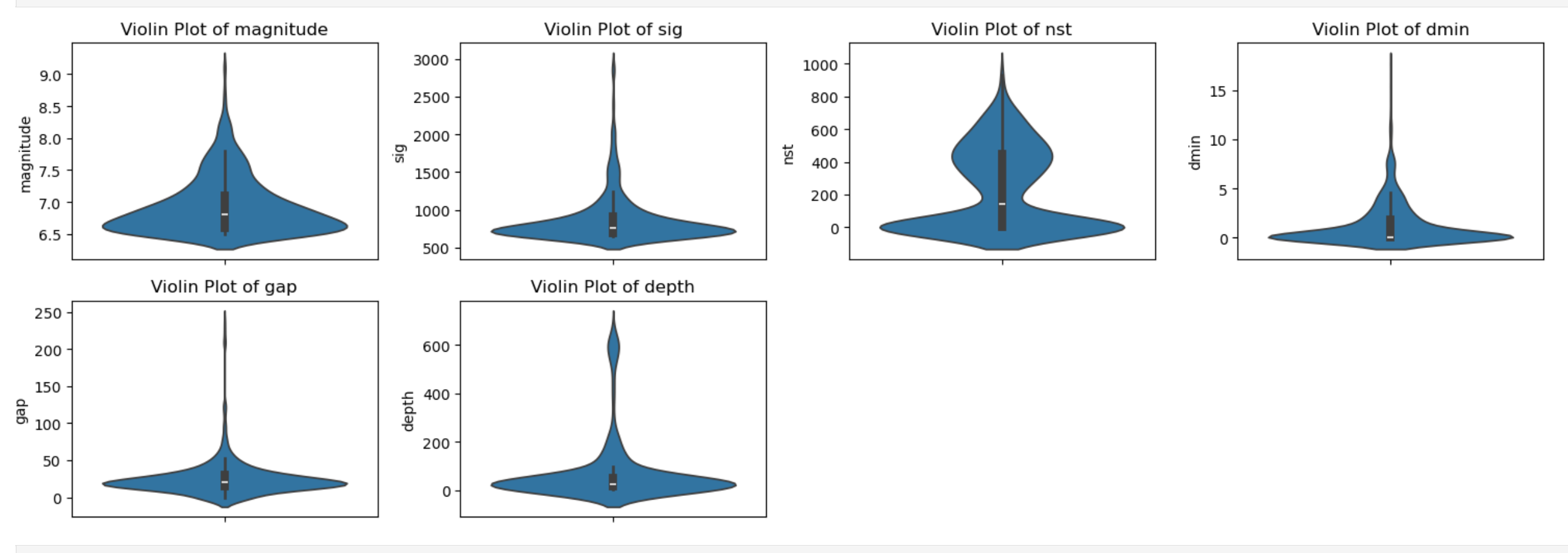
```
Object type columns in X: []
Datetime type columns in X: []
```

Further data visualization and issue identification

In [486…
```python
non_encoded_render = [c for c in X.columns if '_' not in c] #exclude vectorized text features for boxplots because they have many dimensions
plt.figure(figsize=(15,10))
for i, col in enumerate(non_encoded_render):
    plt.subplot(4, 4, i+1)
    sns.boxplot(y=X[col])
    plt.title(f'Boxplot of {col}')
plt.tight_layout()
plt.show()
```



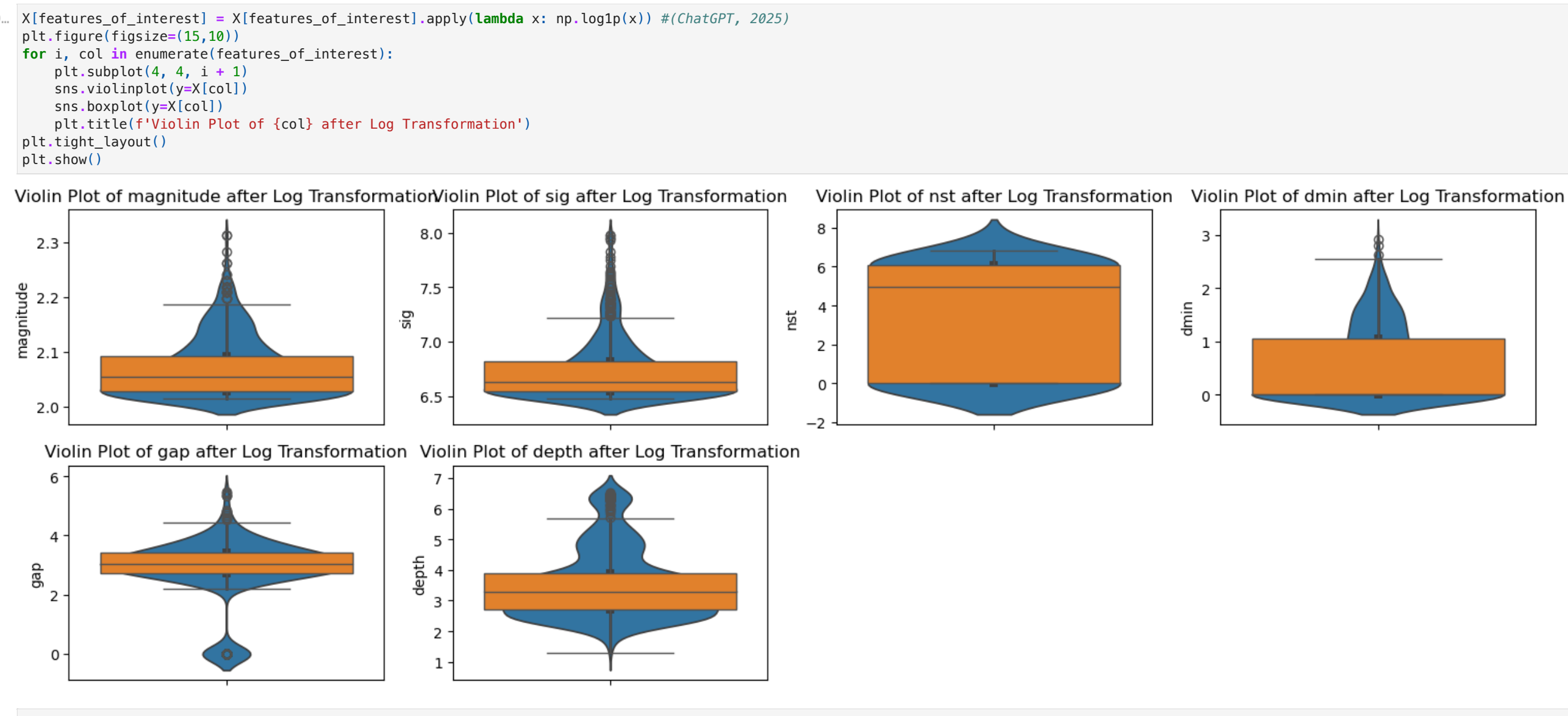
In [487…
```python
features_of_interest = ['magnitude', 'sig', 'nst', 'dmin', 'gap', 'depth']
plt.figure(figsize=(15,10))
for i, col in enumerate(features_of_interest):
    plt.subplot(4, 4, i + 1)
    sns.violinplot(y=X[col])
    plt.title(f'Violin Plot of {col}')
plt.tight_layout()
plt.show()
```



In [488…
```python
for col in features_of_interest:
    min_value = X[col].min()
    print(f'Minimum value in {col}: {min_value}')
```

```
Minimum value in magnitude: 6.5
Minimum value in sig: 650
Minimum value in nst: 0
Minimum value in dmin: 0.0
Minimum value in gap: 0.0
Minimum value in depth: 2.7
```

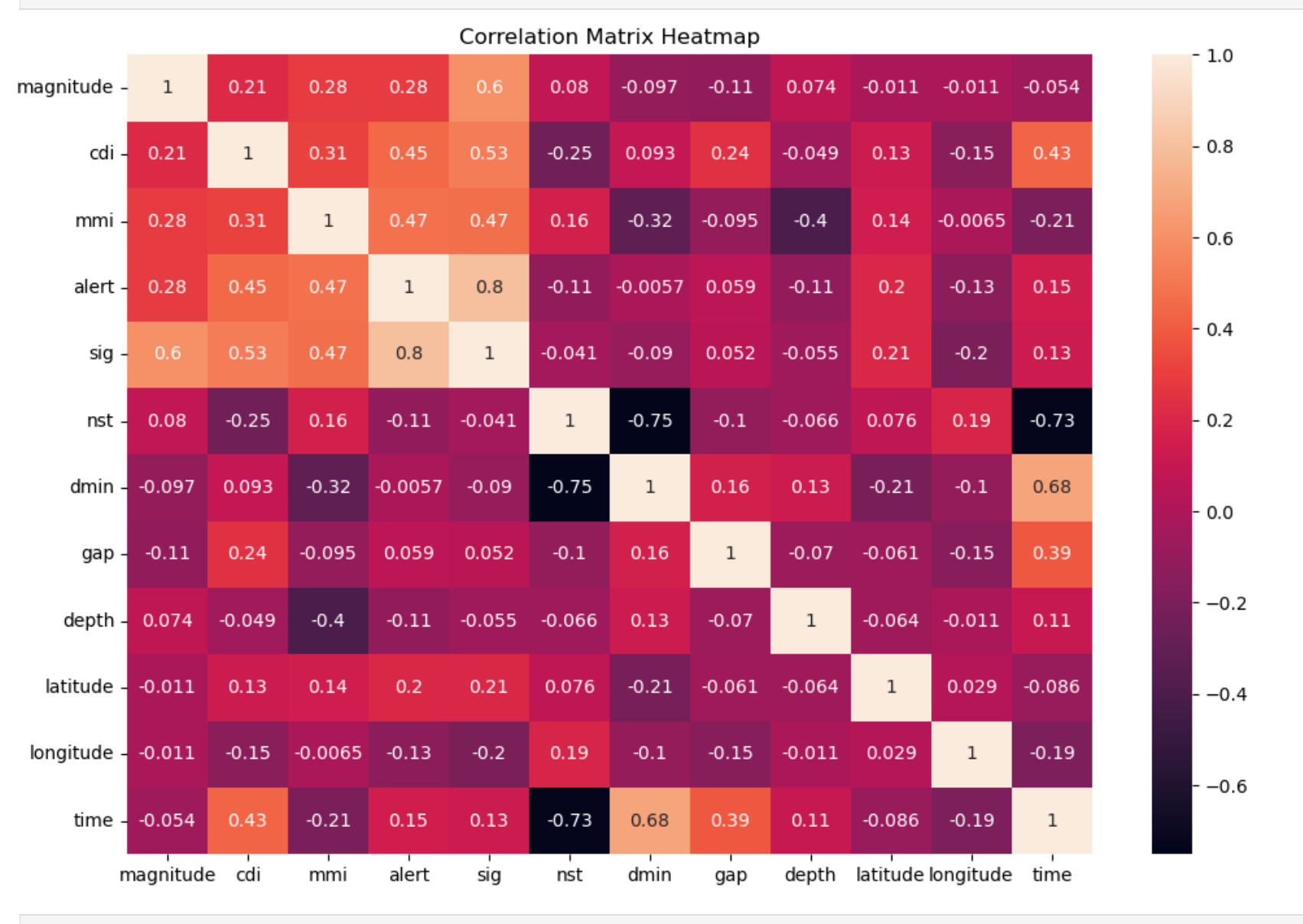
In [489…
```python
X[features_of_interest] = X[features_of_interest].apply(lambda x: np.log1p(x)) #(ChatGPT, 2025)
plt.figure(figsize=(15,10))
for i, col in enumerate(features_of_interest):
    plt.subplot(4, 4, i + 1)
    sns.violinplot(y=X[col])
    sns.boxplot(y=X[col])
    plt.title(f'Violin Plot of {col} after Log Transformation')
plt.tight_layout()
plt.show()
```



In [490…
```python
print(X.isnull().sum().sum())
```

```
0
```

In [491…
```python
corr_matrix = X[non_encoded_render].corr()
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, annot=True)
plt.title('Correlation Matrix Heatmap')
plt.show()
```



In [492…
```python
duplicate_rows = X.duplicated().sum()
print(f'Number of duplicate rows in X: {duplicate_rows}')
```

```
Number of duplicate rows in X: 0
```

Scaling our features

The scaling of features is a critical part of data preprocessing in ML. It involves transforming the features to a common scale without distorting differences in the ranges of values. Without scaling, larger features will receive disproportionate weight updates during training if using gradient descent based algorithms, leading to unstable model training and convergence issues.

Data Splitting

In [493…
```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)
```

In [ ]:
```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler() #This is z-score normalization
scaler = scaler.fit(X_train) #fit only on training data to prevent data leakage
```

In [495…
```python
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
X_train = pd.DataFrame(X_train, columns=X.columns, index=y_train.index)
X_test = pd.DataFrame(X_test, columns=X.columns, index=y_test.index)
```

Feature Reduction, analysis and selection

In [496…
```python
df[['country', 'location','title']].sample(16)
```

Out[496…

	country	location	title
287	Solomon Islands	Gizo, Solomon Islands	M 6.5 - 177 km SE of Gizo, Solomon Islands
273	Chile	Coquimbo, Chile	M 6.9 - 85 km NW of Coquimbo, Chile
729	Papua New Guinea	New Ireland region, Papua New Guinea	M 6.7 - New Ireland region, Papua New Guinea
698	New Caledonia	Tadine, New Caledonia	M 7.3 - 201 km ESE of Tadine, New Caledonia
779	Vanuatu	Port-Olry, Vanuatu	M 7.1 - 16 km NE of Port-Olry, Vanuatu
99	Chile	San Pedro de Atacama, Chile	M 6.8 - 48 km SW of San Pedro de Atacama, Chile
269	Peru	Iñapari, Peru	M 7.6 - 155 km WNW of Iñapari, Peru
457	Indonesia	Kabanjahe, Indonesia	M 6.7 - 68 km WSW of Kabanjahe, Indonesia
268	Peru	Iñapari, Peru	M 7.6 - 185 km WNW of Iñapari, Peru
313	Japan	Miyako, Japan	M 6.7 - 83 km ENE of Miyako, Japan
73	Kermadec Islands region	Kermadec Islands region	M 6.5 - Kermadec Islands region
628	Tonga	Pangai, Tonga	M 8.0 - 50 km ESE of Pangai, Tonga
596	Solomon Islands	Malango, Solomon Islands	M 6.5 - 31 km WSW of Malango, Solomon Islands
549	Indonesia	Manokwari, Indonesia	M 7.7 - 140 km WNW of Manokwari, Indonesia
543	Papua New Guinea	Kokopo, Papua New Guinea	M 6.7 - 190 km ESE of Kokopo, Papua New Guinea
623	Indonesia	Singaparna, Indonesia	M 7.7 - 226 km SSW of Singaparna, Indonesia

In [497…
```python
X_train = X_train.loc[:, ~X_train.columns.str.startswith('title_')]
X_test = X_test.loc[:, ~X_test.columns.str.startswith('title_')]
X_train
```

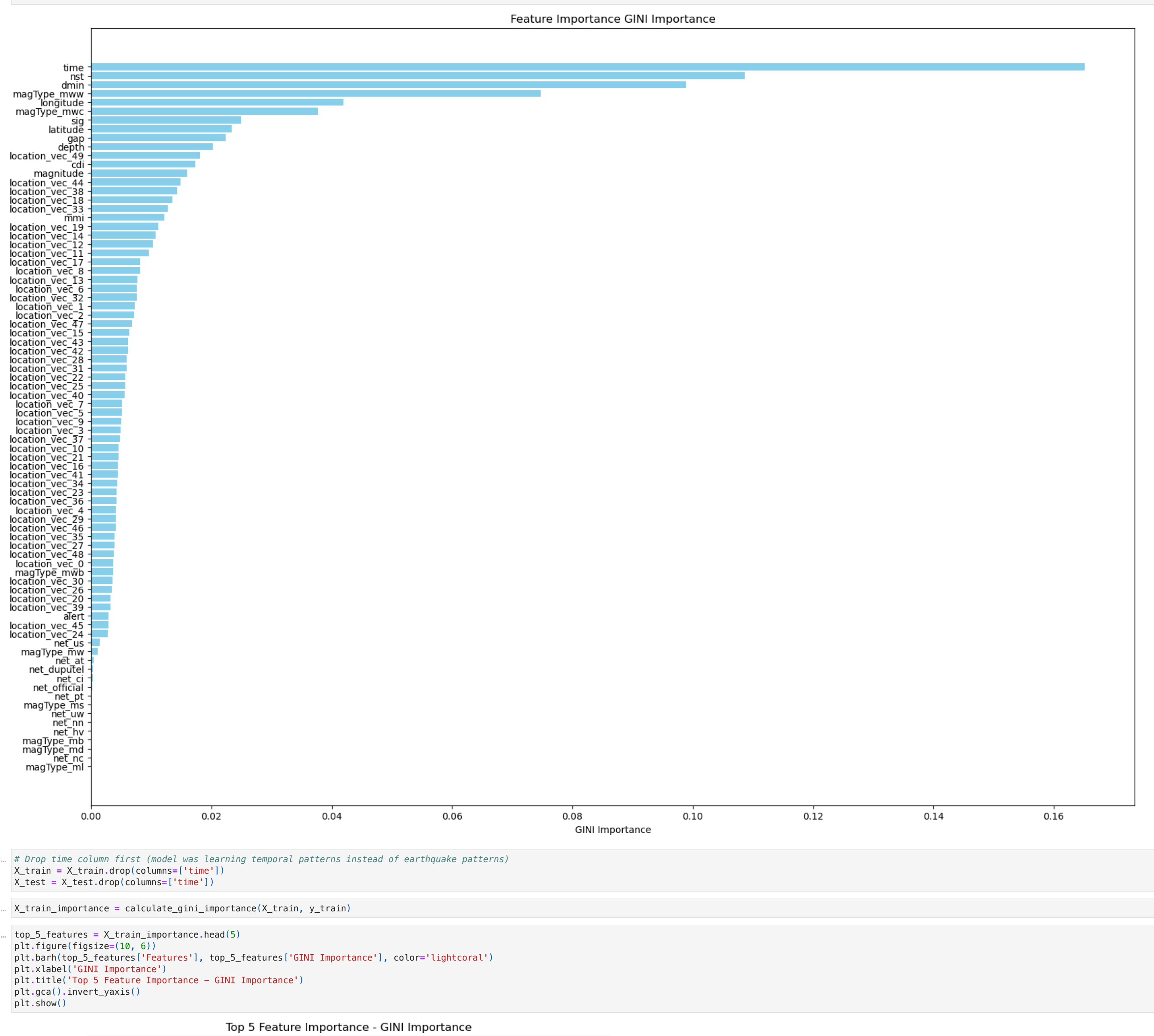
Out[497…

	magnitude	cdi	mmi	alert	sig	nst	dmin	gap	depth	latitude	...	location_vec_41	location_vec_42	location_vec_43	location_vec_44	location_vec_45	location_vec_46	location_vec_47
144	0.353217	-1.359386	-0.690798	-0.420593	-0.254914	-1.058541	2.186257	0.168554	1.215616	-2.323862	...	0.310220	-0.154865	0.534611	-0.147923	0.306386	0.356809	-0.583559
17	-1.031947	-0.423932	-2.773218	-0.420593	-0.871524	0.762332	0.780283	0.572157	2.620881	-0.490490	...	0.063455	0.910081	-0.854457	-0.266549	1.449413	0.894930	-0.212672
755	-0.326043	-1.359386	-0.690798	-0.420593	-0.567498	1.010934	-0.784964	-2.748683	-0.785346	0.732769	...	-2.323440	0.966479	-0.533605	2.023773	-0.496690	0.758354	1.666882
359	-1.031947	-0.112114	-0.690798	-0.420593	-0.790386	-1.058541	0.203609	0.020848	1.114783	0.861601	...	-2.323440	0.966479	-0.533605	2.023773	-0.496690	0.758354	1.666882
645	1.220770	1.135157	0.697483	-0.420593	0.715216	1.121549	-0.784964	0.416433	1.083805	-0.365117	...	0.148750	1.211229	-0.918483	-0.488235	-0.041433	0.963772	0.089733
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
409	0.353217	1.446975	0.003343	-0.420593	-0.236542	0.877574	-0.784964	-0.006145	-0.435380	-0.559634	...	1.427180	0.014319	0.763721	-1.245741	-0.346512	-0.028771	-1.421893
539	1.220770	0.823339	0.003343	1.129878	1.443418	0.845239	-0.784964	0.082688	-0.318570	0.367634	...	-0.654586	-1.531617	2.181798	1.821560	0.149246	1.235504	-1.072015
1	-0.096763	-0.112114	-1.384938	-0.420593	-0.448879	0.474988	0.885594	0.658023	-0.282731	-0.338363	...	0.055360	0.158381	-0.456150	0.796233	0.957665	-0.805154	0.449705
203	-1.031947	1.446975	1.391623	1.129878	-0.424654	-1.058541	1.547856	0.541736	-1.155848	0.257516	...	-0.216071	-0.273803	-1.241013	0.350649	-0.865501	-0.337176	-0.523553
679	-0.793556	-1.359386	1.391623	-0.420593	-0.779706	1.160293	-0.784964	0.736822	-0.670976	1.224506	...	-2.323440	0.966479	-0.533605	2.023773	-0.496690	0.758354	1.666882

623 rows × 80 columns

In [498…
```python
from sklearn.ensemble import RandomForestClassifier
def calculate_gini_importance(X, y):
    clf = RandomForestClassifier(n_estimators=100, random_state=42)
    clf.fit(X, y)
    importances = clf.feature_importances_
    return pd.DataFrame({'Features': X.columns, 'GINI Importance': importances}).sort_values('GINI Importance', ascending=False)

feature_imp_df = calculate_gini_importance(X_train, y_train)
```

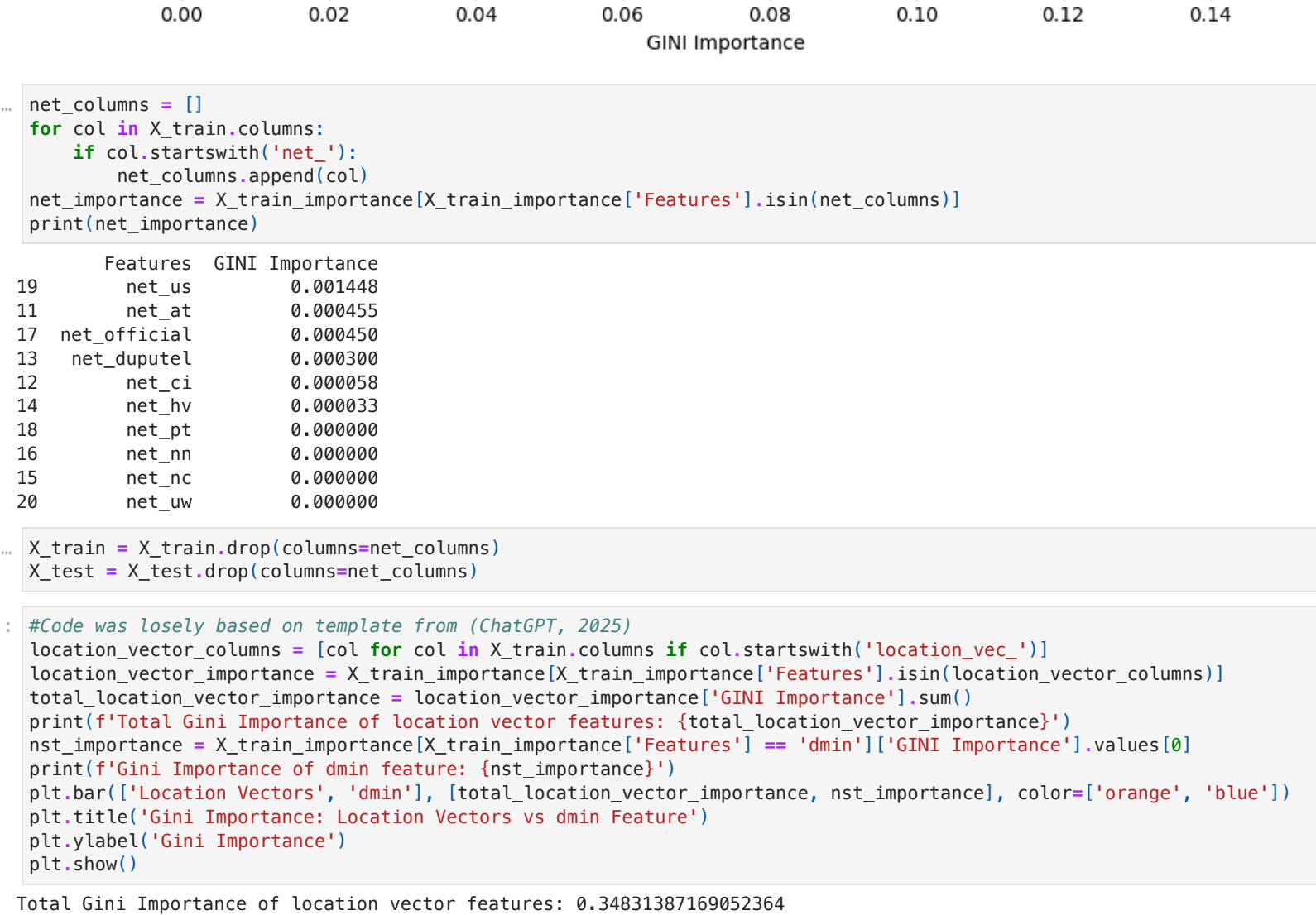
In [499…
```python
plt.figure(figsize=(20, 15))
plt.barh(feature_imp_df['Features'], feature_imp_df['GINI Importance'], color='skyblue')
plt.xlabel('GINI Importance')
plt.title('Feature Importance GINI Importance')
plt.gca().invert_yaxis()
plt.show()
```



In [500…
```python
# Drop time column first (model was learning temporal patterns instead of earthquake patterns)
X_train = X_train.drop(columns=['time'])
X_test = X_test.drop(columns=['time'])
```

In [501…
```python
X_train_importance = calculate_gini_importance(X_train, y_train)
```

In [502…
```python
top_5_features = X_train_importance.head(5)
plt.figure(figsize=(10, 6))
plt.barh(top_5_features['Features'], top_5_features['GINI Importance'], color='lightcoral')
plt.xlabel('GINI Importance')
plt.title('Top 5 Feature Importance - GINI Importance')
plt.gca().invert_yaxis()
plt.show()
```



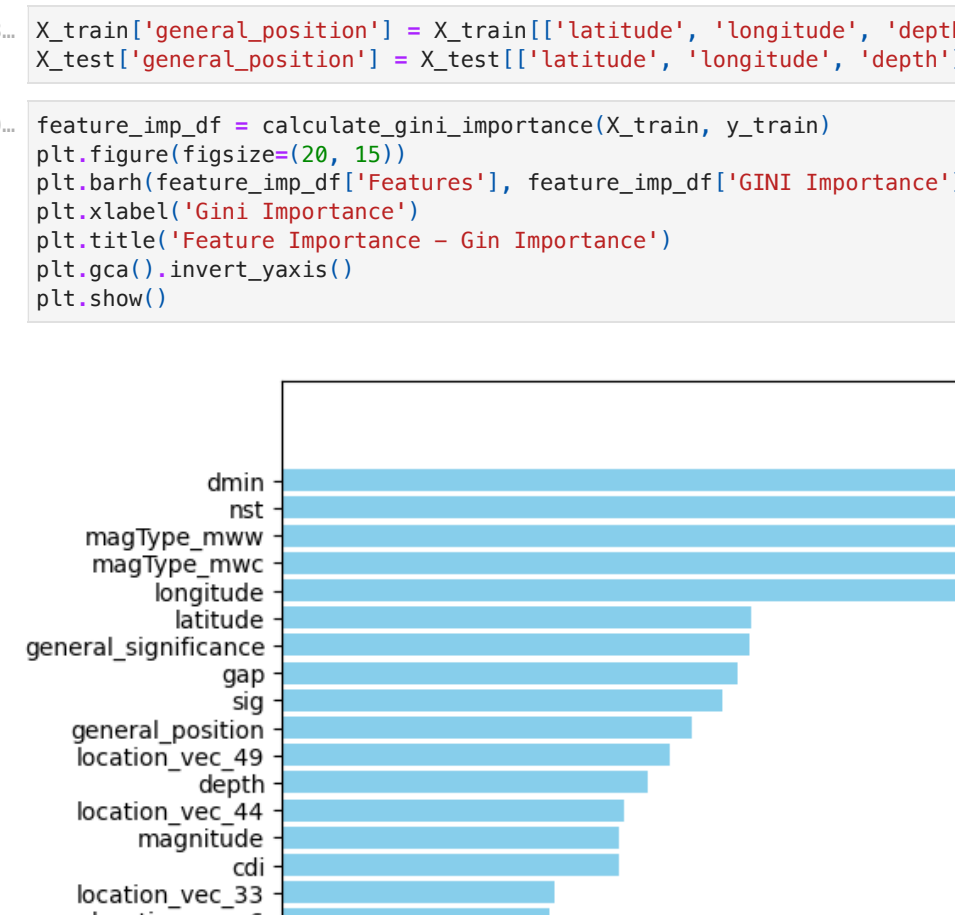
In [503…
```python
net_columns = []
for col in X_train.columns:
    if col.startswith('net_'):
        net_columns.append(col)
net_importance = X_train_importance[X_train_importance['Features'].isin(net_columns)]
print(net_importance)
```

```
       Features  GINI Importance
19       net_us         0.001448
11       net_at         0.000455
17  net_official        0.000450
13   net_duputel        0.000300
12       net_ci         0.000058
14       net_hv         0.000033
18       net_pt         0.000000
15       net_nn         0.000000
16       net_nc         0.000000
20       net_uw         0.000000
```

In [504…
```python
X_train = X_train.drop(columns=net_columns)
X_test = X_test.drop(columns=net_columns)
```

In [ ]:
```python
#Code was losely based on template from (ChatGPT, 2025)
location_vector_columns = [col for col in X_train.columns if col.startswith('location_vec_')]
location_vector_importance = X_train_importance[X_train_importance['Features'].isin(location_vector_columns)]
total_location_vector_importance = location_vector_importance['GINI Importance'].sum()
print(f'Total Gini Importance of location vector features: {total_location_vector_importance}')
nst_importance = X_train_importance[X_train_importance['Features'] == 'dmin']['GINI Importance'].values[0]
print(f'Gini Importance of dmin feature: {nst_importance}')
plt.bar(['Location Vectors', 'dmin'], [total_location_vector_importance, nst_importance], color=['orange', 'blue'])
plt.title('Gini Importance: Location Vectors vs dmin Feature')
plt.ylabel('Gini Importance')
plt.show()
```

```
Total Gini Importance of location vector features: 0.34831387196052364
Gini Importance of dmin feature: 0.14105507425359742
```



In [506…
```python
magtype_drop_columns = ['magType_mb', 'magType_ms', 'magType_ml', 'magType_md', 'magType_mw', 'magType_mwb'] #dropping low importance features
X_train = X_train.drop(columns=magtype_drop_columns)
X_test = X_test.drop(columns=magtype_drop_columns)
```

Feature engineering

In [507…
```python
X_train['general_significance'] = X_train[['sig', 'magnitude', 'cdi', 'mmi']].mean(axis=1)
X_test['general_significance'] = X_test[['sig', 'magnitude', 'cdi', 'mmi']].mean(axis=1)
```

In [508…
```python
X_train['general_position'] = X_train[['latitude', 'longitude', 'depth']].mean(axis=1)
X_test['general_position'] = X_test[['latitude', 'longitude', 'depth']].mean(axis=1)
```

In [509…
```python
feature_imp_df = calculate_gini_importance(X_train, y_train)
plt.figure(figsize=(20, 15))
plt.barh(feature_imp_df['Features'], feature_imp_df['GINI Importance'], color='skyblue')
plt.xlabel('Gini Importance')
plt.title('Feature Importance - Gin Importance')
plt.gca().invert_yaxis()
plt.show()
```

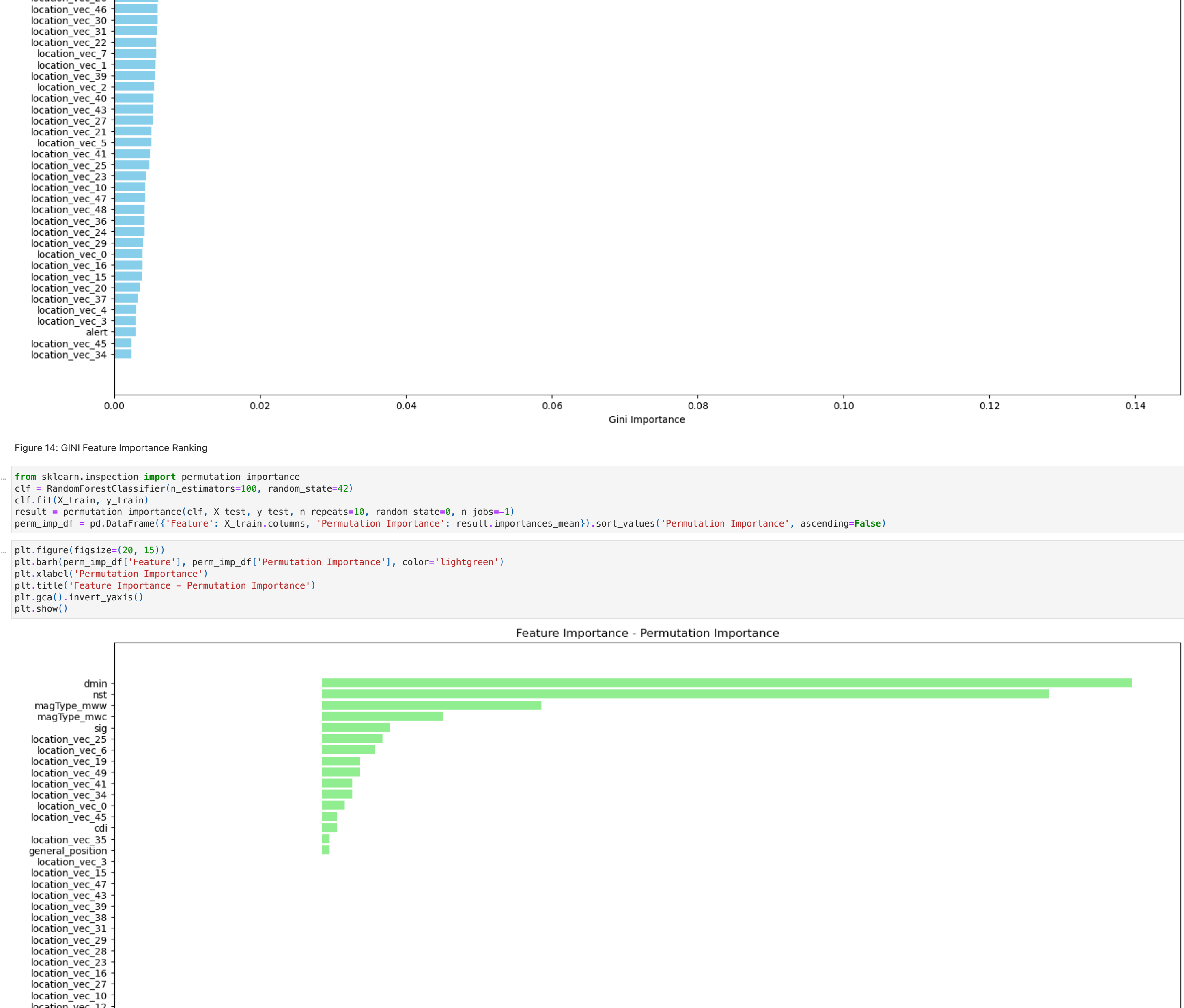
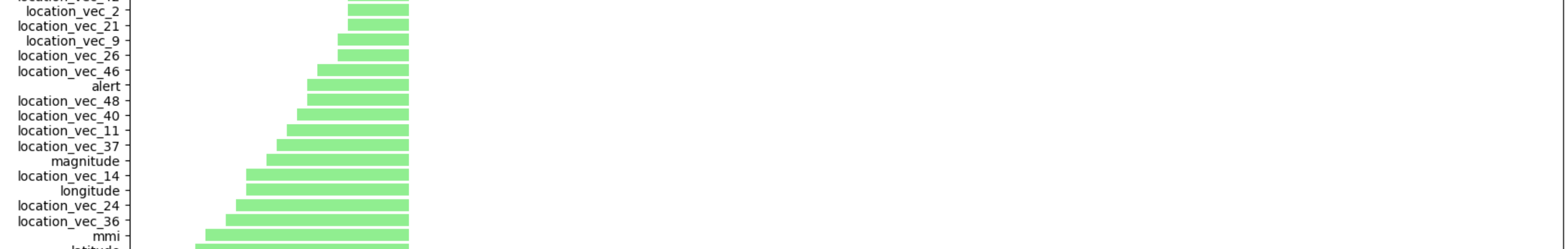


Figure 14: GINI Feature Importance Ranking

In [510…
```python
from sklearn.inspection import permutation_importance
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)
result = permutation_importance(clf, X_test, y_test, n_repeats=10, random_state=0, n_jobs=-1)
perm_imp_df = pd.DataFrame({'Feature': X_train.columns, 'Permutation Importance': result.importances_mean}).sort_values('Permutation Importance', ascending=False)
```

In [511…
```python
plt.figure(figsize=(20, 15))
plt.barh(perm_imp_df['Feature'], perm_imp_df['Permutation Importance'], color='lightgreen')
plt.xlabel('Permutation Importance')
plt.title('Feature Importance - Permutation Importance')
plt.gca().invert_yaxis()
plt.show()
```



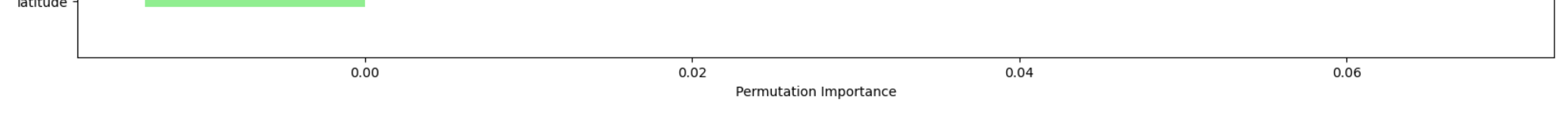


Figure 15: Premier interaction feature importance ranking

```
In [512… import shap
         explainer = shap.TreeExplainer(clf)
         shap_values = explainer.shap_values(X_test)
         shap_values_class1 = shap_values[:, :, 1]
         shap.summary_plot(shap_values_class1, X_test, plot_type='dot')
```

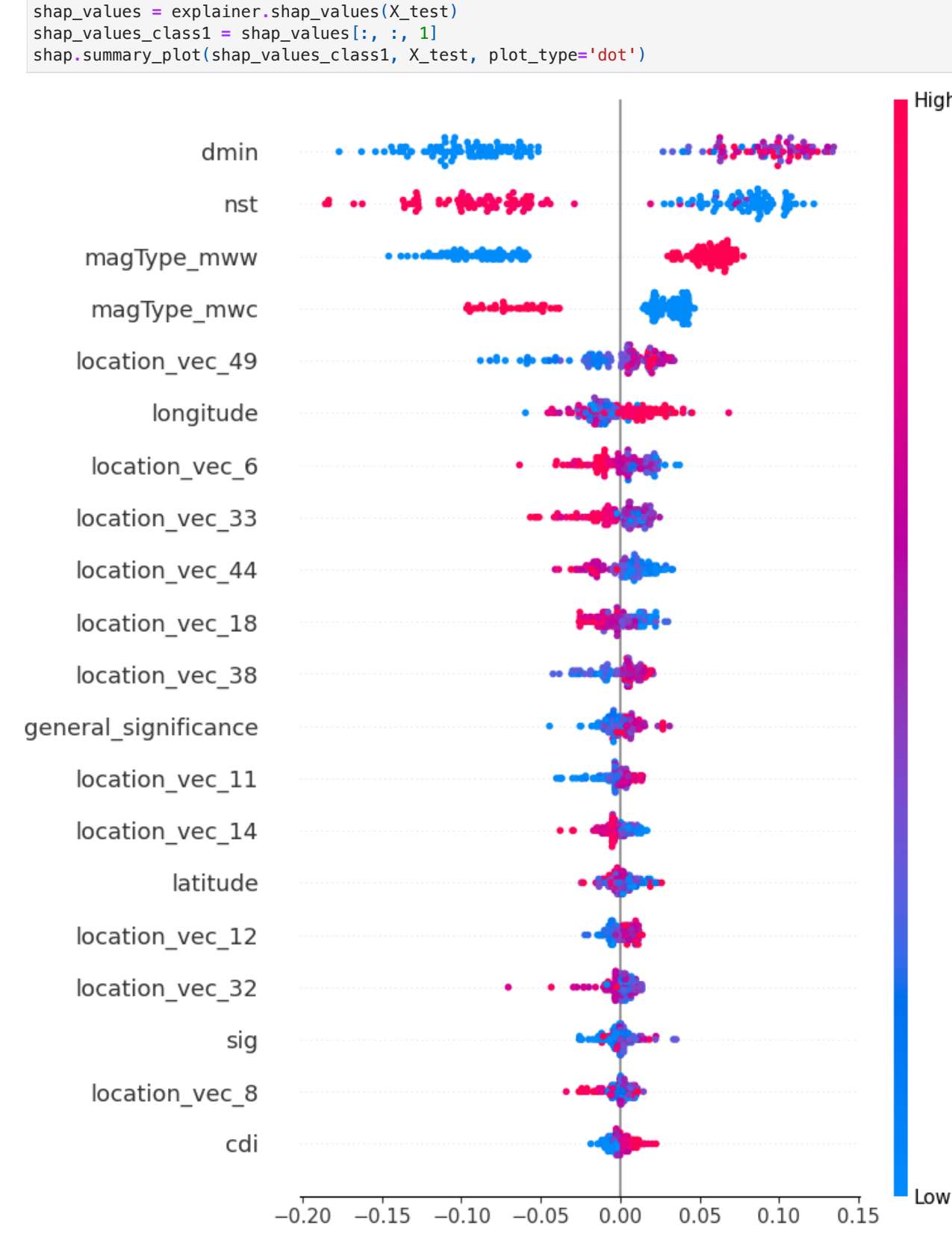


Figure 16: SHAP Summary Plot Showing Feature Importance

Feature Importance Ranking (SHAP and GINI combined) along with Domain Relevance

Domain relevance dermined by consulting seismology literature on earthquake and tsunami relationships (Lomax, Michelini and Piatanesi, 2011; Wald et al., 2011; United States Geological Survey, 1989) and the table provided by (Chauhan, 2024).

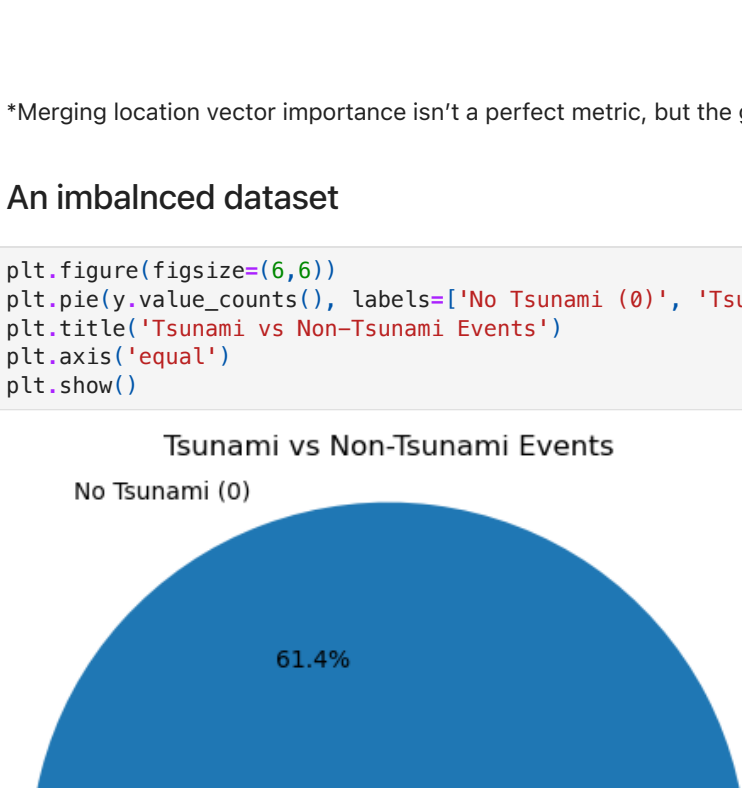
Importance rankings considered both SHAP (figure 16) and GINI (figure 14) feature importance, and (figure 15) premutation importance when dermining overall feature importance.

Rank	Feature Name	Domain Relevance
1	Location Vectors (merged)*	Encodes geographic patterns and seismic hotspots.
2	dmin	Distance to nearest station, affects location accuracy.
3	nst	Number of reporting stations, higher counts improve reliability.
4	magType_mww	Moment magnitude method, affects magnitude measurement.
5	magType_mwc	Alternatve magnitude method, aids in cross-checking readings.
6	longitude	Longitude coordinate, key for mapping and regional analysis.
7	latitude	Latitude coordinate, identifes location.
8	sig	Significance of a seismic event determined by muliple sources
9	cdi	Reported intensity, indicates how strongly the quake was felt.
10	general_significance	Mean of `sig`, `magnitude`, `cdi`, and `mmi`;

*Merging location vector importance isn't a perfect metric, but the generally sensible ranking supports its inclusion.

An imbalnced dataset

```
In [513… plt.figure(figsize=(6,6))
         plt.pie(y.value_counts(), labels=['No Tsunami (0)', 'Tsunami (1)'], autopct='%1.1f%%') #autopact from Matplotlib documentation (Matplotlib, no date)
         plt.title('Tsunami vs Non-Tsunami Events')
         plt.axis('equal')
         plt.show()
```



Dataset imbalance handling techniques discussion

Examining the chart I can see that our dataset is imbalanced, with a significantly higher proportion of non-tsunami events (0) compared to tsunami events (1). In order to address this class imbalance during model training I will use class weights depending on model compatibility (Google LLC, 2025). SMOTE is another technique for handling imbalanced datasets by generating synthetic samples for the minority class. However, given our small dataset size and high dimensionality due to embedding features, using SMOTE could lead to overfitting and may not be effective as explored by Blagus & Lusa.

(Blagus & Lusa,2013)

Data limitation and quality issues

Throughout the EDA and data preprocessing steps, I have identified and addressed several data quality issues that could impact our modeling efforts. Our small dataset size of only 779 samples after cleaning limits the complexity of models I can use without overfitting. Of those samples many have a left skew (figure 2), meaning the model may to bias towards lower impact earthquakes, which are less likely to generate tsunamis as acknowledged in (United States Geological Survey, 2023). Our considerable data imputation to resolve our missing data may also introduce some uncertainty, as imputed values may not perfectly reflect true values. Furthermore, relying on a single data source (USGS) may introduce collection bias or limitations based on their data collection methods and coverage, as acknowledged by (United States Geological Survey, 2022).

Ethics and Privacy Considerations

While our dataset contains no personally identifiable information about individuals, there are still important ethical considerations to keep in mind when working with this data. Especially around deployment, any model used for tsunami prediction must be rigorously tested to avoid false positives or negatives predictions that could lead to unnecessary concern or danger for populations at risk, this will make precision and recall key metrics to focus on during model evaluation. Furthermore, a consideration must be made for accessibility for poorer and remote communities who may not have access to expensive high compute resources.

Model Selection Rationale

As mentioned in the dataset insights and feature types section, our dataset and `tsunami` target variable are well suited for binary classification models (figure 4). Given the small dataset size and class imbalance, I will focus on models that can handle these challenges effectively and acknowling our small dataset size, I must choose models that are less prone to overfitting.

Model 1: Logistic Regression with Class Weights

Logistic regression is a simple but effective method for binary classification (Lee, 2025). It models the probability of a tsunami event using a sigmoid function and allows class weighting to address the dataset's imbalance (Google LLC, 2025). It also performs well when the number of features compares disproportionately to the number of samples, as in our vectorised dataset, though this typically requires careful regularisation and hyperparameter tuning (Wainwright, Ravikumar & Lafferty, 2006).

Logistic regression is considered to be a less resource intensive algorithm compared to more complex models like neural networks or ensemble methods. This makes it a suitable choice for deployment in low-resource settings, such as remote communities that may not have access to high compute resources as discussed in the ethics and privacy considerations section, (Ayer et al., 2010).

Model 2: Gradient Boosting classifier

Gradient Boosting Classifier is an ensemble learning method that builds a strong classifier by combining multiple weak decision trees. It iteratively adds trees that correct the errors of previous trees, leading to improved predictive performance (Friedman, 1999). Gradient boosting is less prone to overfitting compared to single decision trees, making it suitable for our small dataset (Natekin & Knoll, 2013). Gradient decision trees can be highly sesnsitive to class imbalance (Luo, 2024), and our sklearn implementation does not allow for class weighting. This is a limiation that could impact model performance on our imbalanced dataset.

Gradient boosting is more computationally intensive than logistic regression, however in comparison to deep learning models, it is still relatively efficient and can be optimized for deployment in low-resource settings with appropriate hyperparameter tuning (Peter et al., 2017).

Research questions

Will a gradient boosting classifier outperform logistic regression in predicting tsunami occurrence following earthquakes based on seismic event characteristics?

This question aims to compare the performance of two different types of model. This question is influenced by our overall goal of building an accurate tsunami prediction model. The answer to this question is of great importaance as selecting the right model architecture is crucial for achieving high predictive accuracy, something that is vital for effective disaster management and early warning systems intent on predicting tsunamis.

Which feature will be the most important predictor of tsunami occurrence following earthquakes?

Given the diversity of features in our dataset, this question aims to identify which specific feature has the highest predictive power for tsunami occurrence. Understanding feature importance can provide insights into the underlying factors that contribute to tsunami generation, which is valuable for both scientific understanding and practical applications in tsunami risk assessment.

Model Training

```
In [514… from sklearn.model_selection import StratifiedKFold, cross_val_score
         k_folds = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
         X_recombined, y_recombined = pd.concat([X_train, X_test], axis=0), pd.concat([y_train, y_test], axis=0)
```

```
In [ ]: def overfit_underfit_analysis(train_score, cross_val_score):
            print(f'Training Score: {train_score}')
            print(f'Cross Validation Score: {cross_val_score}')
            if train_score - cross_val_score > 0.01:
                print('The model is overfitting.')
                print(f'Difference: {train_score - cross_val_score}')
            elif cross_val_score - train_score > 0.01:
                print('The model is underfitting.')
                print(f'Difference: {cross_val_score - train_score}')
            else:
                print('The model has a good fit.')
```

The above method of detecting overfitting and underfitting is supported by (Aburass, 2023).

Model 1: Logistic Regression

```
In [ ]: from sklearn.linear_model import LogisticRegression
        from sklearn.metrics import accuracy_score
        logclf = LogisticRegression(class_weight='balanced')
        logclf.fit(X_train, y_train)
```

```
Out[ ]: ▾ LogisticRegression
        ▸ Parameters
```

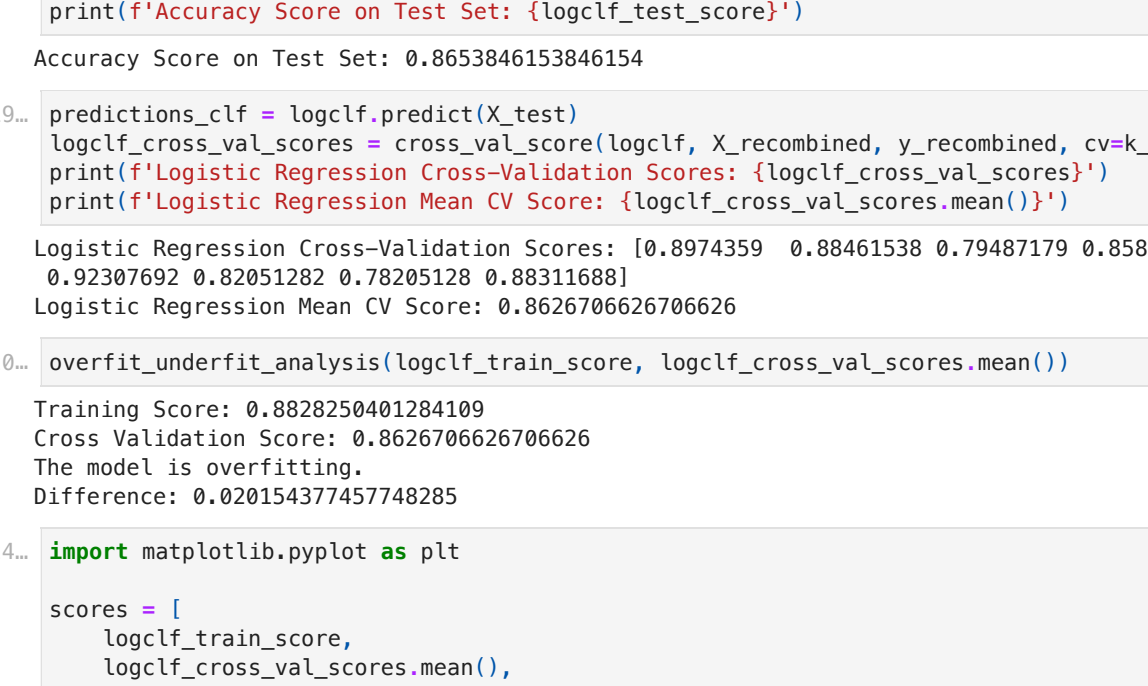
```
In [517… logclf_train_score = accuracy_score(y_train, logclf.predict(X_train))
         print(f'Accuracy Score on Training Set: {logclf_train_score}')

Accuracy Score on Training Set: 0.8828250401284109
```

```
In [590… logclf_test_score = accuracy_score(y_test, logclf.predict(X_test))
         print(f'Accuracy Score on Test Set: {logclf_test_score}')

Accuracy Score on Test Set: 0.8653846153846154
```

```
In [519… predictions_clf = logclf.predict(X_test)
         logclf_cross_val_scores = cross_val_score(logclf, X_recombined, y_recombined, cv=k_folds)
         print(f'Logistic Regression Cross-Validation Scores: {logclf_cross_val_scores}')
         print(f'Logistic Regression Mean CV Score: {logclf_cross_val_scores.mean()}')

Logistic Regression Cross-Validation Scores: [0.8974359  0.88461538 0.79487179 0.85897436 0.88461538 0.8974359
 0.92307692 0.82051282 0.78205128 0.88311688]
Logistic Regression Mean CV Score: 0.8626706626706626
```

```
In [520… overfit_underfit_analysis(logclf_train_score, logclf_cross_val_scores.mean())

Training Score: 0.8828250401284109
Cross Validation Score: 0.8626706626706626
The model is overfitting.
Difference: 0.020154377457748285
```

```
In [624… import matplotlib.pyplot as plt

         scores = [
             logclf_train_score,
             logclf_cross_val_scores.mean(),
             logclf_test_score
         ]

         labels = ['Training', 'Cross Validation', 'Test']

         plt.figure(figsize=(8,5))
         plt.bar(labels, scores, color=['blue', 'orange', 'green'])
         plt.ylim(0.9 * min(scores), 1)# (ChatGPT, 2025)
         plt.ylabel("Accuracy")
         plt.title('Logistic Regression Train/Test/CV Accuracy Scores')
         plt.grid(axis='y', linestyle='--', alpha=0.5)
         plt.show()
```



Hyperparameter Tuning for Logistic Regression

Our logistic regression model has achieved a good baseline performance, but there is still room for improvement through hyperparameter tuning, especially around regularization strength as the model appears to be overfitting slightly given the difference between training and cross-validation scores. I will preform a grid search on a selection of hyperparameters to identify the optimal combination that maximizes model performance while minimizing overfitting (figure 19).

Parameter	Description	Effect on Overfitting problem
C	Inverse of regularization strength. Smaller values mean stronger regularization, simpler model. Larger values mean weaker regularization.	Smaller C → reduces overfitting.
penalty	Type of regularization. 'l2' adds squared weight penalty to shrink weights and reduce overfitting.	'l2' → controls overfitting.
class_weight	Adjusts importance of classes for imbalanced datasets. 'balanced' scales inversely to class frequency.	Balanced classes → reduces bias overfitting.
max_iter	Maximum number of iterations for solver convergence. Too low may prevent convergence; higher values allow full optimization.	Not directly related → does not meaningfully affect overfitting.
tol	Tolerance for stopping criteria. Smaller values give more precise convergence and longer training. Larger values stop earlier and train faster.	Not directly related → minor indirect impact at most.
solver	Algorithm for optimization. 'liblinear' is good for small datasets and supports L1 & L2; 'lbfgs' (default) is better for larger datasets.	Solver choice → affects stability, not overfitting.

figure 19: Explanations of Logistic Regression Hyperparameters

Exploring C (regularization strength)

```
In [ ]: hyperparameters = {
            'penalty':'l2', #regularization type
            'C':0.1, #regularization strength (inverse ratio - smaller values specify stronger regularization)
            'class_weight':'balanced' #address class imbalance (as before)
        }

        # Previous hyperparameter attempts:
        #hyperparameters = {
        #    'penalty':'l2',
        #    'C':0.1,
        #    'class_weight':'balanced'
        #}

        #hyperparameters = {
        #    'penalty':'l2',
        #    'C':0.05,
        #    'class_weight':'balanced'
        #}

        #hyperparameters = {
        #    'penalty':'l2',
        #    'C':0.5,
        #    'class_weight':'balanced'
        #}

        #hyperparameters = {
        #    'penalty':'l1',
        #    'solver':'saga',
        #    'C':0.1,
        #    'class_weight':'balanced'
        #}
```

```
In [523… logclf_tuned = LogisticRegression(**hyperparameters)
         logclf_tuned.fit(X_train, y_train)
```

```
Out[523… ▾ LogisticRegression
         ▸ Parameters
```

```
In [524… logclf_tuned_train_score = accuracy_score(y_train, logclf_tuned.predict(X_train))
         print(f'Accuracy Score on Training Set: {logclf_tuned_train_score}')
         logclf_tuned_test_score = accuracy_score(y_test, logclf_tuned.predict(X_test))
         print(f'Accuracy Score on Test Set: {logclf_tuned_test_score}')

Accuracy Score on Training Set: 0.8764044943820225
Accuracy Score on Test Set: 0.8974358974358975
```

```
In [525… logclf_tuned_cross_val_scores = cross_val_score(logclf_tuned, X_recombined, y_recombined, cv=k_folds)
         print(f'Logistic Regression Tuned Cross-Validation Scores: {logclf_tuned_cross_val_scores}')
         print(f'Logistic Regression Tuned Mean CV Score: {logclf_tuned_cross_val_scores.mean()}')

Logistic Regression Tuned Cross-Validation Scores: [0.88461538 0.8974359  0.83333333 0.91025641 0.87179487 0.8974359
 0.92307692 0.79487179 0.80769231 0.87012987]
Logistic Regression Tuned Mean CV Score: 0.869064269064269
```

```
In [526… overfit_underfit_analysis(logclf_tuned_train_score, logclf_tuned_cross_val_scores.mean())

Training Score: 0.8764044943820225
Cross Validation Score: 0.869064269064269
The model has a good fit.
```

```
In [527… print(f'Previous Logistic Regression Mean CV Score: {logclf_cross_val_scores.mean()}')
         print(f'Tuned Logistic Regression Mean CV Score: {logclf_tuned_cross_val_scores.mean()}')
         print(f'Improvement in Mean CV Score after Tuning: {logclf_tuned_cross_val_scores.mean() - logclf_cross_val_scores.mean()}')

Previous Logistic Regression Mean CV Score: 0.8626706626706626
Tuned Logistic Regression Mean CV Score: 0.869064269064269
Improvement in Mean CV Score after Tuning: 0.006393606393606421
```

Further Hyperparameter Tuning for general performance improvement (max iterations and tolerance)

```
In [ ]: hyperparameters = {
            'C':hyperparameters['C'],
            'class_weight':'balanced',
            'max_iter':100, #increase max iterations to ensure convergence
            'tol':1e-4 #tolerance for stopping criteria (how much improvement is required to continue training)
        }
        #previously attempted:
        #hyperparameters = {
        #    'C':0.01, #keep previous best C value
        #    'class_weight':'balanced',
        #    'max_iter':200,
        #    'tol':1e-3
        #}

        #hyperparameters = {
        #    'C':0.01, #keep previous best C value
        #    'class_weight':'balanced',
        #    'max_iter':300,
        #    'tol':1e-5
        #}

        #hyperparameters = {
        #    'C':0.01, #keep previous best C value
        #    'class_weight':'balanced',
        #    'max_iter':300,
        #    'tol':1e-5
        #}

        #hyperparameters = {
        #    'C':0.01, #keep previous best C value
        #    'class_weight':'balanced',
        #    'max_iter':500,
        #    'tol':1e-4
        #}   'solver':'liblinear' #Known to work well with small datasets
```

```
In [529… logclf_tuned_v2 = LogisticRegression(**hyperparameters)
         logclf_tuned_v2.fit(X_train, y_train)
```

```
Out[529… ▾ LogisticRegression
         ▸ Parameters
```

```
In [530… logclf_tuned_cross_val_scores_general_performance = cross_val_score(logclf_tuned_v2, X_recombined, y_recombined, cv=k_folds)
         print(f'Logistic Regression Tuned Cross-Validation Scores: {logclf_tuned_cross_val_scores_general_performance}')
         print(f'Logistic Regression Tuned Mean CV Score: {logclf_tuned_cross_val_scores_general_performance.mean()}')

Logistic Regression Tuned Cross-Validation Scores: [0.88461538 0.8974359  0.83333333 0.91025641 0.87179487 0.8974359
 0.92307692 0.79487179 0.80769231 0.87012987]
Logistic Regression Tuned Mean CV Score: 0.869064269064269
```

```
In [531… overfit_underfit_analysis(logclf_tuned_train_score, logclf_tuned_cross_val_scores.mean())

Training Score: 0.8764044943820225
Cross Validation Score: 0.869064269064269
The model has a good fit.
```

```
In [532… print(f'Previous Logistic Regression Tuned Mean CV Score: {logclf_tuned_cross_val_scores.mean()}')
         print(f'New Tuned Logistic Regression Mean CV Score: {logclf_tuned_cross_val_scores_general_performance.mean()}')
         print(f'Improvement in Mean CV Score after Further Tuning: {logclf_tuned_cross_val_scores_general_performance.mean() - logclf_tuned_cross_val_scores.mean()}')

Previous Logistic Regression Tuned Mean CV Score: 0.869064269064269
New Tuned Logistic Regression Mean CV Score: 0.869064269064269
Improvement in Mean CV Score after Further Tuning: 0.0
```

Class weight tuning

```
In [ ]: hyperparameters = {
            'C':0.01, #keep previous best C value
            'penalty':'l2',
            'class_weight':None, #remove class weights
        }

        #previously attempted:

        # hyperparameters = {
        #     'C':0.01, #keep previous best C value
        #     'penalty':'l2',
        #     'class_weight': {0: 2.0, 1: 1.0}, #adjust class weights to give more importance to non-tsunami events
        # }
```

```
In [ ]: logclf_tuned_no_class_weights = LogisticRegression(**hyperparameters)
        logclf_tuned_no_class_weights.fit(X_train, y_train)
```

```
Out[ ]: ▾ LogisticRegression
        ▸ Parameters
```

```
In [ ]: logclf_tuned_no_class_weights_train_score = accuracy_score(y_train, logclf_tuned_no_class_weights.predict(X_train))
        print(f'Accuracy Score on Training Set without Class Weights: {logclf_tuned_no_class_weights_train_score}')
        logclf_tuned_no_class_weights_test_score = accuracy_score(y_test, logclf_tuned_no_class_weights.predict(X_test))
        print(f'Accuracy Score on Test Set without Class Weights: {logclf_tuned_no_class_weights_test_score}')

Accuracy Score on Training Set without Class Weights: 0.8812199036918138
Accuracy Score on Test Set without Class Weights: 0.8974358974358975
```

```
In [ ]: logclf_tuned_no_class_weights_cross_val_scores = cross_val_score(logclf_tuned_no_class_weights, X_recombined, y_recombined, cv=k_folds)
        print(f'Logistic Regression without Class Weights Cross-Validation Scores: {logclf_tuned_no_class_weights_cross_val_scores}')
        print(f'Logistic Regression without Class Weights Mean CV Score: {logclf_tuned_no_class_weights_cross_val_scores.mean()}')

Logistic Regression without Class Weights Cross-Validation Scores: [0.8974359  0.88461538 0.8974359  0.92307692 0.85897436 0.88461538
 0.92307692 0.83333333 0.82051282 0.84415584]
Logistic Regression without Class Weights Mean CV Score: 0.8729104229104229
```

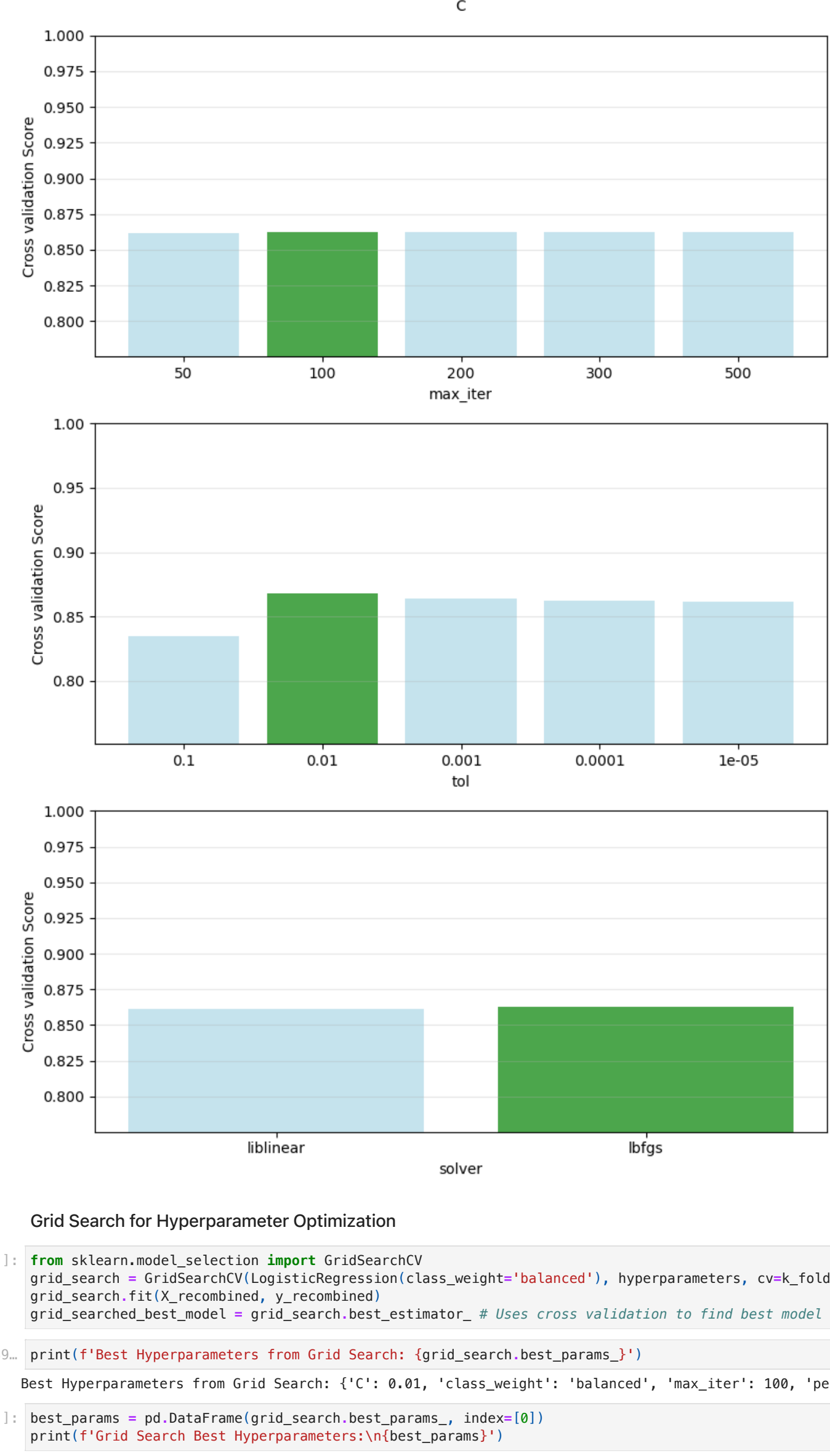
```
In [ ]: print(f'Previous Best Tuned Logistic Regression Mean CV Score: {logclf_tuned_cross_val_scores.mean()}')
        print(f'Logistic Regression without Class Weights Mean CV Score: {logclf_tuned_no_class_weights_cross_val_scores.mean()}')
        print(f'Change in Mean CV Score after Removing Class Weights: {logclf_tuned_no_class_weights_cross_val_scores.mean() - logclf_tuned_cross_val_scores.mean()}')

Previous Best Tuned Logistic Regression Mean CV Score: 0.869064269064269
Logistic Regression without Class Weights Mean CV Score: 0.8729104229104229
Change in Mean CV Score after Removing Class Weights: 0.0038461538461538325
```

Hyperparameter Optimization and Visualization

```
In [ ]: hyperparameters = {
            'C':[0.005,0.01, 0.1, 0.3, 0.5,0.8],
            'penalty':['l2'], #keep l2 regularization
            'class_weight':['balanced'], #keep class weights to address class imbalance
            'max_iter':[50,100, 200, 300, 500], #vary max iterations
            'tol':[1e-1,1e-2,1e-3, 1e-4, 1e-5], #vary tolerance
            'solver':['liblinear', 'lbfgs'] #try different solvers, other solvers don't support l2 penalty
        }

        results = {}
        for param, values in hyperparameters.items():
            scores = []
            for val in values:
                params = {param: None}
                params[param] = val
                model = LogisticRegression(**params)
                cv = cross_val_score(model, X_recombined, y_recombined, cv=k_folds)
                scores.append(cv.mean())
            results[param] = (values, scores)
```

```
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 50 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=50).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
In [ ]: fig, axes = plt.subplots(4, 1, figsize=(8, 15))

        changable_hyperparameters = [param for param in hyperparameters.keys() if param != 'penalty' and param != 'class_weight']

        for i, param in enumerate(changable_hyperparameters):

            scores = results[param][1]
            best_idx = scores.index(max(scores))
            colors = ['green' if j == best_idx else 'lightblue' for j in range(len(scores))] # (ChatGPT, 2025)
            axes[i].bar(range(len(results[param][0])), results[param][1], color=colors, alpha=0.7)
            axes[i].set_xlabel(param)
            axes[i].set_ylabel('Cross validation Score')
            axes[i].set_xticks(range(len(results[param][0])))
            axes[i].set_xticklabels(results[param][0])
            axes[i].set_ylim(0.9 * min(results[param][1]), 1)
            axes[i].grid(True, alpha=0.3, axis='y')

        plt.tight_layout()
        plt.show()
```



Grid Search for Hyperparameter Optimization

```
In [ ]: from sklearn.model_selection import GridSearchCV
        grid_search = GridSearchCV(LogisticRegression(class_weight='balanced'), hyperparameters, cv=k_folds)
        grid_search.fit(X_recombined, y_recombined)
        grid_searched_best_model = grid_search.best_estimator_ # Uses cross validation to find best model hyperparameters
```

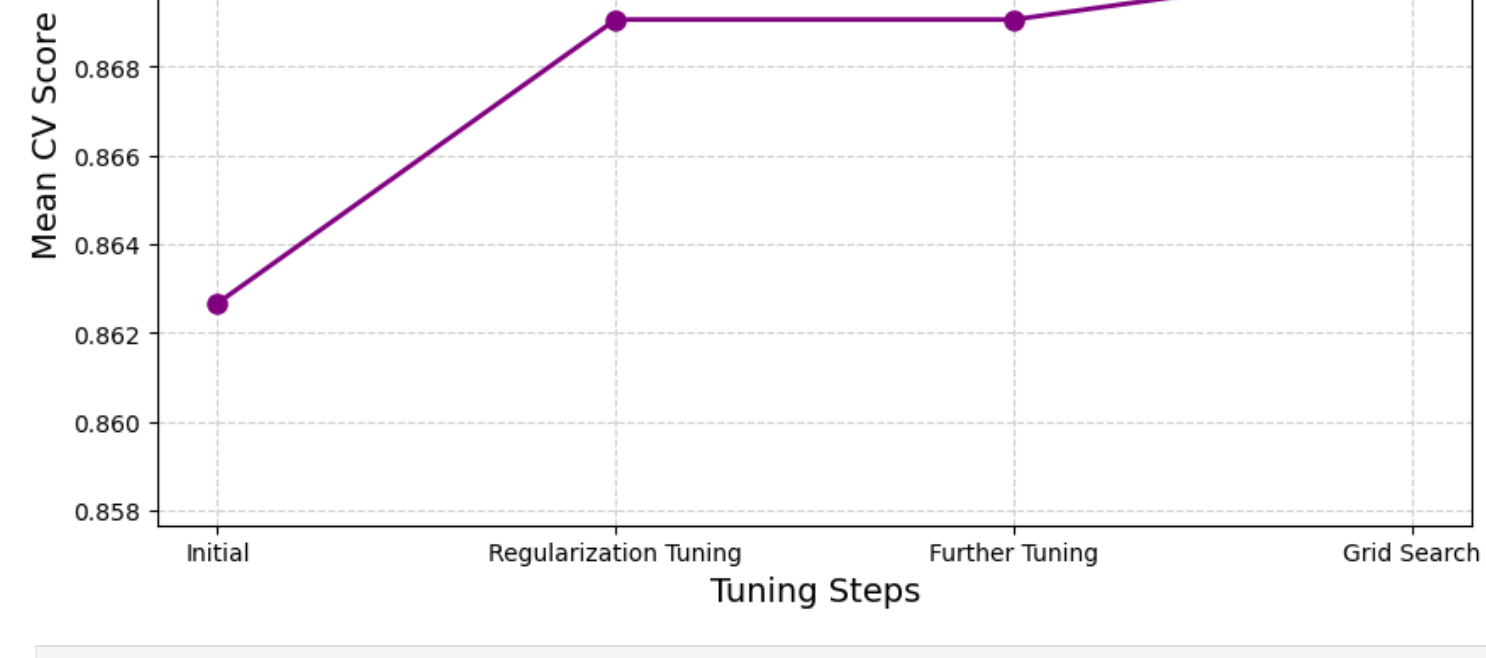
```
In [539… print(f'Best Hyperparameters from Grid Search: {grid_search.best_params_}')

Best Hyperparameters from Grid Search: {'C': 0.01, 'class_weight': 'balanced', 'max_iter': 100, 'penalty': 'l2', 'solver': 'lbfgs', 'tol': 0.0001}
```

```
In [ ]: best_params = pd.DataFrame(grid_search.best_params_, index=[0])
        print(f'Grid Search Best Hyperparameters:\n{best_params}')

Grid Search Best Hyperparameters:
      C class_weight  max_iter penalty solver     tol
0  0.01     balanced       100      l2  lbfgs  0.0001
```

```
In [540… grid_searched_logclf_train_score = accuracy_score(y_train, grid_searched_best_model.predict(X_train))
         print(f'Accuracy Score on Training Set: {grid_searched_logclf_train_score}')
         grid_searched_logclf_test_score = accuracy_score(y_test, grid_searched_best_model.predict(X_test))
         print(f'Accuracy Score on Test Set: {grid_searched_logclf_test_score}')
         grid_searched_logclf_cross_val_scores = cross_val_score(grid_searched_best_model, X_recombined, y_recombined, cv=k_folds)
         print(f'Grid Searched Logistic Regression Cross-Validation Scores: {grid_searched_logclf_cross_val_scores}')
         print(f'Grid Searched Logistic Regression Mean CV Score: {grid_searched_logclf_cross_val_scores.mean()}')

Accuracy Score on Training Set: 0.8780096308186196
Accuracy Score on Test Set: 0.9038461538461539
Grid Searched Logistic Regression Cross-Validation Scores: [0.8974359  0.88461538 0.85897436 0.91025641 0.87179487 0.8974359
 0.91025641 0.79487179 0.82051282 0.85714286]
Grid Searched Logistic Regression Mean CV Score: 0.8703296703296702
```

```
In [541… #compare with previous best tuned model
         print(f'Previous Best Tuned Logistic Regression Mean CV Score: {logclf_tuned_cross_val_scores.mean()}')
         print(f'Grid Searched Logistic Regression Mean CV Score: {grid_searched_logclf_cross_val_scores.mean()}')
         print(f'Improvement in Mean CV Score after Grid Search: {grid_searched_logclf_cross_val_scores.mean() - logclf_tuned_cross_val_scores.mean()}')

Previous Best Tuned Logistic Regression Mean CV Score: 0.869064269064269
Grid Searched Logistic Regression Mean CV Score: 0.8703296703296702
Improvement in Mean CV Score after Grid Search: 0.0012654012654012003
```

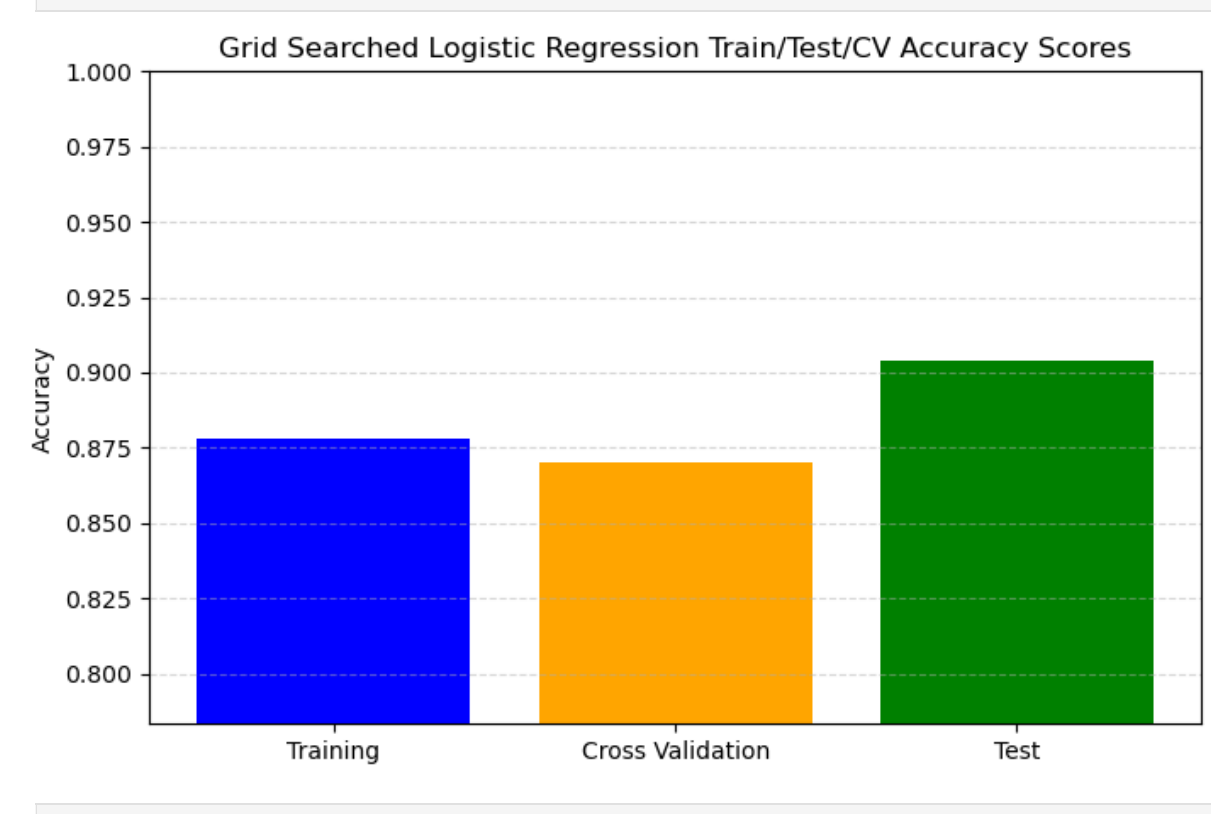
```
In [ ]: import matplotlib.pyplot as plt

        tuning_steps = ['Initial', 'Regularization Tuning', 'Further Tuning', 'Grid Search']
        mean_cv_scores = [
            logclf_cross_val_scores.mean(), # initial
            logclf_tuned_cross_val_scores.mean(), # after manual regularization tuning (experimental)
            logclf_tuned_cross_val_scores_general_performance.mean(), # after further tuning (experimental)
            grid_searched_logclf_cross_val_scores.mean() # after grid search tuning
        ]
        plt.figure(figsize=(10,6))
        plt.plot(tuning_steps, mean_cv_scores, marker='o', linestyle='-', color='purple', linewidth=2, markersize=8)

        plt.title('Improvement in Mean CV Score after Each Tuning Step', fontsize=16)
        plt.xlabel('Tuning Steps', fontsize=14)
        plt.ylabel('Mean CV Score', fontsize=14)

        # Zoom in on the range of scores to make small improvements more visible
        plt.ylim(min(mean_cv_scores)-0.005, max(mean_cv_scores)+0.005) #(ChatGPT, 2025)

        plt.grid(True, linestyle='--', alpha=0.6)
        plt.show()
```



```
In [543… print("Logistic Regression Hyperparameter Tuning Summary:")
         print(f'Initial Mean CV Score: {logclf_cross_val_scores.mean()}')
         print(f'After Regularization Tuning Mean CV Score: {logclf_tuned_cross_val_scores.mean()}')
         print(f'After Further Tuning Mean CV Score: {logclf_tuned_cross_val_scores_general_performance.mean()}')
         print(f'After Grid Search Mean CV Score: {grid_searched_logclf_cross_val_scores.mean()}')
         overfit_underfit_analysis(grid_searched_logclf_train_score, grid_searched_logclf_cross_val_scores.mean())
```

```
Logistic Regression Hyperparameter Tuning Summary:
Initial Mean CV Score: 0.8626706626706626
After Regularization Tuning Mean CV Score: 0.869064269064269
After Further Tuning Mean CV Score: 0.869064269064269
After Grid Search Mean CV Score: 0.8703296703296702
Training Score: 0.8780096308186196
Cross Validation Score: 0.8703296703296702
The model has a good fit.
```

In [625…
```python
import matplotlib.pyplot as plt

scores = [
    grid_searched_logclf_train_score,
    grid_searched_logclf_cross_val_scores.mean(),
    grid_searched_logclf_test_score
]

labels = ['Training', 'Cross Validation', 'Test']

plt.figure(figsize=(8,5))
plt.bar(labels, scores, color=['blue', 'orange', 'green'])
plt.ylim(0.9 * min(scores), 1)# (ChatGPT, 2025)
plt.ylabel("Accuracy")
plt.title("Grid Searched Logistic Regression Train/Test/CV Accuracy Scores")
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()
```



In [545…
```python
logclf_top_perf = grid_searched_best_model #top preforming model after hyperparameter tuning (identical to previous tuned model)
```

In [546…
```python
with open('logistic_regression_best_cv_score.txt', 'w') as f:
    f.write(f'Best Logistic Regression Mean CV Score: {grid_searched_logclf_cross_val_scores.mean()}')
    for i, score in enumerate(grid_searched_logclf_cross_val_scores):
        f.write(f'\nFold {i+1} CV Score: {score}')
    f.close()
```

Model 2: Gradient Boosting Classifier

In [547…
```python
from sklearn.ensemble import GradientBoostingClassifier
gbc = GradientBoostingClassifier()
gbc.fit(X_train, y_train)
```

Out[547…
```
▾ GradientBoostingClassifier
▸ Parameters
```

In [548…
```python
gbc_train_score = gbc.score(X_train, y_train)
print(f'Accuracy Score on Training Set: {gbc_train_score}')
```

```
Accuracy Score on Training Set: 0.985553772070626
```

In [584…
```python
gbc_test_score = gbc.score(X_test, y_test)
print(f'Accuracy Score on Test Set: {gbc_test_score}')
```

```
Accuracy Score on Test Set: 0.8846153846153846
```

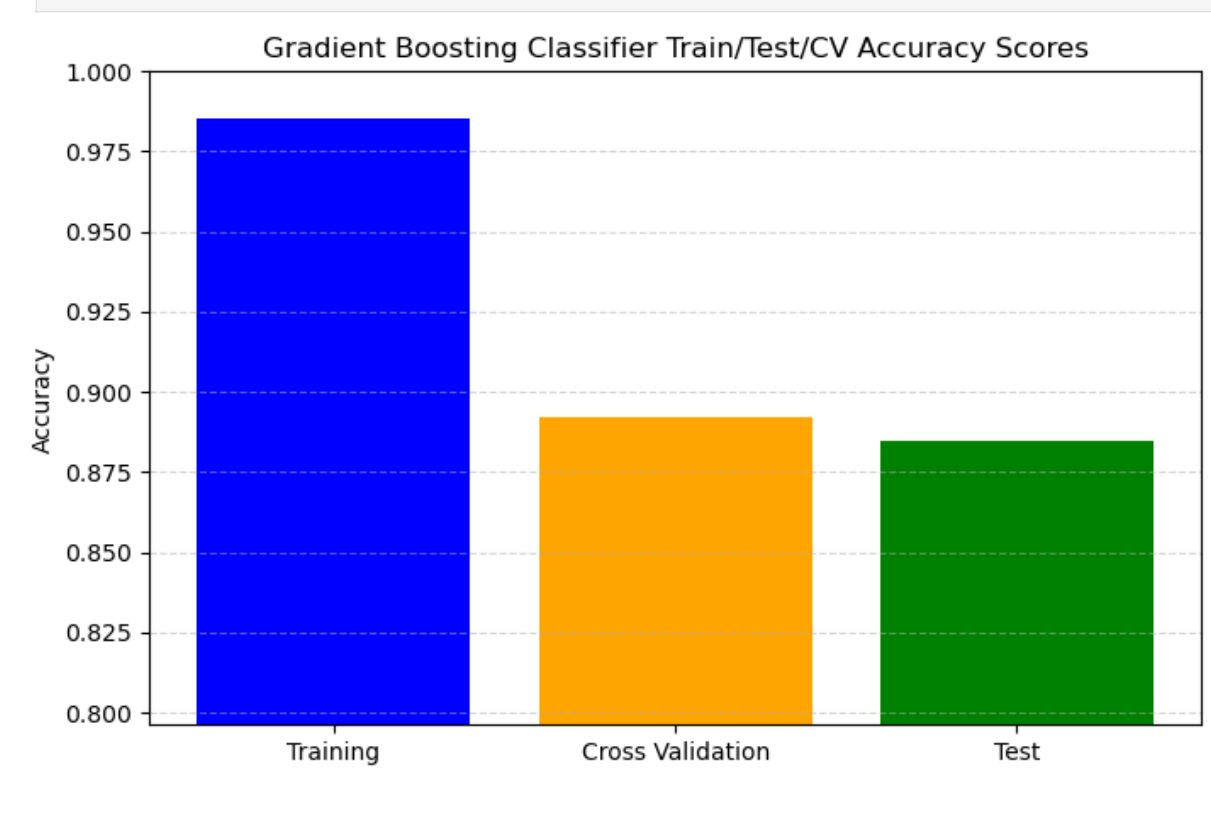
In [550…
```python
predictions_gbc = gbc.predict(X_test)
gbc_cross_val_scores = cross_val_score(gbc, X_recombined, y_recombined, cv=k_folds)
print(f'Gradient Boosting Classifier Cross-Validation Scores: {gbc_cross_val_scores}')
print(f'Gradient Boosting Classifier Mean CV Score: {gbc_cross_val_scores.mean()}')
```

```
Gradient Boosting Classifier Cross-Validation Scores: [0.8974359  0.8974359  0.83333333 0.8974359  0.94871795 0.94871795
 0.94871795 0.80769231 0.84615385 0.8961039 ]
Gradient Boosting Classifier Mean CV Score: 0.8921744921744921
```

In [551…
```python
overfit_underfit_analysis(gbc_train_score, gbc_cross_val_scores.mean())
```

```
Training Score: 0.985553772070626
Cross Validation Score: 0.8921744921744921
The model is overfitting.
Difference: 0.09337927989613393
```

In [621…
```python
import matplotlib.pyplot as plt

scores = [
    gbc_train_score,
    gbc_cross_val_scores.mean(),
    gbc_test_score
]

labels = ['Training', 'Cross Validation', 'Test']

plt.figure(figsize=(8,5))
plt.bar(labels, scores, color=['blue', 'orange', 'green'])
plt.ylim(0.9 * min(scores), 1)# (ChatGPT, 2025)
plt.ylabel("Accuracy")
plt.title("Gradient Boosting Classifier Train/Test/CV Accuracy Scores")
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()
```



Hyperparameter Tuning for Gradient Boosting Classifier

Our analysis of the gradient boosting classifier has revealed some considerable overfitting, as indicated by the higher training score compared to cross-validation score. To combat this, I will explore hyperparameter tuning to improve model generalization. This will require changes to tree depth and number of estimators (number of trees in our ensemble). This will help us find the optimal balance between bias and variance, allowing for better performance on unseen data. I will use a grid search to systematically explore combinations of these hyperparameters and identify the best configuration for our model (figure 20).

Hyperparameter	What it does	Effect on our Overfitting problem
`max_depth`	Limits the maximum depth of each tree. Deeper trees capture more complex patterns but are more likely to overfit.	Shallower → reduces overfitting.
`max_leaf_nodes`	Sets the maximum number of leaf nodes per tree. More leaves allow for more complex patterns, increasing overfitting risk.	Fewer leaves → reduces overfitting.
`min_samples_leaf`	Minimum number of samples required at a leaf node. Higher values prevent very small leaves, reducing overfitting.	Higher → reduces overfitting.
`min_samples_split`	Minimum number of samples required to split a node. Larger values prevent splits on tiny nodes, reducing overfitting.	Higher → reduces overfitting.
`learning_rate`	Shrinks contribution of each tree in the ensemble. Smaller values slow learning but reduce overfitting.	Smaller → reduces overfitting.
`n_estimators`	Number of boosting iterations (trees). More trees can increase complexity and overfitting, especially with high learning rates.	Fewer trees → reduces overfitting.

Figure 20: Gradient Boosting Classifier Hyperparameter Explanations

Tree depth and number of estimators tuning

In [553…
```python
hyperparameters = {
    'max_depth': 6, #limit tree depth to reduce overfitting
    'n_estimators': 150,#reduce number of trees to prevent overfitting
}

# Previous hyperparameter attempts:
# hyperparameters = {
#     'max_depth': 5,
#     'n_estimators': 200,
# }
# hyperparameters = {
#     'max_depth': 4,
#     'n_estimators': 100,
# }
# hyperparameters = {
#     'max_depth': 3,
#     'n_estimators': 200,
# }
# hyperparameters = {
#     'max_depth': 7,
#     'n_estimators': 200,
# }
```

In [554…
```python
gbc_tuned = GradientBoostingClassifier(**hyperparameters)
gbc_tuned.fit(X_train, y_train)
```

Out[554…
```
▾ GradientBoostingClassifier
▸ Parameters
```

In [555…
```python
gbc_tuned_train_score = gbc_tuned.score(X_train, y_train)
print(f'Accuracy Score on Training Set: {gbc_tuned_train_score}')
gbc_tuned_test_score = accuracy_score(y_test, gbc_tuned.predict(X_test))
print(f'Accuracy Score on Test Set: {gbc_tuned_test_score}')
```

```
Accuracy Score on Training Set: 1.0
Accuracy Score on Test Set: 0.8910256410256411
```

In [556…
```python
gbc_tuned_cross_val_scores = cross_val_score(gbc_tuned, X_recombined, y_recombined, cv=k_folds)
print(f'Gradient Boosting Classifier Tuned Cross-Validation Scores: {gbc_tuned_cross_val_scores}')
print(f'Gradient Boosting Classifier Tuned Mean CV Score: {gbc_tuned_cross_val_scores.mean()}')
```

```
Gradient Boosting Classifier Tuned Cross-Validation Scores: [0.8974359  0.91025641 0.80769231 0.88461538 0.92307692 0.91025641
 0.94871795 0.85897436 0.85897436 0.90909091]
Gradient Boosting Classifier Tuned Mean CV Score: 0.8909090909090909
```

In [557…
```python
overfit_underfit_analysis(gbc_tuned_train_score, gbc_tuned_cross_val_scores.mean())
```

```
Training Score: 1.0
Cross Validation Score: 0.8909090909090909
The model is overfitting.
Difference: 0.10909090909090913
```

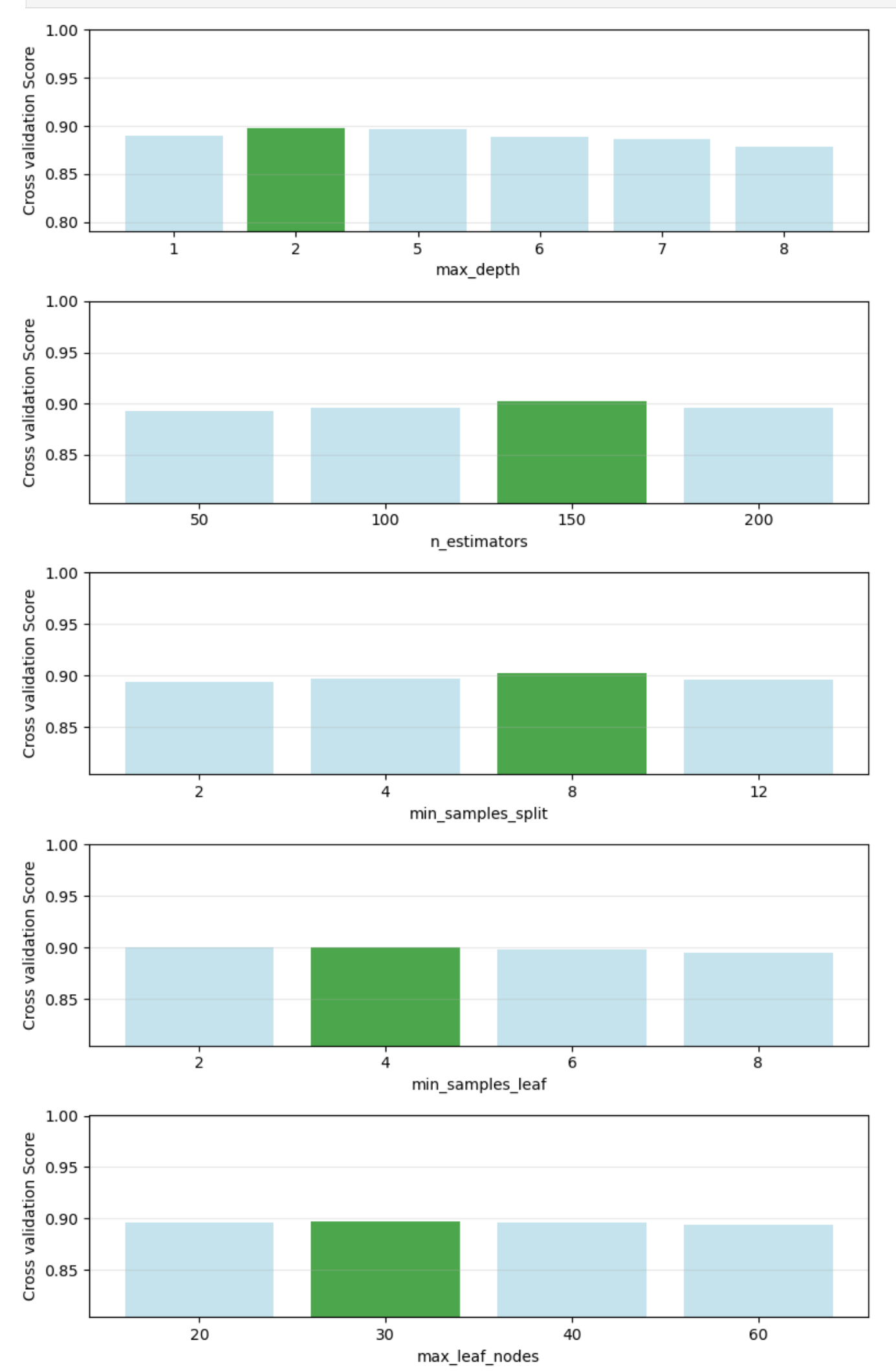
In [558…
```python
print(f'Previous Gradient Boosting Classifier Mean CV Score: {gbc_cross_val_scores.mean()}')
print(f'Tuned Gradient Boosting Classifier Mean CV Score: {gbc_tuned_cross_val_scores.mean()}')
print(f'Improvement in Mean CV Score after Tuning: {gbc_tuned_cross_val_scores.mean() - gbc_cross_val_scores.mean()}')
```

```
Previous Gradient Boosting Classifier Mean CV Score: 0.8921744921744921
Tuned Gradient Boosting Classifier Mean CV Score: 0.8909090909090909
Improvement in Mean CV Score after Tuning: -0.0012654012654012003
```

Hyperparameter Optimization and Visualization

In [668…
```python
hyperparameters = { #
    'max_depth':[1,2,5,6,7,8],
    'n_estimators':[50,100,150,200],
    'min_samples_split':[2,4,8,12],
    'min_samples_leaf':[2,4,6,8],
    'max_leaf_nodes':[20,30,40,60],
    'learning_rate':[0.01,0.05,0.1, 0.5]
}

results = {}
for param, values in hyperparameters.items(): #Loop through each hyperparameter, to see its effect individually
    scores = []
    for val in values:
        params = {param: None}
        params[param] = val
        model = GradientBoostingClassifier(**params)
        cv = cross_val_score(model, X_recombined, y_recombined, cv=k_folds)
        scores.append(cv.mean())
    results[param] = (values, scores)
```

In [669…
```python
fig, axes = plt.subplots(6, 1, figsize=(8, 15))

for i, param in enumerate(hyperparameters.keys()):
    scores = results[param][1]
    best_idx = scores.index(max(scores))
    colors = ['green' if j == best_idx else 'lightblue' for j in range(len(scores))] # (ChatGPT, 2025)
    axes[i].bar(range(len(results[param][0])), results[param][1], color=colors, alpha=0.7)
    axes[i].set_xlabel(param)
    axes[i].set_ylabel('Cross validation Score')
    axes[i].set_xticks(range(len(results[param][0])))
    axes[i].set_xticklabels(results[param][0])
    axes[i].set_ylim(0.9 * min(results[param][1]), 1)
    axes[i].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



Grid Search for Hyperparameter Optimization

In [ ]:
```python
grid_search_gbc = GridSearchCV(GradientBoostingClassifier(), hyperparameters, cv=k_folds)
grid_search_gbc.fit(X_recombined, y_recombined)
grid_searched_best_model_gbc = grid_search_gbc.best_estimator_ # Uses cross validation to find best model hyperparameters
```

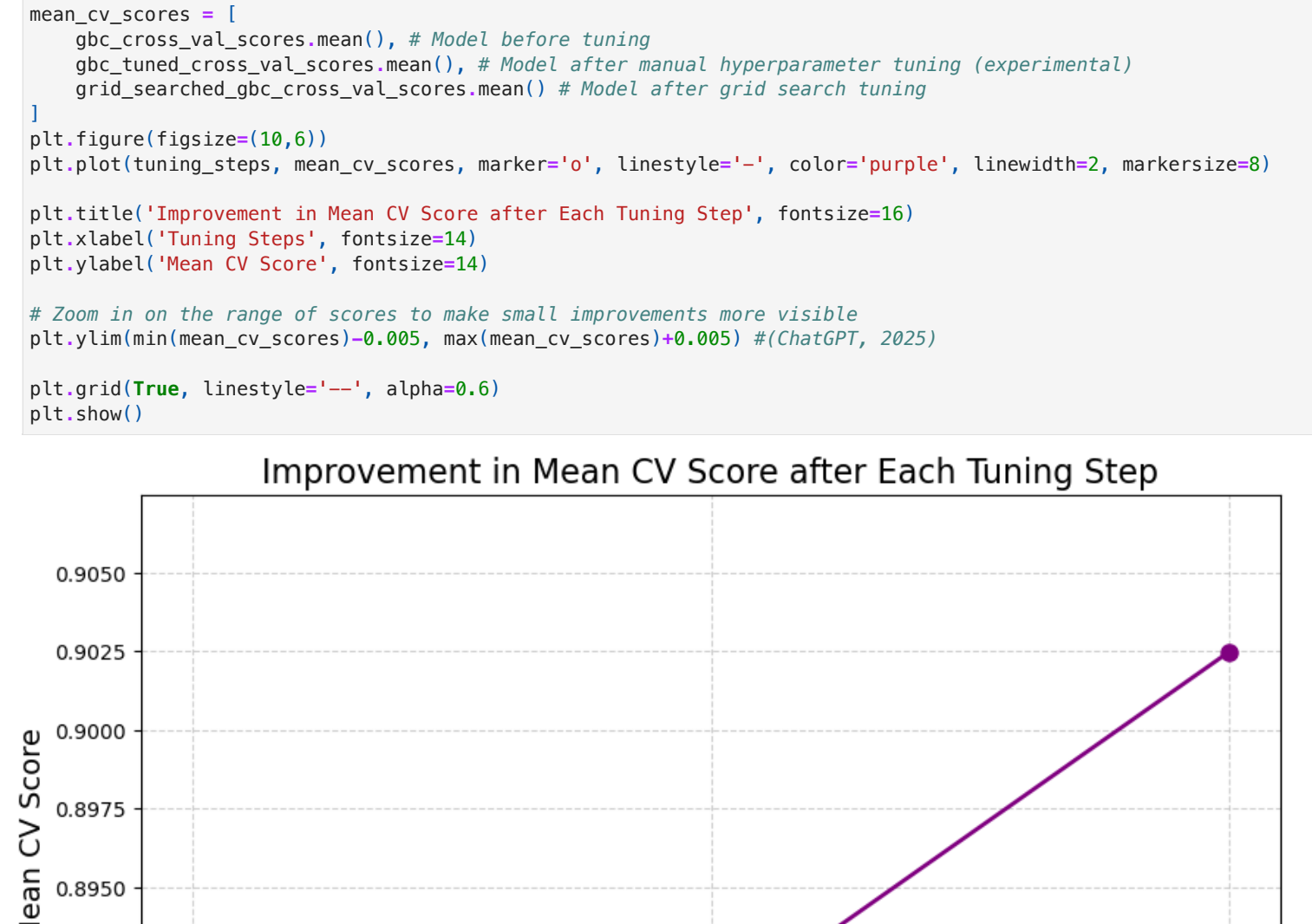
In [602…
```python
print(f'Best Hyperparameters from Grid Search: {grid_search_gbc.best_params_}')
grid_search_gbc_train_score = grid_searched_best_model_gbc.score(X_train, y_train)
print(f'Accuracy Score on Training Set: {grid_search_gbc_train_score}')
grid_search_gbc_test_score = accuracy_score(y_test, grid_searched_best_model_gbc.predict(X_test))
print(f'Accuracy Score on Test Set: {grid_search_gbc_test_score}')
grid_searched_gbc_cross_val_scores = cross_val_score(grid_searched_best_model_gbc, X_recombined, y_recombined, cv=k_folds)
print(f'Grid Searched Gradient Boosting Classifier Cross-Validation Scores: {grid_searched_gbc_cross_val_scores}')
print(f'Grid Searched Gradient Boosting Classifier Mean CV Score: {grid_searched_gbc_cross_val_scores.mean()}')
overfit_underfit_analysis(grid_search_gbc_train_score, grid_searched_gbc_cross_val_scores.mean())
```

```
Best Hyperparameters from Grid Search: {'learning_rate': 0.1, 'max_depth': 8, 'max_leaf_nodes': 40, 'min_samples_leaf': 4, 'min_samples_split': 8, 'n_estimators': 150}
Accuracy Score on Training Set: 1.0
Accuracy Score on Test Set: 0.9230769230769231
Grid Searched Gradient Boosting Classifier Cross-Validation Scores: [0.91025641 0.93589744 0.83333333 0.91025641 0.92307692 0.94871795
 0.94871795 0.83333333 0.85897436 0.92207792]
Grid Searched Gradient Boosting Classifier Mean CV Score: 0.9024642024642023
Training Score: 1.0
Cross Validation Score: 0.9024642024642023
The model is overfitting.
Difference: 0.09753579753579766
```

In [ ]:
```python
best_params = pd.DataFrame(grid_search_gbc.best_params_, index=[0])
print(f'Grid Search Best Hyperparameters:\n{best_params}')
```

```
Grid Search Best Hyperparameters:
   learning_rate  max_depth  max_leaf_nodes  min_samples_leaf  \
0            0.1          8              40                 4

   min_samples_split  n_estimators
0                  8           150
```

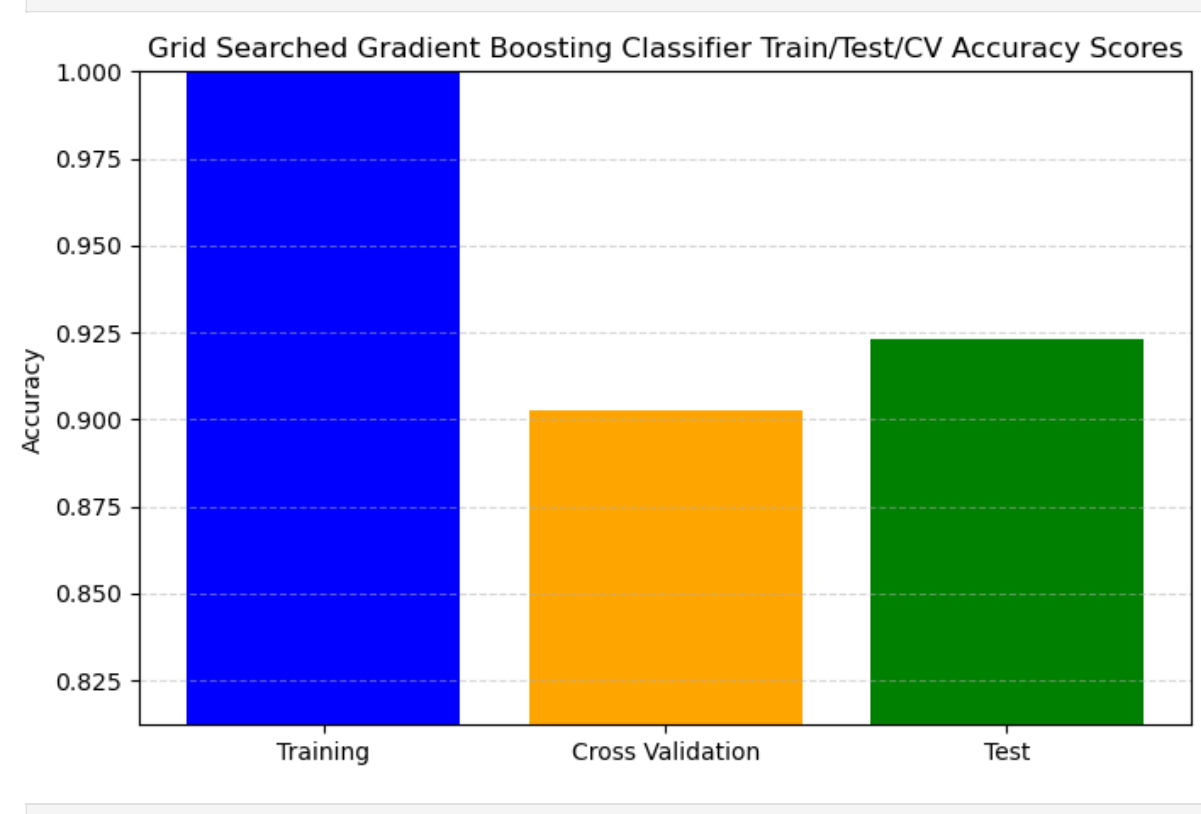
In [ ]:
```python
import matplotlib.pyplot as plt

tuning_steps = ['Initial', 'Regularization Tuning', 'Grid Search']
mean_cv_scores = [
    gbc_cross_val_scores.mean(), # Model before tuning
    gbc_tuned_cross_val_scores.mean(), # Model after manual hyperparameter tuning (experimental)
    grid_searched_gbc_cross_val_scores.mean() # Model after grid search tuning
]
plt.figure(figsize=(10,6))
plt.plot(tuning_steps, mean_cv_scores, marker='o', linestyle='-', color='purple', linewidth=2, markersize=8)

plt.title('Improvement in Mean CV Score after Each Tuning Step', fontsize=16)
plt.xlabel('Tuning Steps', fontsize=14)
plt.ylabel('Mean CV Score', fontsize=14)

# Zoom in on the range of scores to make small improvements more visible
plt.ylim(min(mean_cv_scores)-0.005, max(mean_cv_scores)+0.005) #(ChatGPT, 2025)

plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



In [604…
```python
print(f'Previous Best Tuned Gradient Boosting Classifier Mean CV Score: {gbc_tuned_cross_val_scores.mean()}')
print(f'Grid Searched Gradient Boosting Classifier Mean CV Score: {grid_searched_gbc_cross_val_scores.mean()}')
print(f'Improvement in Mean CV Score after Grid Search: {grid_searched_gbc_cross_val_scores.mean() - gbc_tuned_cross_val_scores.mean()}')
```

```
Previous Best Tuned Gradient Boosting Classifier Mean CV Score: 0.8909090909090909
Grid Searched Gradient Boosting Classifier Mean CV Score: 0.9024642024642023
Improvement in Mean CV Score after Grid Search: 0.011555111555111464
```

In [623…
```python
import matplotlib.pyplot as plt

scores = [
    grid_search_gbc_train_score,
    grid_searched_gbc_cross_val_scores.mean(),
    grid_search_gbc_test_score
]

labels = ['Training', 'Cross Validation', 'Test']

plt.figure(figsize=(8,5))
plt.bar(labels, scores, color=['blue', 'orange', 'green'])
plt.ylim(0.9 * min(scores), 1)# (ChatGPT, 2025)
plt.ylabel("Accuracy")
plt.title("Grid Searched Gradient Boosting Classifier Train/Test/CV Accuracy Scores")
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()
```



In [606…
```python
gbc_top_perf = grid_searched_best_model_gbc #top preforming model after hyperparameter tuning
```

In [607…
```python
with open('gradient_decsion_tree_scores.txt', 'w') as f:
    f.write(f'Best Gradient Boosting Classifier Mean CV Score: {grid_searched_gbc_cross_val_scores.mean()}')
    for i, score in enumerate(grid_searched_gbc_cross_val_scores):
        f.write(f'\nFold {i+1} CV Score: {score}')
    f.close()
```

Results and Evaluation

In [614…
```python
print('Results Summary for Top Performing Models:')
print('\n')
print('Logistic Regression Classifier:')
print(f"Training Score: {grid_searched_logclf_train_score}")
print(f" Test Score: {grid_searched_logclf_test_score}")
print(f" Mean CV Score: {grid_searched_logclf_cross_val_scores.mean()}")
print('\n')
print('Gradient Boosting Classifier:')
print(f"Training Score: {grid_search_gbc_train_score}")
print(f" Test Score: {grid_search_gbc_test_score}")
print(f" Mean CV Score: {grid_searched_gbc_cross_val_scores.mean()}")
mean_cv_score_difference = grid_searched_logclf_cross_val_scores.mean() - grid_searched_gbc_cross_val_scores.mean()
```

```
Results Summary for Top Performing Models:


Logistic Regression Classifier:
Training Score: 0.8780096308186196
 Test Score: 0.9038461538461539
 Mean CV Score: 0.8703296703296702


Gradient Boosting Classifier:
Training Score: 1.0
 Test Score: 0.9230769230769231
 Mean CV Score: 0.9024642024642023
```

In [609…
```python
if mean_cv_score_difference > 0:
    print(f'Logistic Regression outperforms Gradient Boosting Classifier by {mean_cv_score_difference} on cross validation.')
else:
    print(f'Gradient Boosting Classifier outperforms Logistic Regression by {-mean_cv_score_difference} om cross validation.')
```

```
Gradient Boosting Classifier outperforms Logistic Regression by 0.0321345321345321 om cross validation.
```

In [610…
```python
print('Overfit/Underfit Analysis for Top Performing Models:')
print("Logistic Regression Overfit/Underfit Analysis:")
overfit_underfit_analysis(grid_searched_logclf_train_score, grid_searched_logclf_cross_val_scores.mean())
print ("\n")
print("Gradient Boosting Classifier Overfit/Underfit Analysis:")
overfit_underfit_analysis(grid_search_gbc_train_score, grid_searched_gbc_cross_val_scores.mean())
```

```
Overfit/Underfit Analysis for Top Performing Models:
Logistic Regression Overfit/Underfit Analysis:
Training Score: 0.8780096308186196
Cross Validation Score: 0.8703296703296702
The model has a good fit.


Gradient Boosting Classifier Overfit/Underfit Analysis:
Training Score: 1.0
Cross Validation Score: 0.9024642024642023
The model is overfitting.
Difference: 0.09753579753579766
```

Training, Validation, and Test Performance comparison

In [617…
```python
import numpy as np
labels = ['Training Score', 'Test Score', 'Mean CV Score']
logclf_scores = [grid_searched_logclf_train_score, grid_searched_logclf_test_score, grid_searched_logclf_cross_val_scores.mean()]
gbc_scores = [grid_search_gbc_train_score, grid_search_gbc_test_score, grid_searched_gbc_cross_val_scores.mean()]
x = np.arange(len(labels))  # the label locations
width = 0.35  # the width of the bars
fig, ax = plt.subplots(figsize=(10,6))
bars1 = ax.bar(x - width/2, logclf_scores, width, label='Logistic Regression', color='blue') #ChatGPT, 2025
bars2 = ax.bar(x + width/2, gbc_scores, width, label='Gradient Boosting Classifier', color='orange') #ChatGPT, 2025
ax.set_ylabel('Scores', fontsize=14)
ax.set_title('Comparison of Top Performing Models', fontsize=16)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontsize=12)
ax.legend(fontsize=7)
def autolabel(bars):
    """Attach a text label above each bar in *bars*, displaying its height."""
    for bar in bars: #ChatGPT, 2025
        height = bar.get_height()
        ax.annotate(f'{height:.4f}',
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3),  # 3 points vertical offset
                    textcoords="offset points",
                    ha='center', va='bottom', fontsize=10)
autolabel(bars1)
autolabel(bars2)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

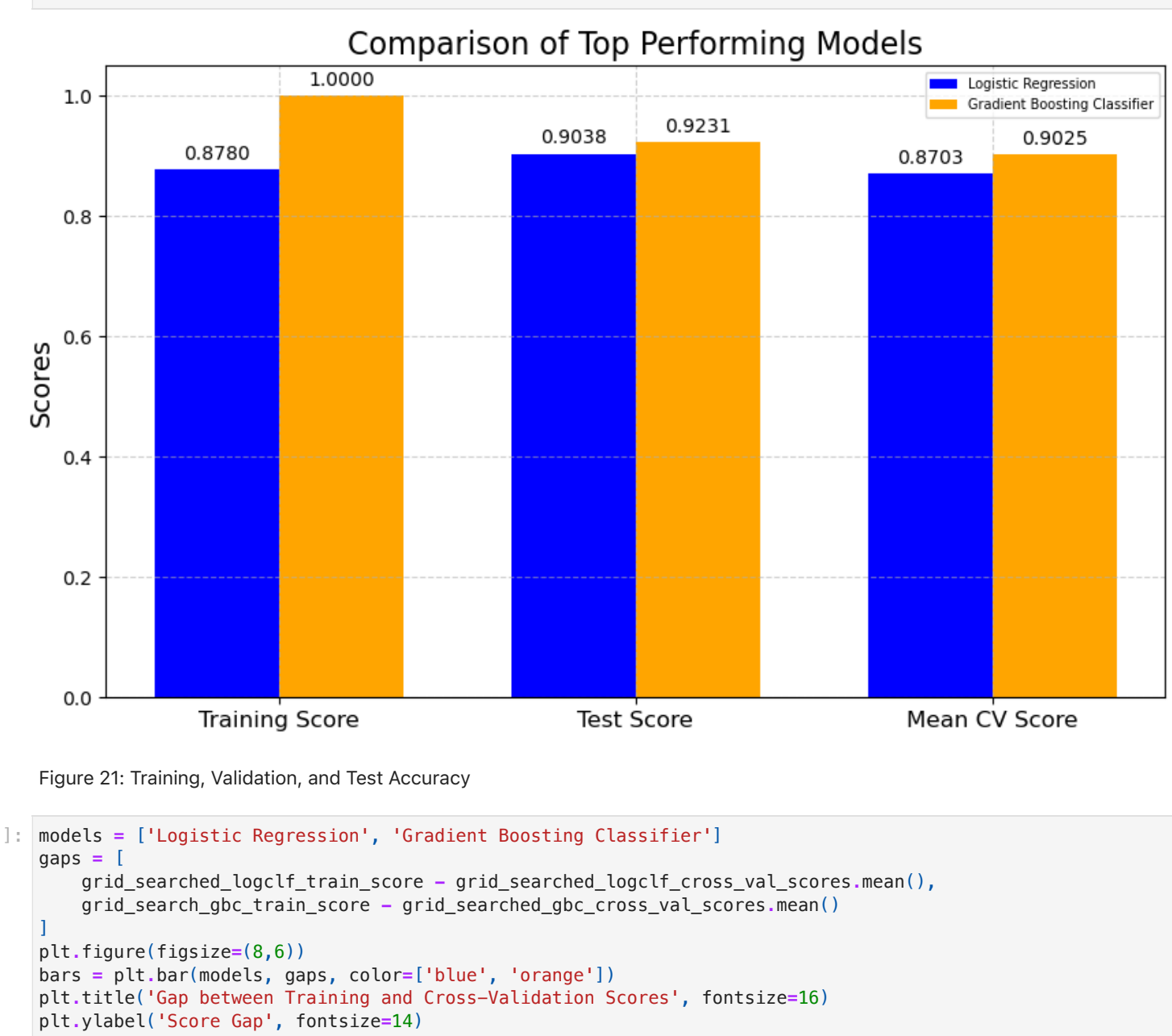


Figure 21: Training, Validation, and Test Accuracy

In [ ]:
```python
models = ['Logistic Regression', 'Gradient Boosting Classifier']
gaps = [
    grid_searched_logclf_train_score - grid_searched_logclf_cross_val_scores.mean(),
    grid_search_gbc_train_score - grid_searched_gbc_cross_val_scores.mean()
]
plt.figure(figsize=(8,6))
bars = plt.bar(models, gaps, color=['blue', 'orange'])
plt.title('Gap between Training and Cross-Validation Scores', fontsize=16)
plt.ylabel('Score Gap', fontsize=14)
for bar in bars: #ChatGPT, 2025
    height = bar.get_height()
    plt.annotate(f'{height:.4f}',
                 xy=(bar.get_x() + bar.get_width() / 2, height),
                 xytext=(0, 3),  # 3 points vertical offset
                 textcoords="offset points",
                 ha='center', va='bottom', fontsize=10)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

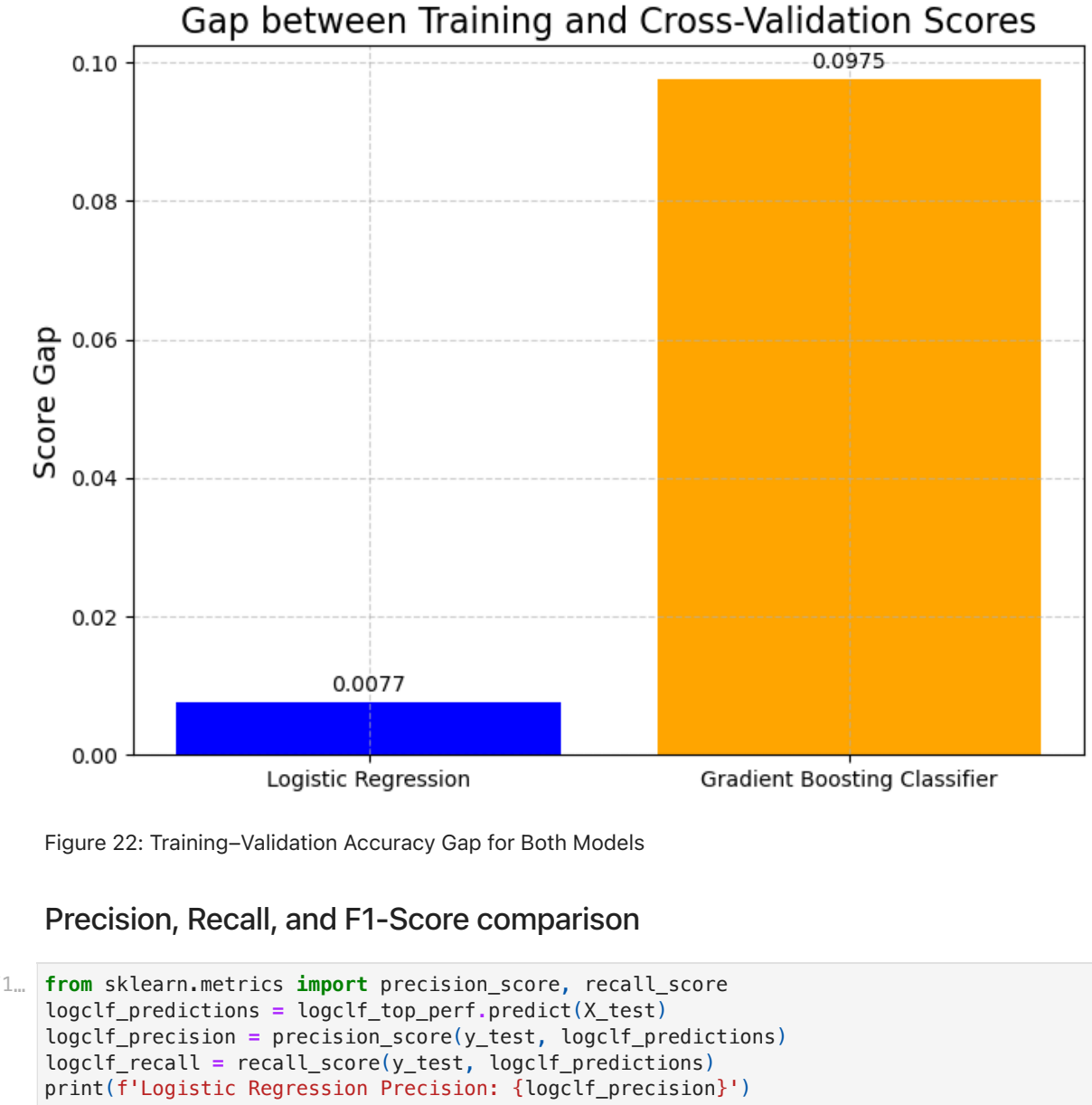


Figure 22: Training–Validation Accuracy Gap for Both Models

Precision, Recall, and F1-Score comparison

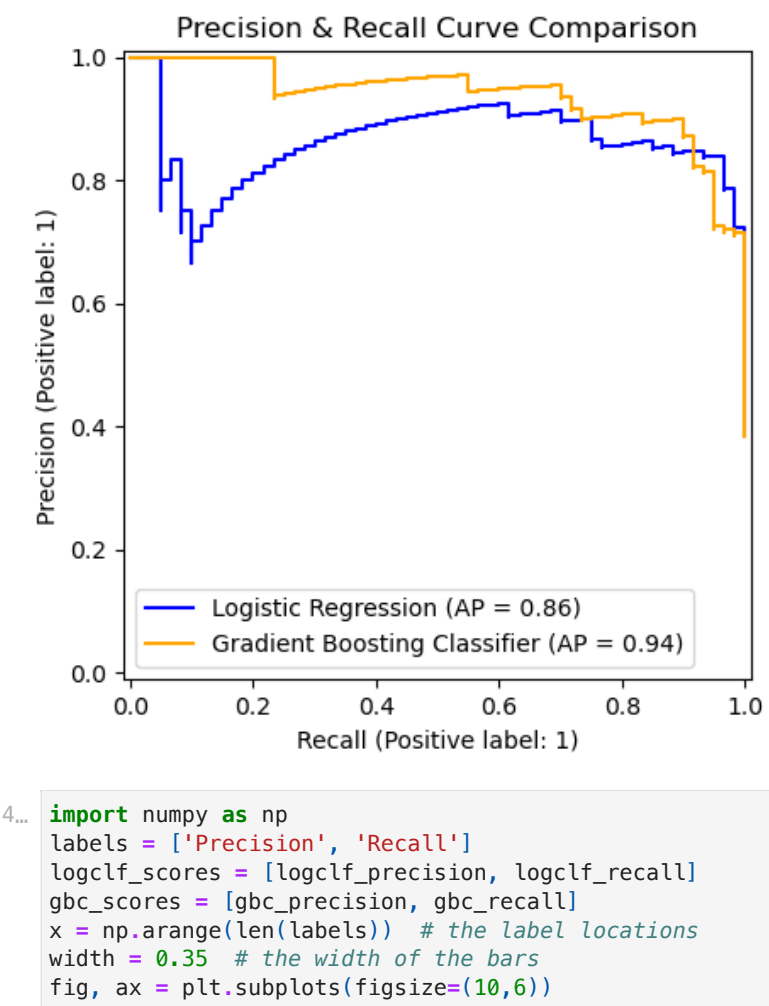
In [571…
```python
from sklearn.metrics import precision_score, recall_score
logclf_predictions = logclf_top_perf.predict(X_test)
logclf_precision = precision_score(y_test, logclf_predictions)
logclf_recall = recall_score(y_test, logclf_predictions)
print(f'Logistic Regression Precision: {logclf_precision}')
print(f'Logistic Regression Recall: {logclf_recall}')
```

```
Logistic Regression Precision: 0.8169014084507042
Logistic Regression Recall: 0.9666666666666667
```

In [572…
```python
gbc_predictions = gbc_top_perf.predict(X_test)
gbc_precision = precision_score(y_test, gbc_predictions)
gbc_recall = recall_score(y_test, gbc_predictions)
print(f'Gradient Boosting Classifier Precision: {gbc_precision}')
print(f'Gradient Boosting Classifier Recall: {gbc_recall}')
```

```
Gradient Boosting Classifier Precision: 0.9
Gradient Boosting Classifier Recall: 0.9
```

In [573…
```python
from sklearn.metrics import PrecisionRecallDisplay
logclf_disp = PrecisionRecallDisplay.from_estimator(logclf_top_perf, X_test, y_test, name="Logistic Regression", color='blue')
gbc_disp = PrecisionRecallDisplay.from_estimator(gbc_top_perf, X_test, y_test, name="Gradient Boosting Classifier", color='orange', ax=logclf_disp.ax_)
logclf_disp.ax_.set_title('Precision & Recall Curve Comparison')
plt.show()
```



In [574…
```python
import numpy as np
labels = ['Precision', 'Recall']
logclf_scores = [logclf_precision, logclf_recall]
gbc_scores = [gbc_precision, gbc_recall]
x = np.arange(len(labels))  # the label locations
width = 0.35  # the width of the bars
fig, ax = plt.subplots(figsize=(10,6))
bars1 = ax.bar(x - width/2, logclf_scores, width, label='Logistic Regression', color='blue') #ChatGPT, 2025
bars2 = ax.bar(x + width/2, gbc_scores, width, label='Gradient Boosting Classifier', color='orange') #ChatGPT, 2025

# Add some text for labels, title and custom x-axis tick labels, etc.
ax.set_ylabel('Scores', fontsize=14)
ax.set_title('Precision and Recall Comparison of Top Performing Models', fontsize=16)
ax.set_xticks(x)
ax.set_xticklabels(labels, fontsize=12)
ax.legend(fontsize=12)
# Attach a text label above each bar in *bars*, displaying its height.
def autolabel(bars):
    """Attach a text label above each bar in *bars*, displaying its height."""
    for bar in bars: ##ChatGPT, 2025
        height = bar.get_height()
        ax.annotate(f'{height:.4f}',
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3),  # 3 points vertical offset
                    textcoords="offset points",
                    ha='center', va='bottom', fontsize=10)
autolabel(bars1)
autolabel(bars2)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

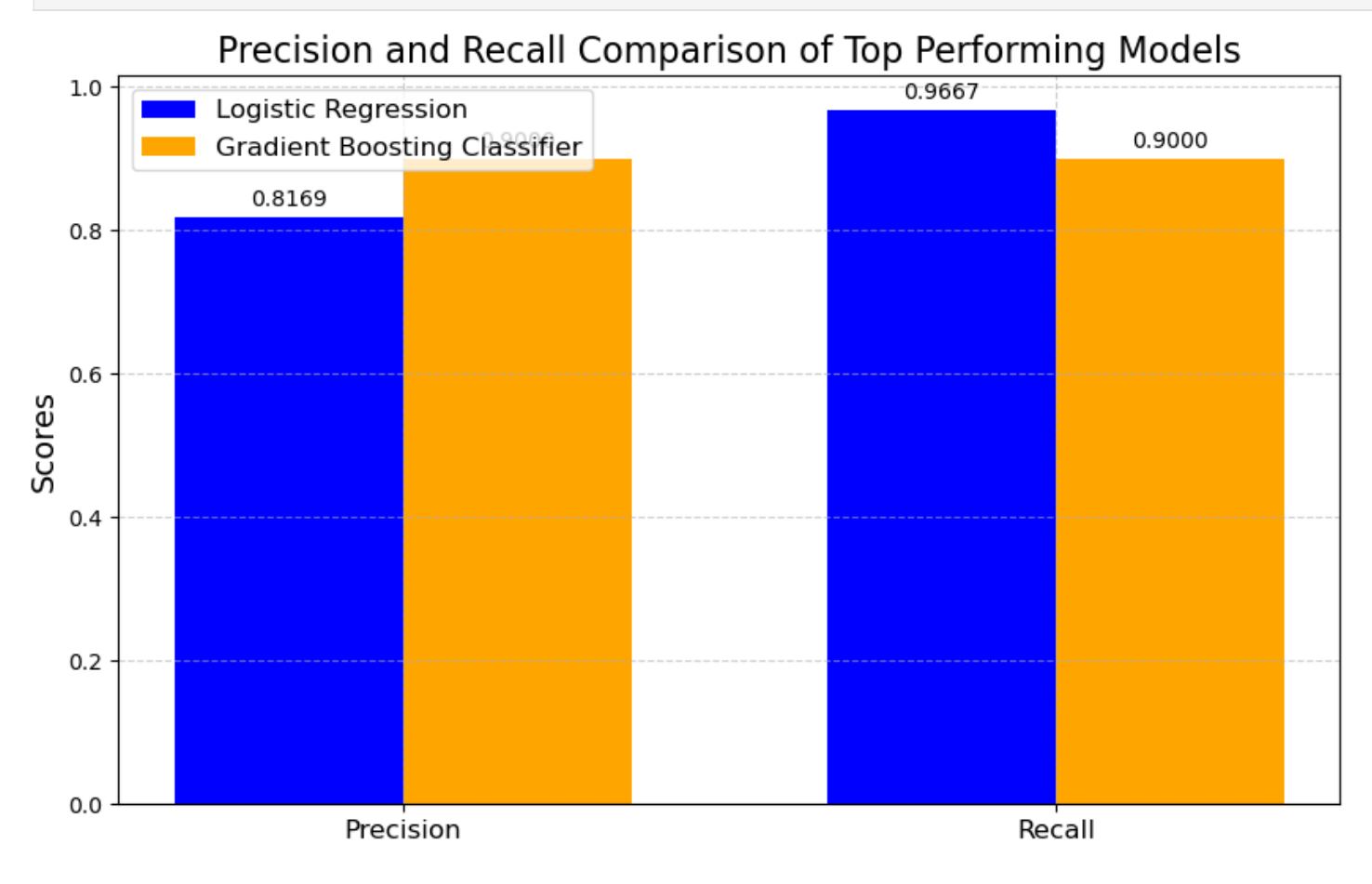


Figure 24: Precision and Recall for Both Models

In [575…
```python
from sklearn.metrics import f1_score
logclf_f1 = f1_score(y_test, logclf_predictions)
gbc_f1 = f1_score(y_test, gbc_predictions)
print(f'Logistic Regression F1 Score: {logclf_f1}')
print(f'Gradient Boosting Classifier F1 Score: {gbc_f1}')
```

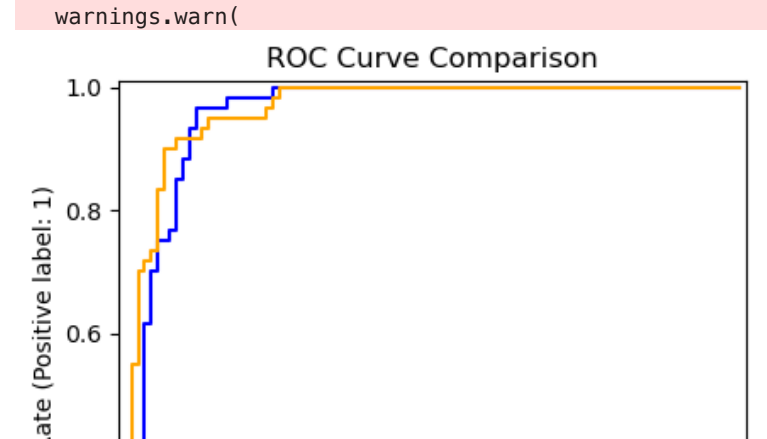
```
Logistic Regression F1 Score: 0.8854961832061069
Gradient Boosting Classifier F1 Score: 0.9
```

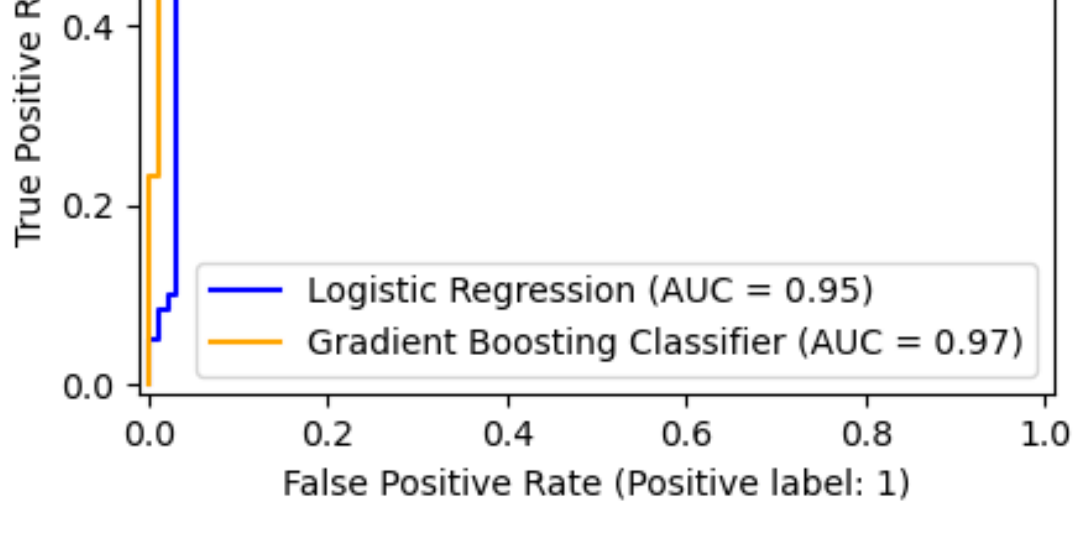
figure 25: F1 scores for both models

ROC and AUC performance comparison

In [576…
```python
from sklearn.metrics import RocCurveDisplay
logclf_roc_disp = RocCurveDisplay.from_estimator(logclf_top_perf, X_test, y_test, name="Logistic Regression", color='blue')
gbc_roc_disp = RocCurveDisplay.from_estimator(gbc_top_perf, X_test, y_test, name="Gradient Boosting Classifier", color='orange', ax=logclf_roc_disp.ax_)
logclf_roc_disp.ax_.set_title('ROC Curve Comparison')
plt.show()
```

```
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/utils/_plotting.py:175: FutureWarning: `**kwargs` is deprecated and will be removed in 1.9. Pass all matplotlib arguments to `curve_kwargs` as a dictio
nary instead.
  warnings.warn(
/opt/anaconda3/envs/TsunamiRiskModelConda/lib/python3.12/site-packages/sklearn/utils/_plotting.py:175: FutureWarning: `**kwargs` is deprecated and will be removed in 1.9. Pass all matplotlib arguments to `curve_kwargs` as a dictio
nary instead.
  warnings.warn(
```





In [577…

```
from sklearn.metrics import roc_auc_score
logclf_auc_roc = roc_auc_score(y_test, logclf_predictions)
print(f'Logistic Regression AUC ROC: {logclf_auc_roc}')
gbc_auc_roc = roc_auc_score(y_test, gbc_predictions)
print(f'Gradient Boosting Classifier AUC ROC: {gbc_auc_roc}')
```

```
Logistic Regression AUC ROC: 0.9156250000000001
Gradient Boosting Classifier AUC ROC: 0.91875
```

figure 27: ROC scores for both models

Confusion Matrix Analysis

In [578…

```
from sklearn.metrics import ConfusionMatrixDisplay
logclf_cm_disp = ConfusionMatrixDisplay.from_estimator(logclf_top_perf, X_test, y_test, display_labels=['No Tsunami', 'Tsunami'], cmap='Blues')
logclf_cm_disp.ax_.set_title('Logistic Regression Confusion Matrix')
plt.show()
```

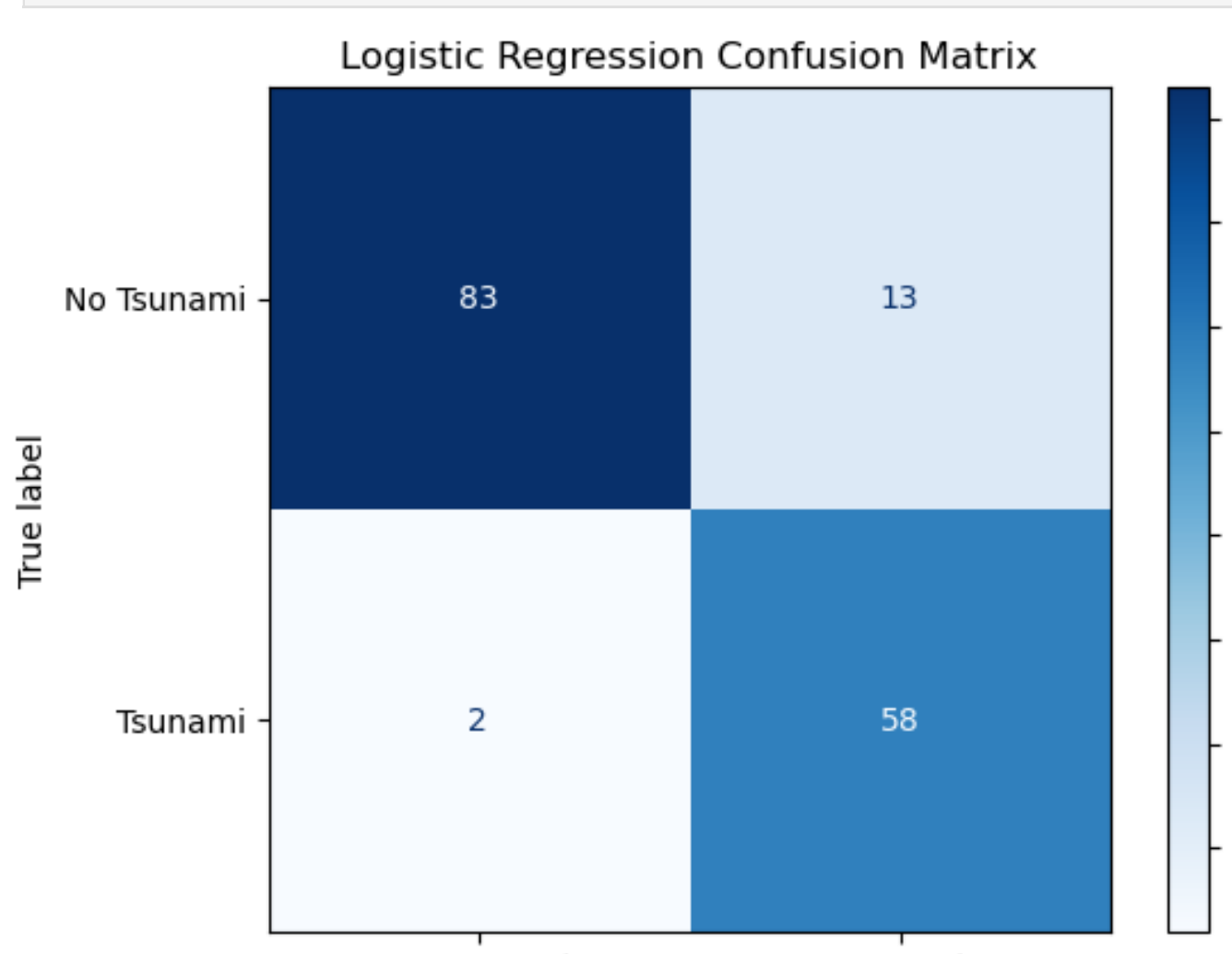


Figure 28: Logistic Regression Confusion Matrices

In [579…

```
gbc_cm_disp = ConfusionMatrixDisplay.from_estimator(gbc_top_perf, X_test, y_test, display_labels=['No Tsunami', 'Tsunami'], cmap='Blues')
gbc_cm_disp.ax_.set_title('Gradient Boosting Classifier Confusion Matrix')
plt.show()
```

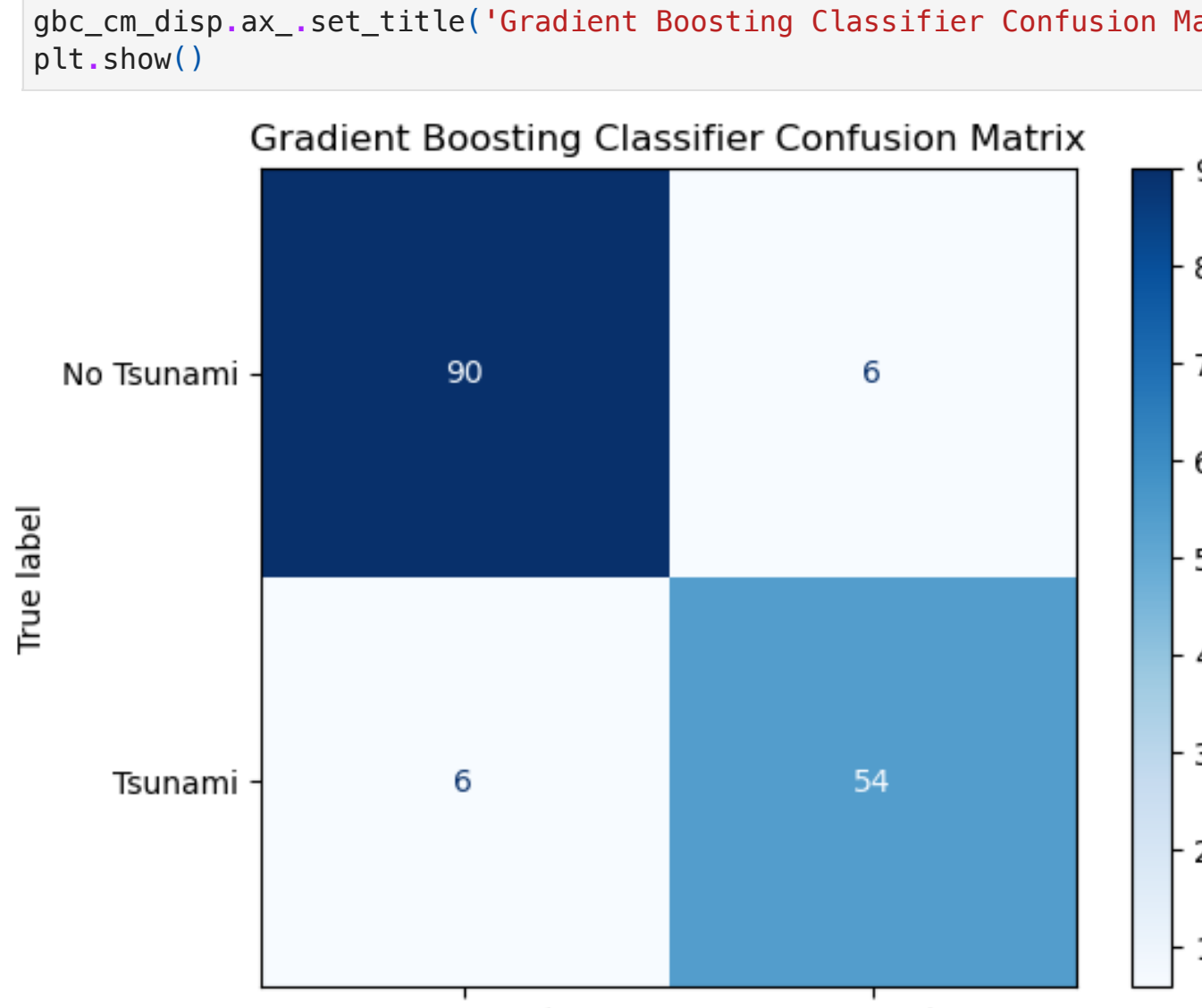


figure 29: Confusion Matrices for Gradient Boosting Classifier

Statistical Significance Testing

In [ ]:

```
from scipy.stats import ttest_rel
t_stat, p_value = ttest_rel(grid_searched_logclf_cross_val_scores, grid_searched_gbc_cross_val_scores)
print(f'T-statistic: {t_stat}, P-value: {p_value}')
if p_value < 0.05:
    print('The difference in model performance is statistically significant.')
else:
    print('The difference in model performance is not statistically significant.')
```

```
T-statistic: -2.2774092319489765, P-value: 0.04876866577246905
The difference in model performance is statistically significant.
```

Figure 30: T-Test Results Comparing Model CV Accuracies

Model comparison summary

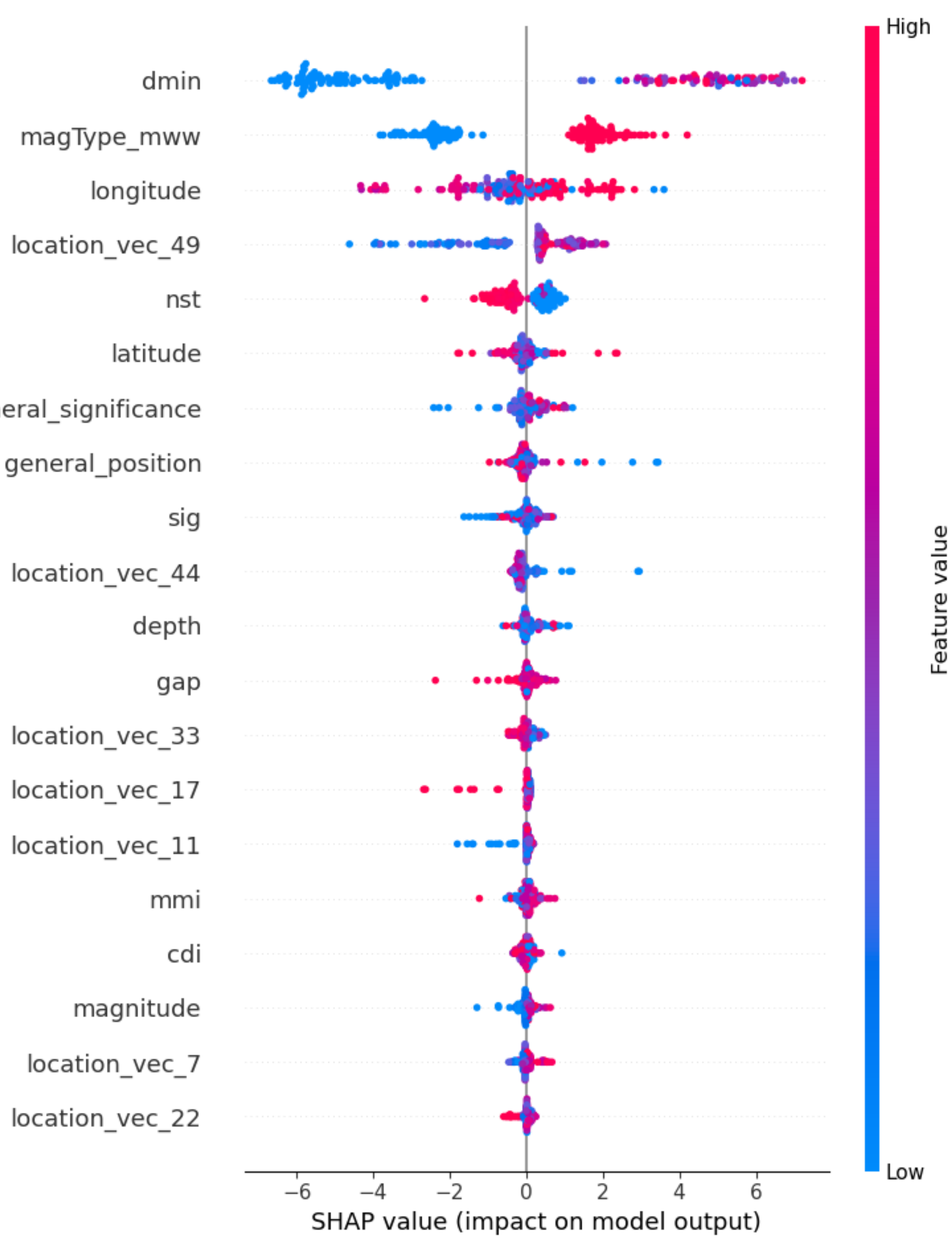
The results inform us that both models performed well on there training data, before and after hyperparameter tuning, achieving over 85% accuracy. Demostrating that both models understanding the underlying patterns of the data. The higher training score, 100%, of the gradient boosting classifier indicating it has a more developed understanding of the training data compared to logistic regression. It continued to marginally outperform logistic regression on test and validation data (figure 21) indicating its superior generalization capabilities. Yet the large gap between training and validation scores (figure 22) indicates greater overfitting compared to logistic regression.

Both models achieved similar precision, recall, and F1-scores (figures 24 and 25), indicating they are comparably effective at identifying tsunami events. Precsion was slightly higher for the gradient boosting classifier, showing it makes fewer false positive predictions. However, recall was marginally better for logistic regression, indicating it captures more true tsunami events. The higher F1-score of the gradient boosting classifier informs us a better balance between precision and recall overall, something that is vital for effective tsunami prediction to minimize both false alarms and missed events. Although minimally, the higher AUC of the gradient boosting classifier (figure 27) further supports its superior overall performance in distinguishing between tsunami and non-tsunami events across various classification thresholds.

The confusion matrices (figures 28 and 29) reveal that gradient boosting classifier makes fewer misclassifications overall compared to logistic regression, though it had slightly more false negatives, confirming the precision and recall results observed earlier. Our statistical significance testing on cross validation accuracies (figure 30), showed the gradient boosting classifier's accuracy advantage was statistically significant. This acknowledges a real performance difference between the models, this is important given marginal differences between the models accross most performance metrics.

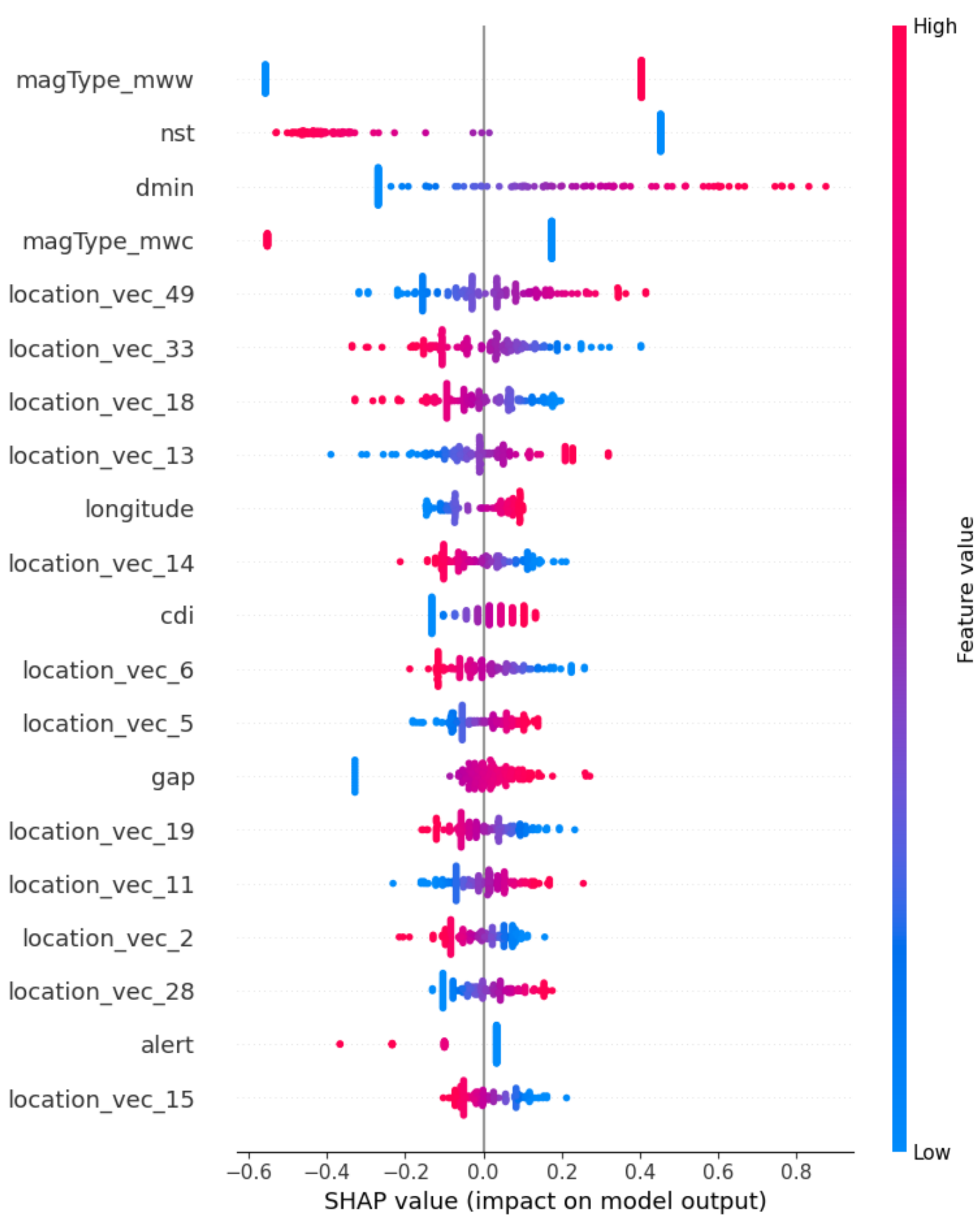
In [581…

```
explainer_gbc = shap.TreeExplainer(gbc_top_perf)
shap_values_gbc = explainer_gbc.shap_values(X_test)
shap.summary_plot(shap_values_gbc, X_test, plot_type='dot')
```



In [582…

```
explainer = shap.Explainer(logclf_top_perf, X_test)
shap_values = explainer(X_test)
shap.summary_plot(shap_values.values, X_test)
```



Research questions

`Will a gradient decsion boosting classifier outperform logistic regression in predicting tsunami occurrence following earthquakes based on seismic event characteristics?`

Based on our analysis, the gradient boosting classifier modestly outperformed logistic regression acrosss a majority of evaluation metrics, including accuracy, precision, F1-score, and ROC AUC scores. Logistic regression achieved a slightly higher recall score, indicating it may be better at identifying actual tsunami events. but the higher F1 score of the gradient boosting classifier suggests it is better overall at balancing false positives and false negatives. These finding are verified in statisical significance testing, which confirmed that the performance difference between the two models is statistically significant in favour of the gradient boosting classifier. Therefore, I conculude that the gradient boosting classifier is the better model for predicting tsunami occurrence following earthquakes based on seismic event characteristics.

`Which feature will be the most important predictor of tsunami occurrence following earthquakes?`

The SHAP summary plots inform us that `dmin` (horizontal distance from the epicenter to the nearest station) is the most important predictor of tsunami occurrence following earthquakes while `magtype` (total number of seismic stations used to determine earthquake location) is of highest important to the Logistic Regression model, the Gradient Boosting classifier achieved greater overall performance, so I conclude that `dmin` is the most important predictor overall. Although this may appear unconventional, the literature notes that the majority of seismic stations are located on land, meaning that earthquakes occurring further from land (higher `dmin` values) are more likely to be undersea events capable of generating tsunamis.

(Marra et al., 2017; United States Geological Survey, 2025)

The location text vectors also ranked highly in feature importance for both models, indicating that geographic patterns and seismic hotspots play a significant role in tsunami prediction. However, `dmin` has a more direct relationship with tsunami occurrence, making it the most important predictor overall.

Novelty

The primary novelty of this project lies in incorporating textual location data through Word2Vec embeddings, distinguishing it from conventional tsunami prediction models that rely heavily on numerical seismic parameters (Mulia et al., 2022). While traditional approaches use only values such as: coordinates, magnitude, and depth, I extracted semantic patterns from unstructured earthquake location descriptions using pre-trained embeddings. To manage dimensionality constraints given our small dataset, I averaged word vectors rather than concatenating them, producing fixed 50 representations that preserve semantic meaning without massive feature expansion. This text based feature encoding complements our unqiue imputation strategies, all of which helped achieve a highly performant model despite the limited dataset size.

Reflection

This project enhanced my understanding of machine learning through practical challenges I encountered. My model comparison revealed through various metrics that the performance tradeoff where logistic regression's simplicity versus gradient boostings complexity showed algorithmic sophistication doesn't guarantee proportional gains.

A significant challenge I faced was addressing extreme outliers through logarithmic transformations, new to my experience. I was aware that outliers could skew model training and degrade performance, so I applied the log1p transformation to normalize skewed distributions while preserving zero values. SHAP analysis revealed models prioritized instrumental factors (`dmin`) over traditional seismic characteristics, challenging my initial assumptions and emphasizing data driven feature importance. Through systematic hyperparameter optimization I reinforced iterative model development, with cross validation proving essential for robust performance estimation. This experience shifted my approach from algorithm centered implementation toward comprehensive problem solving encompassing data quality, feature engineering, and model interpretability.

References:

Chauhan, C. (2025) Earthquake dataset. Kaggle. Available from: https://www.kaggle.com/datasets/warcoder/earthquake-dataset?select=earthquake_data.csv [Accessed 24 October 2025].

United States Geological Survey (USGS) (no date) Earthquake Hazards Program Glossary. U.S. Geological Survey. Available from: https://www.usgs.gov/glossary/earthquake-hazards-program [Accessed 26 October 2025].

CDI Wald, D.J., Quitoriano, V., Worden, C.B., Hopper, M. and Dewey, J.W. (2011) 'USGS "Did You Feel It?": Internet-based macroseismic intensity maps', Annals of Geophysics, 54(6), pp. 688–707. Available from: https://www.researchgate.net/publication/268368064_USGS_Did_You_Feel_It_Internet-based_macroseismic_intensity_maps [Accessed 26 October 2025].

United States Geological Survey (USGS) (1989) Modified Mercalli Intensity Scale. U.S. Geological Survey. Available from: https://www.usgs.gov/programs/earthquake-hazards/modified-mercalli-intensity-scale [Accessed 26 October 2025].

United States Geological Survey (USGS) (2010) PAGER Scientific Background. U.S. Geological Survey. Available from: https://earthquake.usgs.gov/data/pager/background.php [Accessed 26 October 2025].

itdxer (2017) 'What does average of word2vec vector mean?', Cross Validated – Statistics Stack Exchange, 14 December. Available from: https://stats.stackexchange.com/questions/318882/what-does-average-of-word2vec-vector-mean%5BAccessed 3 November 2025].

United States Geological Survey (2022) Why is the earthquake that was reported/recorded by network X, or that I felt, not on the Latest Earthquakes map/list? Available from: https://www.usgs.gov/faqs/why-earthquake-was-reportedrecorded-network-x-or-i-felt-not-latest-earthquakes-maplist [Accessed 3 November 2025]

AlwaysSunny, 2019. pandas convert from datetime to integer timestamp Stack Overflow. Available at: https://stackoverflow.com/questions/54312802/pandas-convert-from-datetime-to-integer-timestamp [Accessed 4 Nov. 2025].

GeeksforGeeks, 2025. Feature Importance with Random Forests Available at: https://www.geeksforgeeks.org/machine-learning/feature-importance-with-random-forests [Accessed 4 November 2025].

ChatGPT, 2025. General multi-use session. Available at: https://chatgpt.com/c/6909a310-4050-832b-94b1-35a82b81cec3 [Accessed 7-15th November 2025].

Blagus, R. & Lusa, L., 2013. SMOTE for high-dimensional class-imbalanced data. BMC Bioinformatics, 14, p.106. Available at: https://bmcbioinformatics.biomedcentral.com/articles/10.1186/1471-2105-14-106 [Accessed 5 November 2025].

Huang, X., Khetan, A., Cvitkovic, M., & Karnin, Z. (2020). TabTransformer: Tabular Data Modeling Using Contextual Embeddings. Available at: https://doi.org/10.48550/arXiv.2012.06678 (Accessed: 13 November 2025).

Lomax, A., Michelini, A. and Piatanesi, A., 2011. An assessment of rapid magnitude and tsunami potential estimation for shallow earthquakes. Available at: https://academic.oup.com/gji/article/185/1/283/602131 (Accessed: 13 November 2025).

United States Geological Survey (USGS) 2023, Tsunamis and tsunami-hazards, U.S. Geological Survey https://www.usgs.gov/water-science-school/science/tsunamis-and-tsunami-hazards. (Accessed: 13 November 2025).

Google LLC (2025) Class-imbalanced datasets. Google Developers, viewed 13 November 2025, https://developers.google.com/machine-learning/crash-course/overfitting/imbalanced-datasets. (Accessed: 13 November 2025).

Wainwright, M. J., Ravikumar, P. & Lafferty, J. D. (2006) High-Dimensional Graphical Model Selection Using Regularized Logistic Regression. Neural Information Processing Systems Foundation. Available at: https://proceedings.neurips.cc/paper_files/paper/2006/file/86b20716fbd5b253d27cec43127089bc-Paper.pdf (Accessed: 13 November 2025).

Ayer, T., Chhatwal, J., Alagoz, O., Kahn, C. E. Jr., Woods, R. W. & Burnside, E. S. (2010) 'Informatics in Radiology: Comparison of Logistic Regression and Artificial Neural Network Models in Breast Cancer Risk Estimation'. Available at: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3709515 (Accessed: 14 November 2025).

Lee, F. (2025) What is logistic regression? IBM. Available at: https://www.ibm.com/think/topics/logistic-regression (Accessed: 13 November 2025).

Friedman, J.H., 1999. Greedy function approximation: A gradient boosting machine. Available at: https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full (Accessed: 14 November 2025).

Natekin, A. & Knoll, A., 2013. Gradient boosting machines, a tutorial. Frontiers in Neurorobotics. Available at: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3885826/ [Accessed 13 November 2025].

Luo, J., Yuan, Y. & Xu, S., 2024. Improving GBDT Performance on Imbalanced Datasets: An Empirical Study of Class-Balanced Loss Functions. Available at: https://doi.org/10.48550/arXiv.2407.14381 (Accessed: 13 November 2025).

Peter, S., Diego, F., Hamprecht, F.A. & Nadler, B., 2017. Cost efficient gradient boosting. Available at: https://papers.nips.cc/paper/6753-cost-efficient-gradient-boosting.pdf (Accessed: 13 November 2025).

Matplotlib (no date) Matplotlib. Available from: https://matplotlib.org/stable/gallery/pie_and_polar_charts/pie_features.html [Accessed 3 November 2025].

Marra, G. et al. (2017) 'Seismology with optical links: enabling a global network for submarine earthquake monitoring'. Available at: https://arxiv.org/abs/1801.02698 (Accessed: 26 November 2025).

United States Geological Survey (USGS) (2025) What is it about an earthquake that causes a tsunami? Available at: https://www.usgs.gov/faqs/what-it-about-earthquake-causes-a-tsunami (Accessed: 26 November 2025).

Mulia, I.E. et al., 2022. Machine learning-based tsunami inundation prediction derived from offshore observations. Nature Communications, 13, p.5489. Available at: https://www.nature.com/articles/s41467-022-33253-5 [Accessed 5 November 2025].

Aburass, S. (2023) Quantifying Overfitting: Introducing the Overfitting Index. arXiv. Available from: https://arxiv.org/abs/2308.08682 (Accessed: 28 November 2025).

Appendix

AI Usage Disclosure

All code generated by AI was carefully checked using my own knowledge of programming from a career in software development as well as cross referncing with online searches and documentation to ensure accuracy and appropriateness for the task at hand.

All AI generated code is marked with a reference to (ChatGPT, 2025)

Queries used:

- "Write a Python snippet using pandas and numpy that applies a log transformation to a subset of columns in a dataframe. The code should select a group of columns and apply np.log1p to each value in those columns in a vectorized way"
- "make a bit of python using pandas and matplotlib that takes a dataset of features and a separate dataset of feature importances selects a group of features based on a common naming pattern calculates the total importance of that feature group takes the importance of one individual feature and visualizes the comparison with a bar chart"
- "use matplotlib that adjusts the y-axis limits of a matplotlib plot to zoom in on a specific range so that small differences are more visible" templated code reused in different places
- 'Given python matplotlib code X highlight the highest bar in the chart' - templated code reused in different places'
- "python code using matplotlib that creates a grouped bar chart comparing two sets of numeric values the code should plot the bars side by side with different colors and labels, and annotate each bar with its exact height value on top" templated code reused in different places

All of these queries produced code snippets that were then adapted significantly and integrated into the project as appropriate.

I also used AI to format my references correctly from a rough list of sources into proper Harvard referencing style.

Multi use chatgpt session used (ChatGPT, 2025) in order to store keep track of all queries and responses - https://chatgpt.com/c/6909a310-4050-832b-94b1-35a82b81cec3